Module 14: Web Services

Lab 14-1: Transforming Billing Data with Web Services: Number-to-Words Conversion

Overview:

A web service allows automated communication between machines over a network, ensuring applications are independent of specific platforms and technologies.

In this lab, you will develop a business service and implement a mapping that utilizes this service via a web service transformation to solve a business need.

The goal is to update the billing format for NH Retail outlets by converting numerical price information into written words. This transformation can be achieved by leveraging a business service designed for number-to-dollars conversion.

Objectives:

- Create a business service that interfaces with the **NumberToDollars** web service.
- Create a mapping that uses this business service to transform numerical input into word format.
- Output the data and the converted number to a flat file.

Duration:

25 minutes

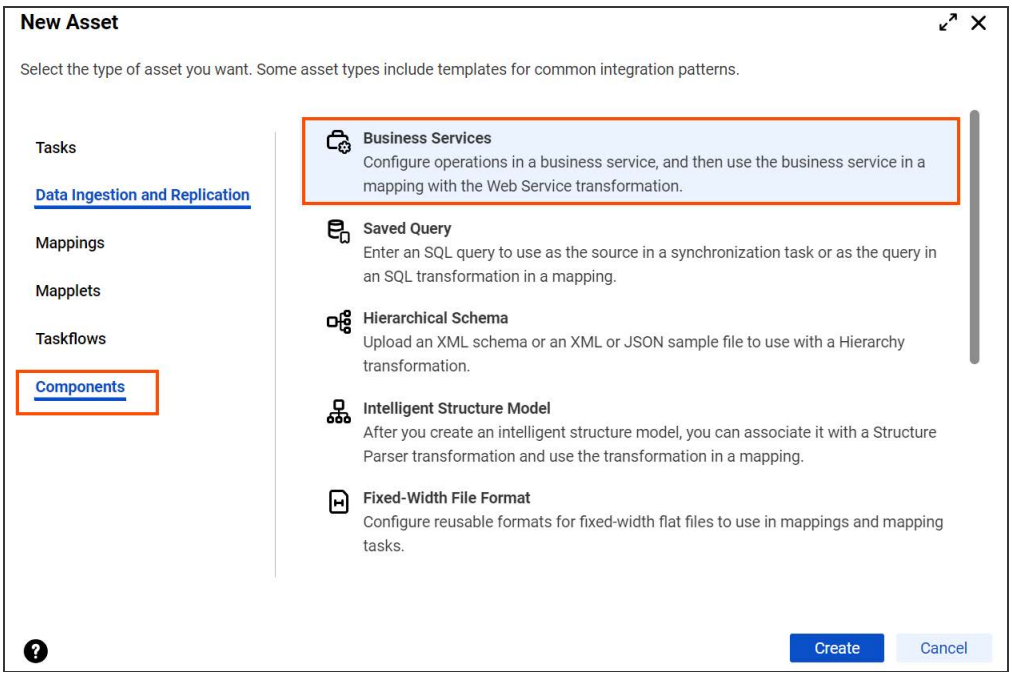
Tasks:

Create a Business Service

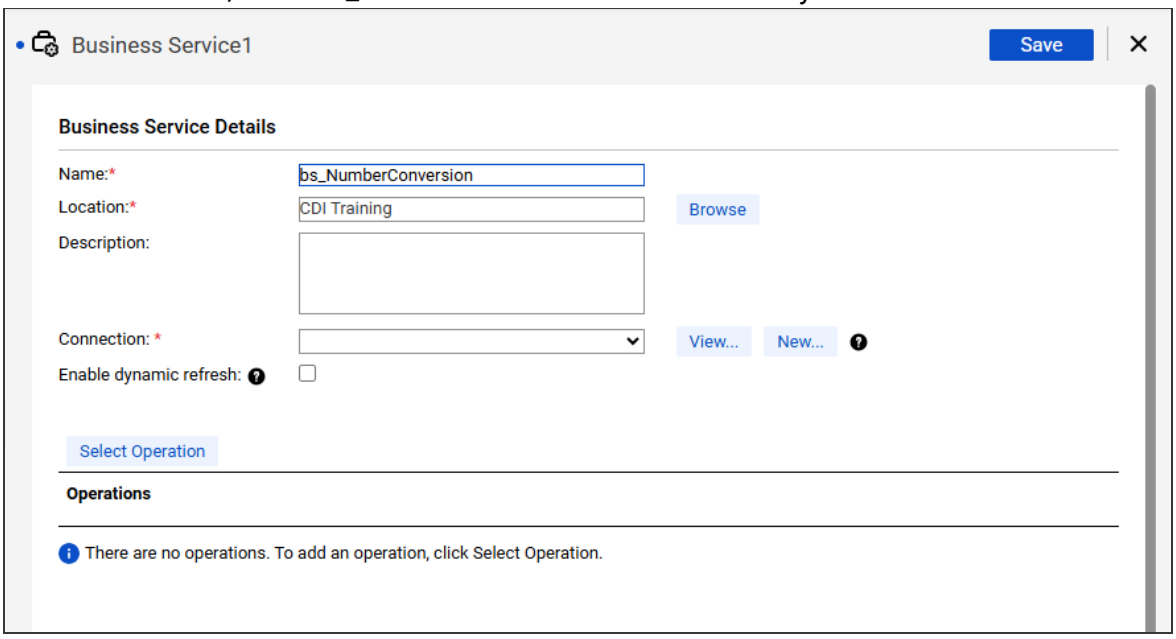
A Web service transformation requires a preconfigured business service to work so before building a mapping with a webservice, we must first define the business service along with the web service consumer connection.

1. In the **Data Integration** service, click **New**.

2. From the New Asset window, click the **Components** tab and select **Business Services** and click **Create**.



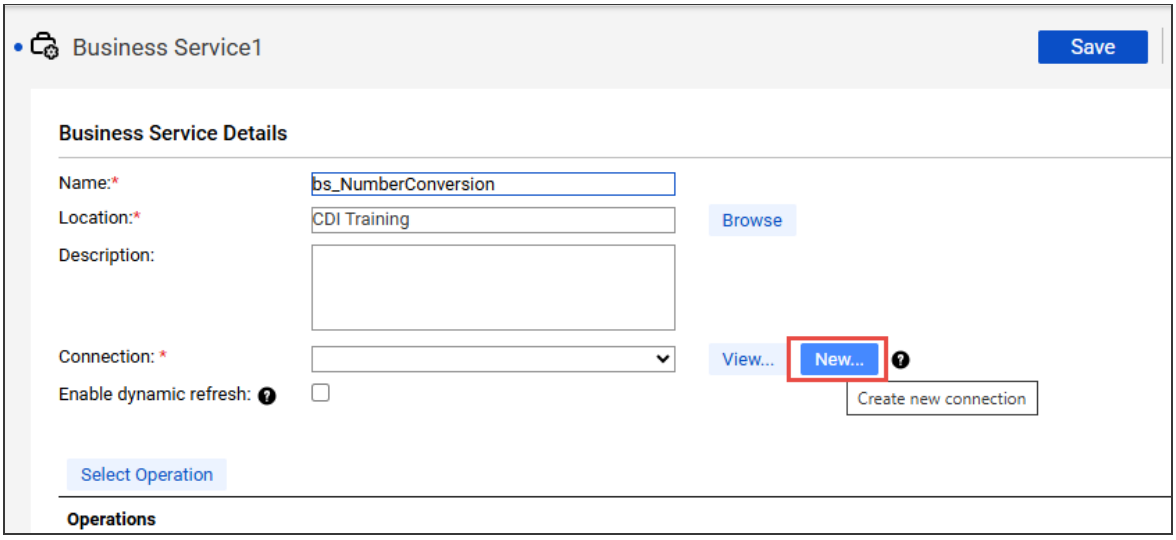
3. In the Name field, enter **bs_NumberConversion** and save it in your folder.



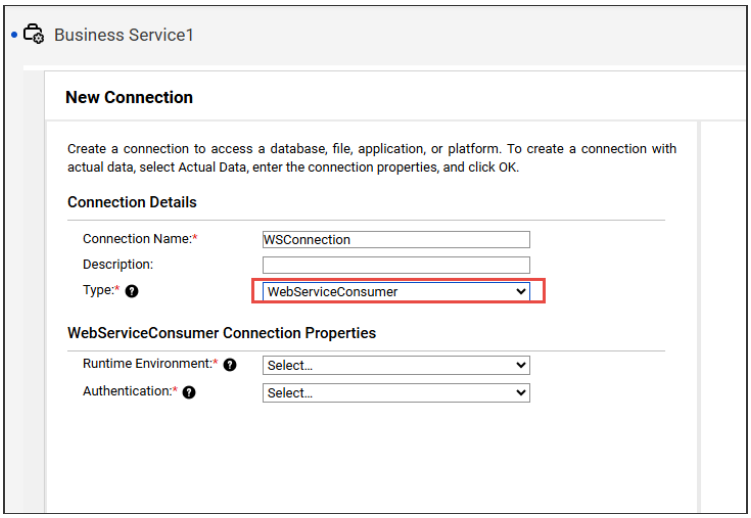
Define a Web Service Consumer Connection

The business service requires a connection. This can be defined in the Administrator ahead of creating the business service, or as part of the business service configuration.

4. To create the web service consumer connection, click **New**. We can define the connection here.



5. In the Name field, enter **WSConnection**, and from the Type drop-down, select **WebServiceConsumer**.

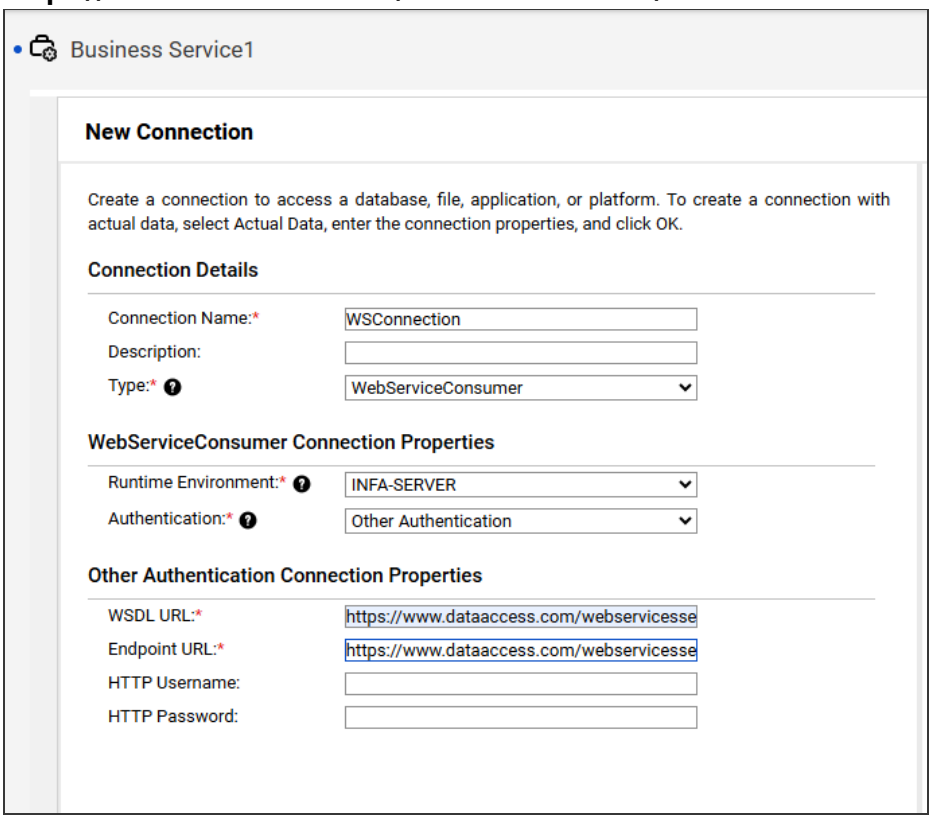


6. From the Runtime Environment drop-down, select INFA-SERVER.

7. From the Authentication drop-down, select **Other Authentication**.

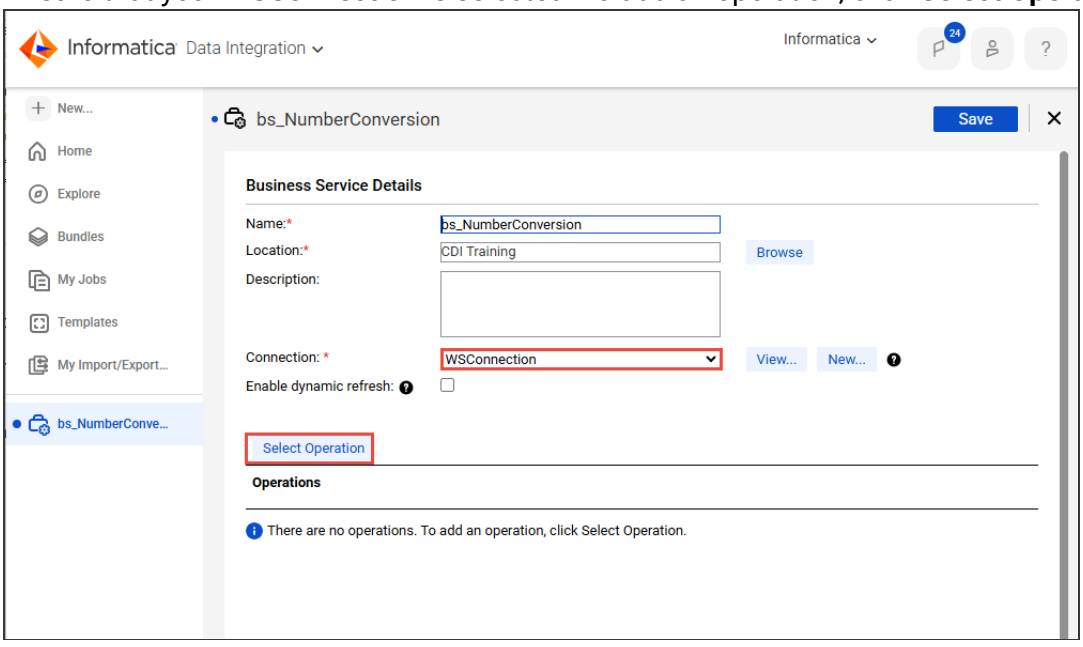
8. In the WSDL URL field, enter the following URL:
   - **https://www.dataaccess.com/webservicesserver/numberconversion.wso?WSDL**

9. In the Endpoint URL field, enter the following URL:
   - **https://www.dataaccess.com/webservicesserver/numberconversion.wso**

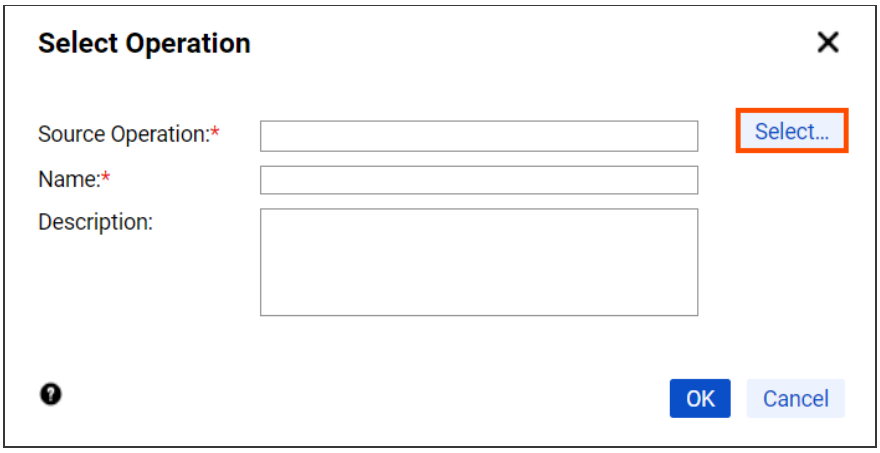


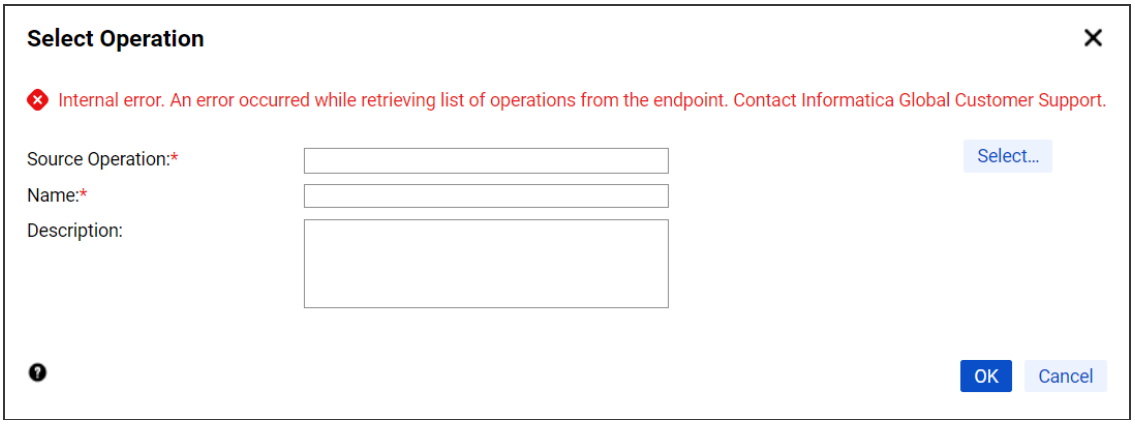
10. Click **OK**.

11. Ensure that your **WSConnection** is selected. To add an operation, click **Select Operation**.

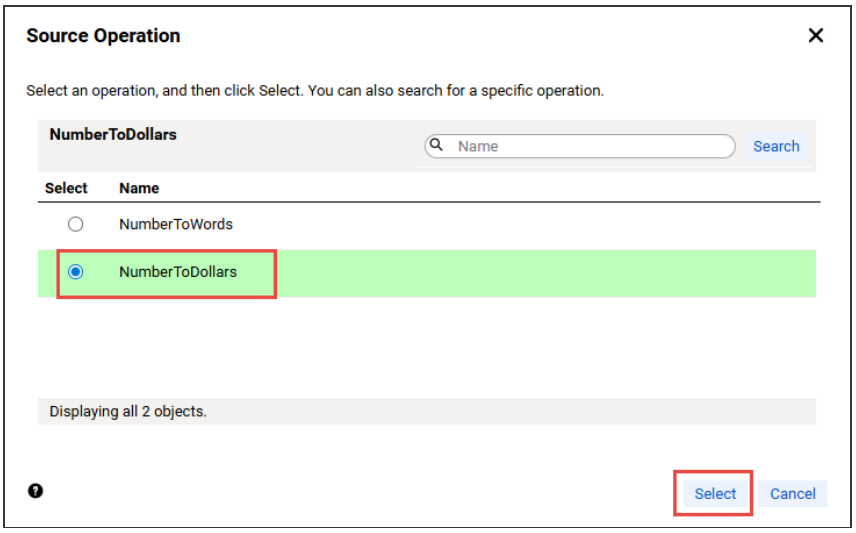


12. To select the source operation, click **Select**. If you get the Internal error message as shown in the screenshot below, click **Cancel** and click **Select** again from the Source Operation field.

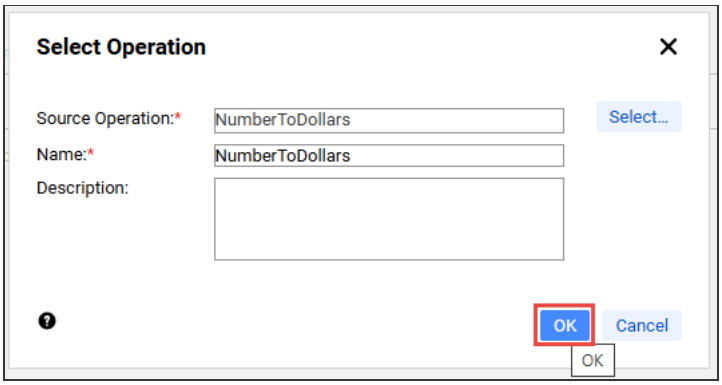




13. From the list, select **NumberToDollars**. We want the numeric values in our file converted to words in the dollar format.



14. Click **Select**.

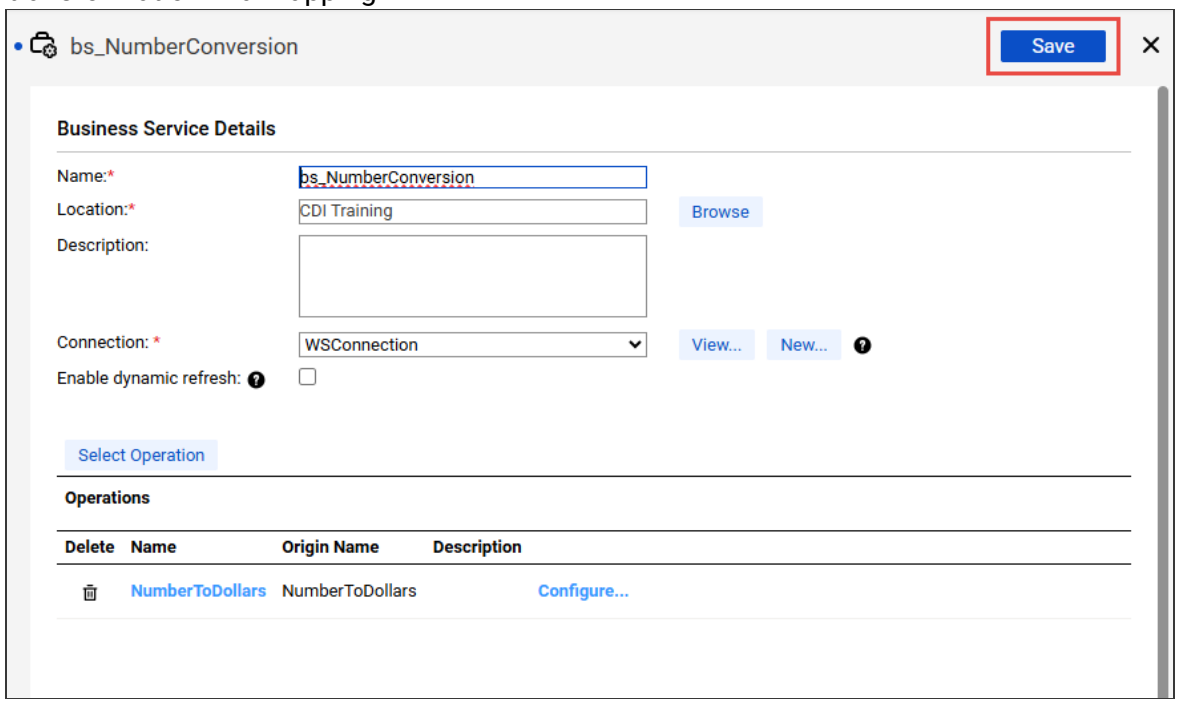


15. Click **OK**.

16. Click **Configure** to set the request, response, and fault parameters.

17. View all the parameters in the Request, Response, and Fault tabs. We will send a numerical value, and the response will be a number to dollars result. Should any error occur, we will get fault information such as the fault code and string. Click **OK**.

18. **Save** and close the business service. It is now ready for use in a web service transformation in a mapping.



Create a Mapping Containing a Web Service Transformation to Call the Defined Business Service

19. Click **New** and choose **Mappings > Mapping** and press **Create**.

20. In the Name field, enter **m_WS_NumberToDollars** and **save** it into your folder.

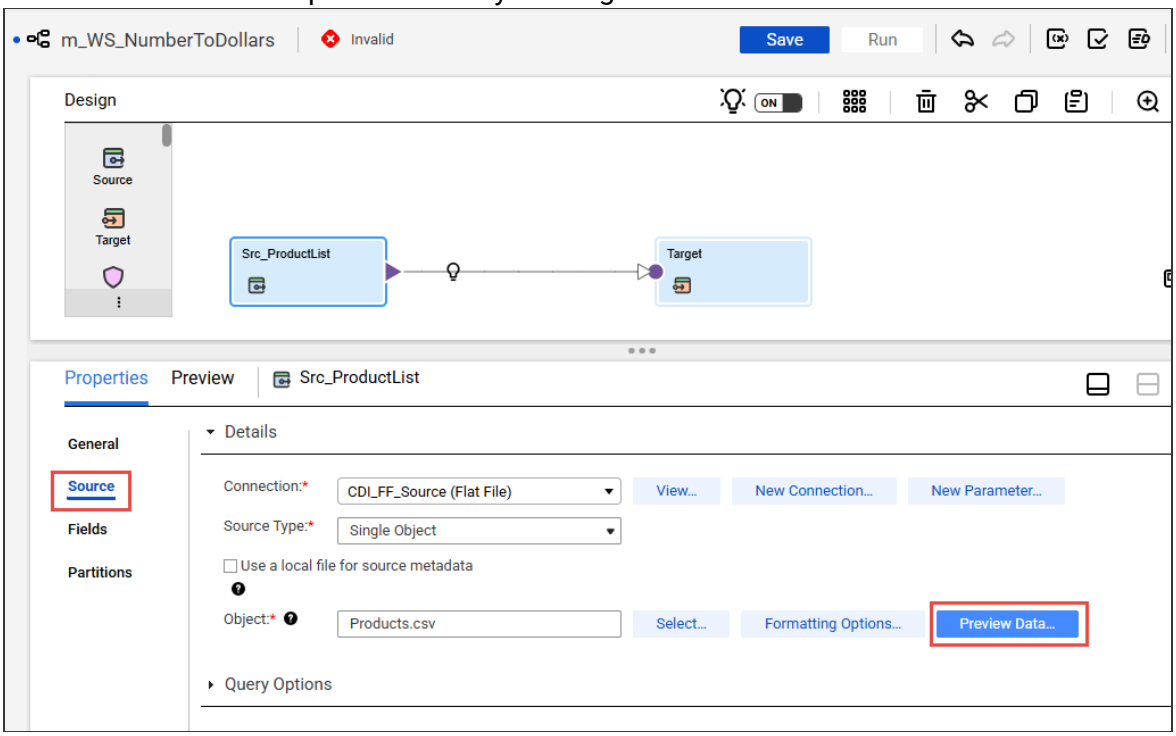
21. To configure the source, on the mapping canvas, click the **Source** transformation and set the name as **Src_ProductList**.

22. From the properties pane, click **Source**.

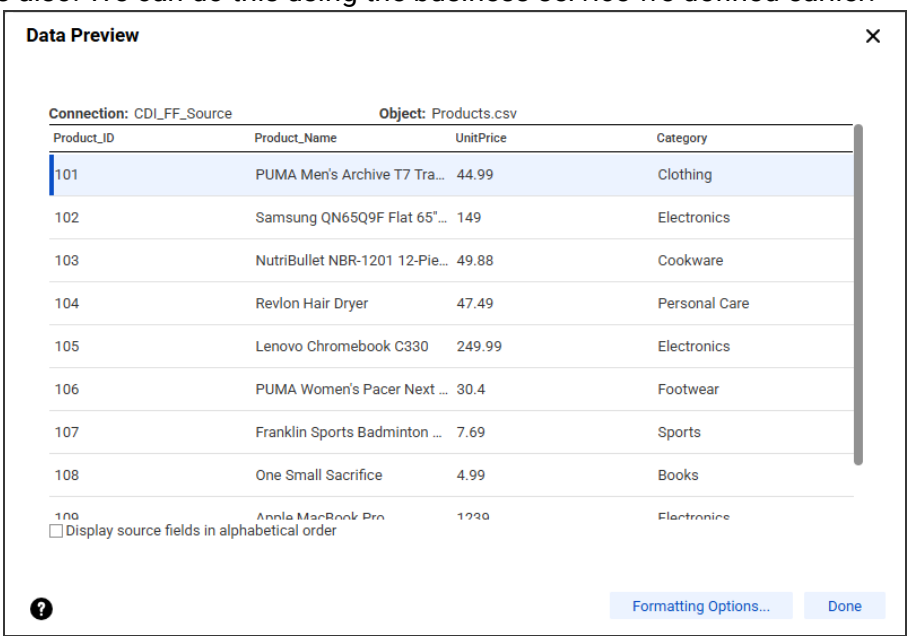
23. From the Connection drop-down, select **CDI_FF_Source**.

24. To select the source object from the Object field, click **Select** and from the list, select **Products.csv** and click **OK**.

25. Preview the data in the products file by clicking the **Preview Data** button.



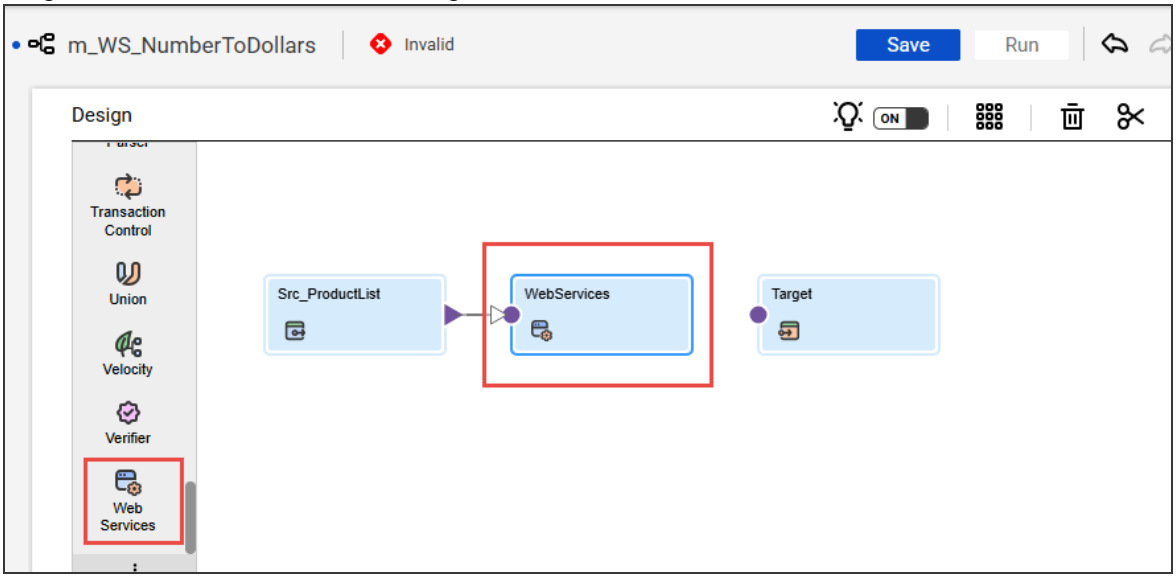
The file contains a list of company products and their cost. We would like to include the unit price in words also. We can do this using the business service we defined earlier.



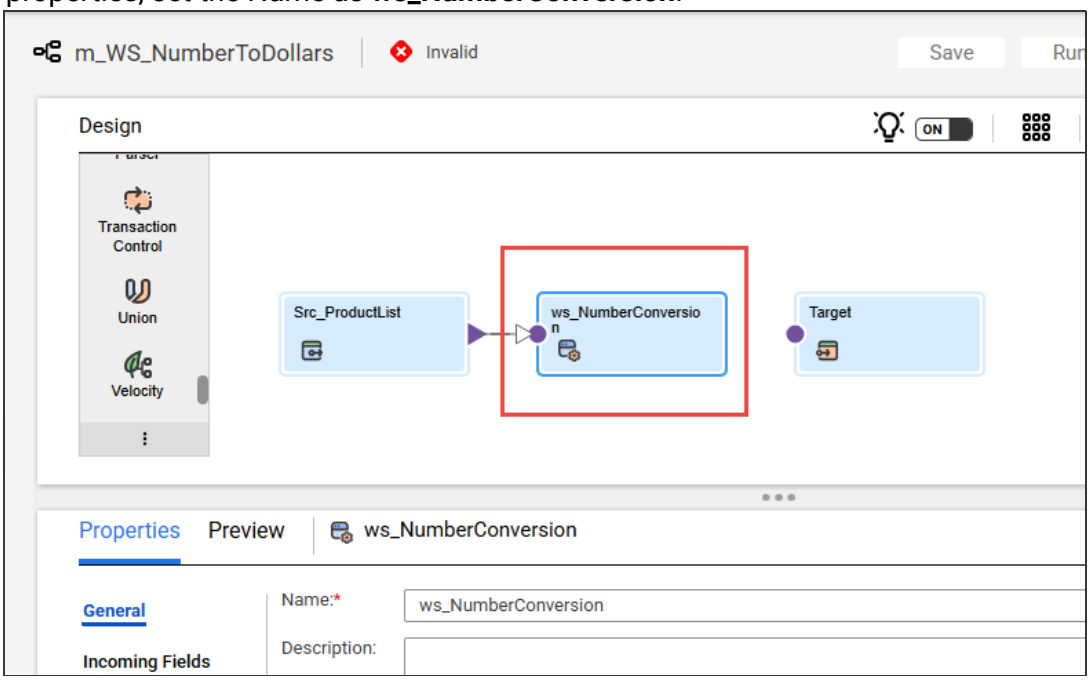
26. Click **Done** when you have reviewed the data and **save** the mapping.

Configure a Web Services Transformation to use the business service we created

27. Add a Web Services transformation to the mapping between the **Src_ProductList** and the target.The Source transformation gets linked to the Web Service transformation.



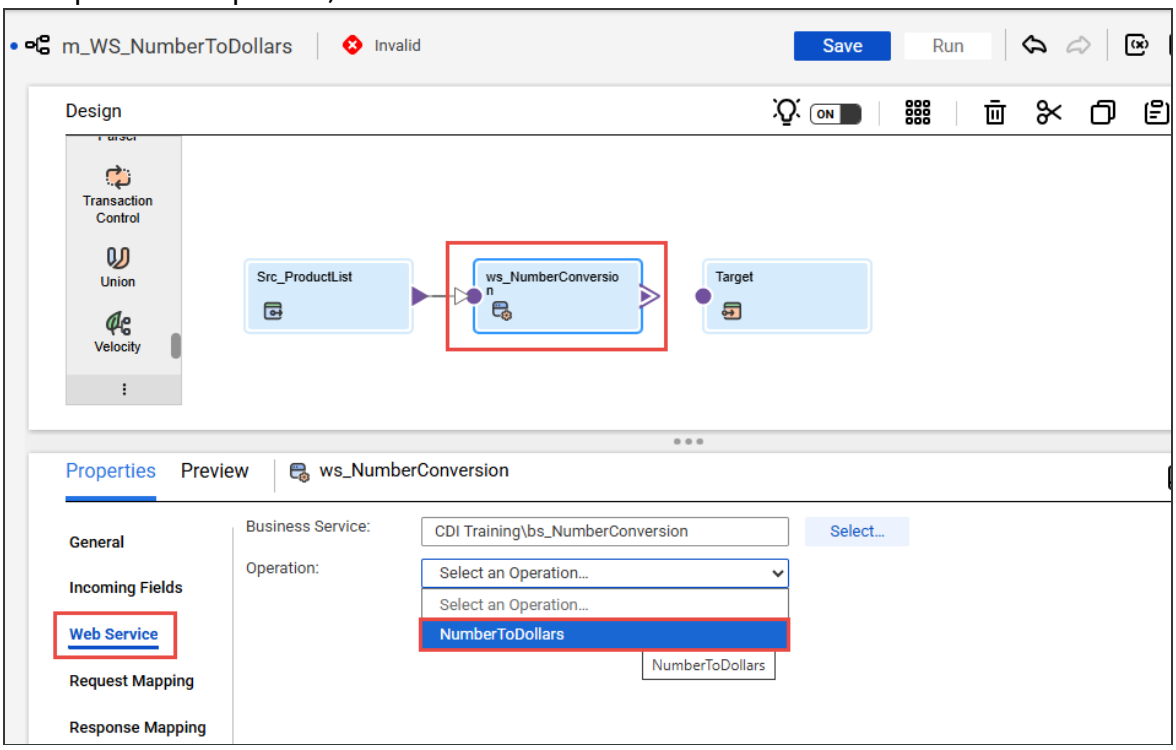
28. Select the **WebServices** transformation and in the **General** section of the WebServices properties, set the Name as **ws_NumberConversion**.



29. Move to the **Web Service** tab and click **Select** to select the business service.

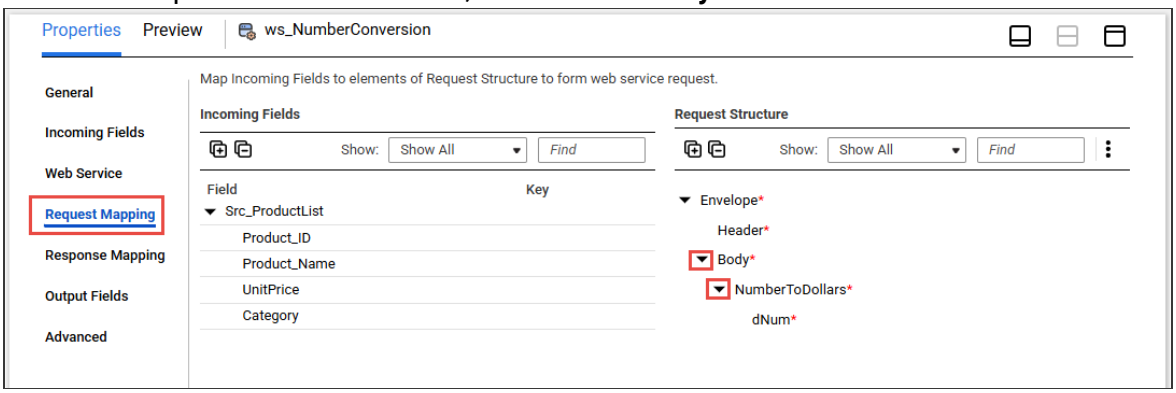
30. Navigate to your working directory and select the **bs_NumberConversion** business service.

From the Operation drop-down, select **NumberToDollars**.

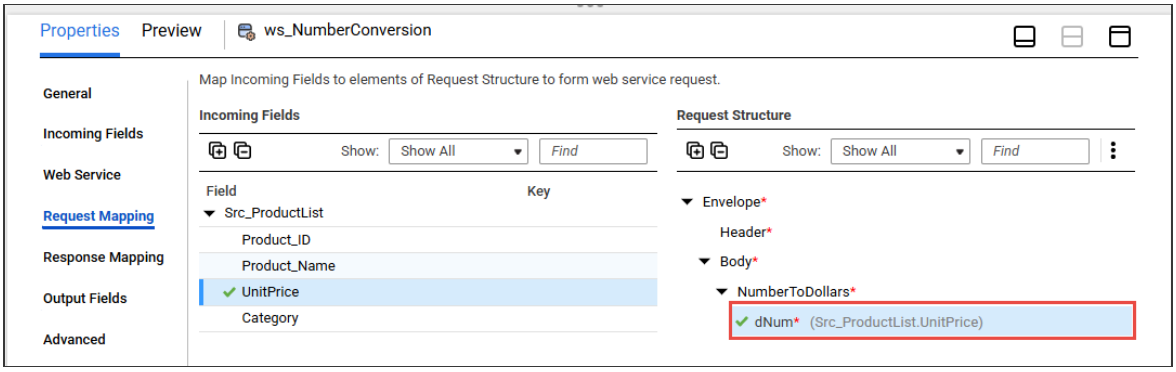


31. From the properties pane, click **Request Mapping**. The fields in the source file are listed under Incoming Fields.

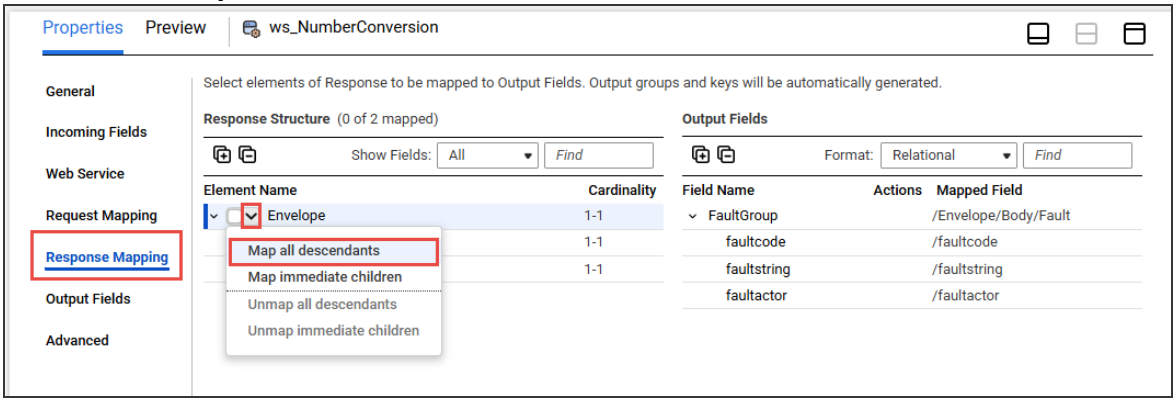
32. From the Request Structure section, drill-down to **Body** > **NumberToDollars** > **dNum**.

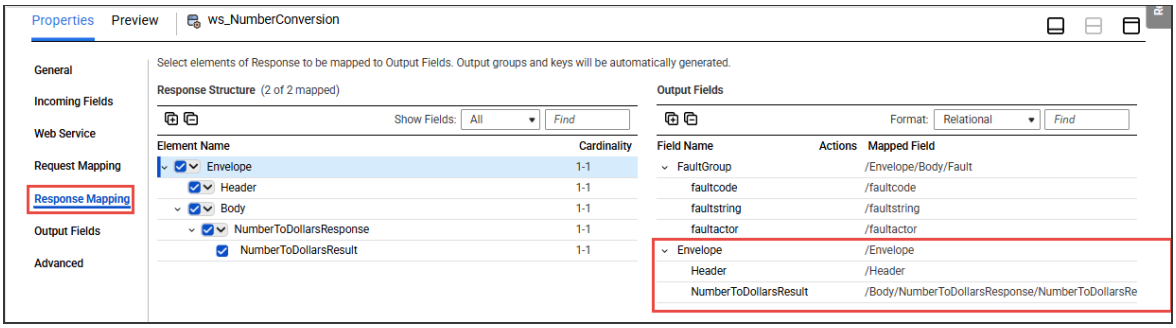


33. Drag and drop the **UnitPrice** incoming field and drop it on **dNum**.



34. From the properties pane, move to the **Response Mapping** tab. From the Envelope drop-down, select **Map all descendants**.



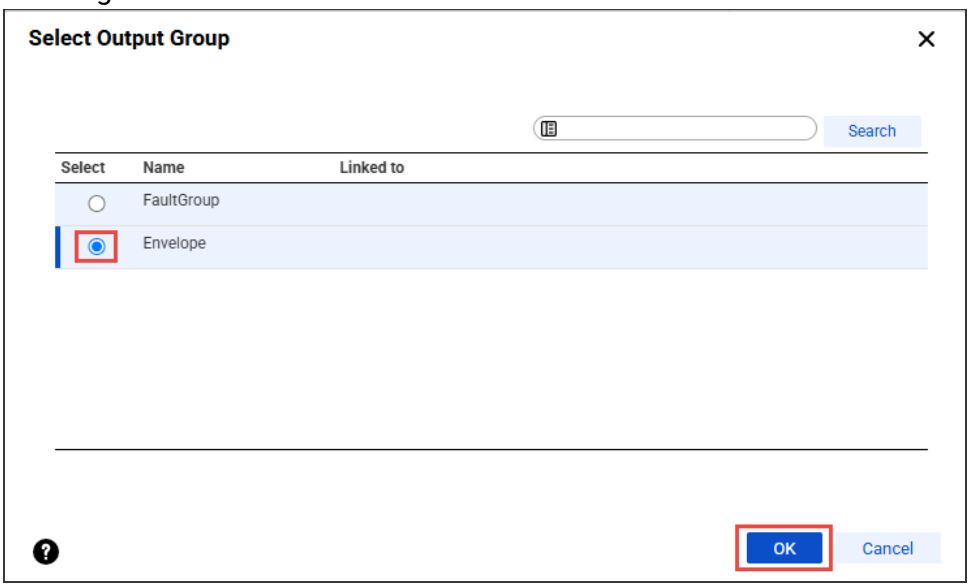


35. **Save** your changes.

Configure the Target Transformations

36. Connect the **ws_NumberConversion** transformation to the Target.

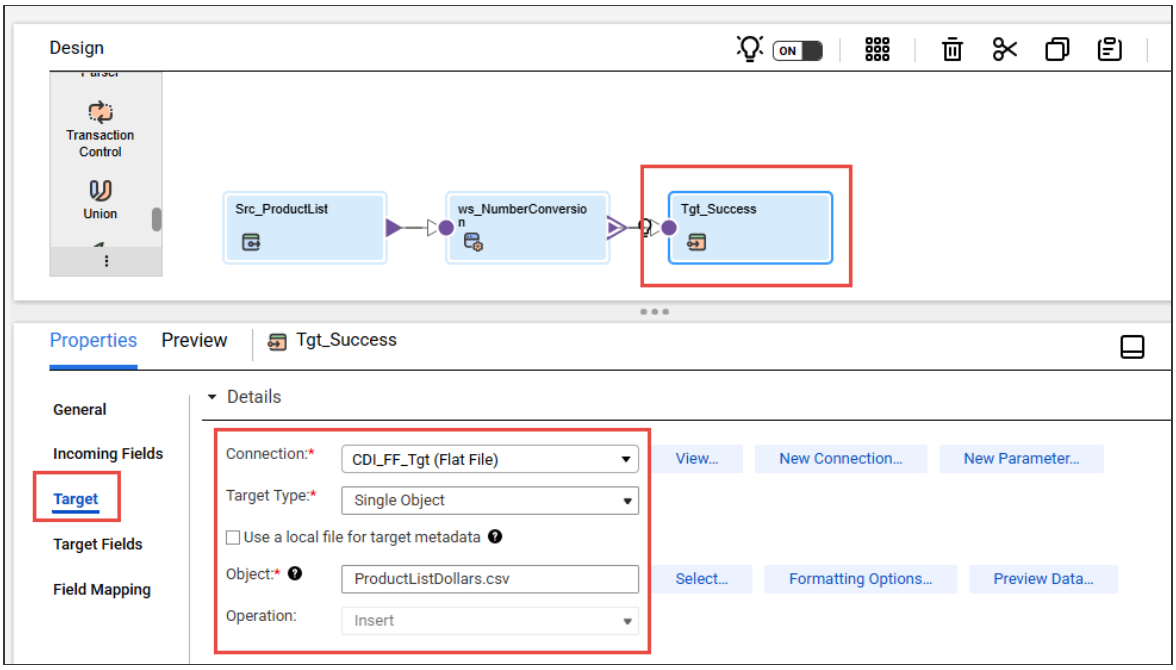
37. Select **Envelope** in the dialog. We will write out the results with the converted number to this target.



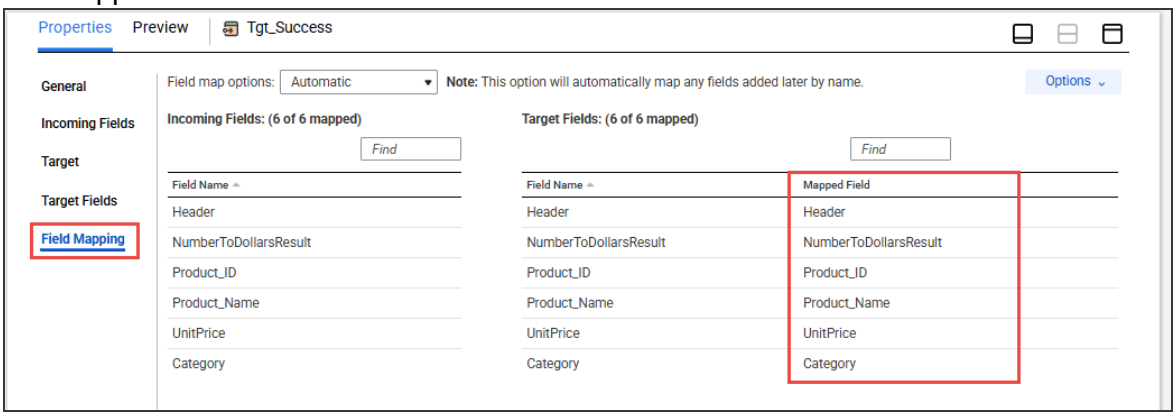
38. Select the target and set the name as **Tgt_Success.**

39. On the **Target** tab, set the Connection to your flat file target connection and click **Select** to define the target.

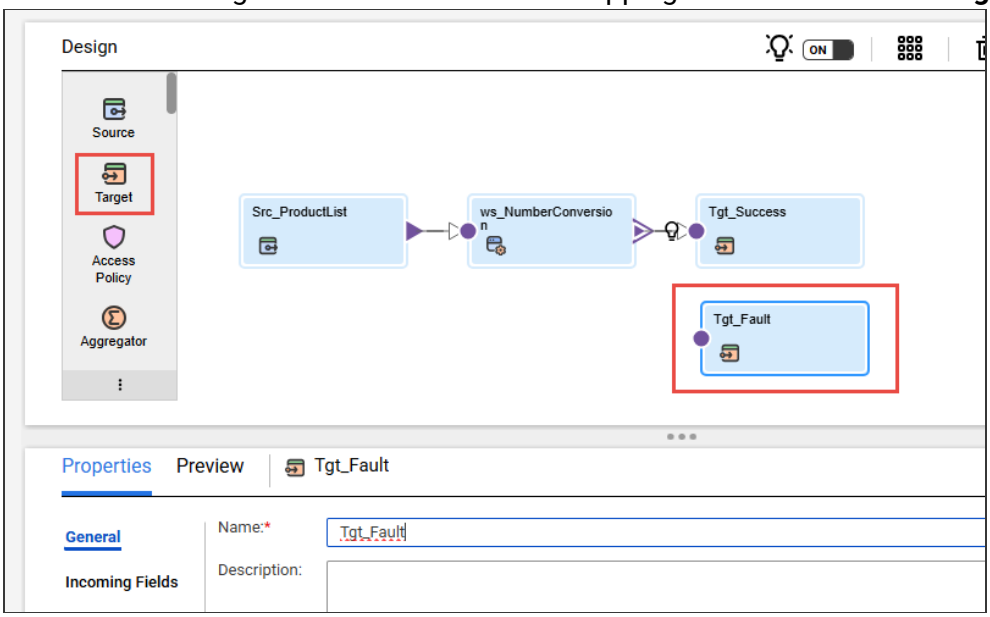
40. From the list, choose **ProductListDollars.csv**. Click **OK**



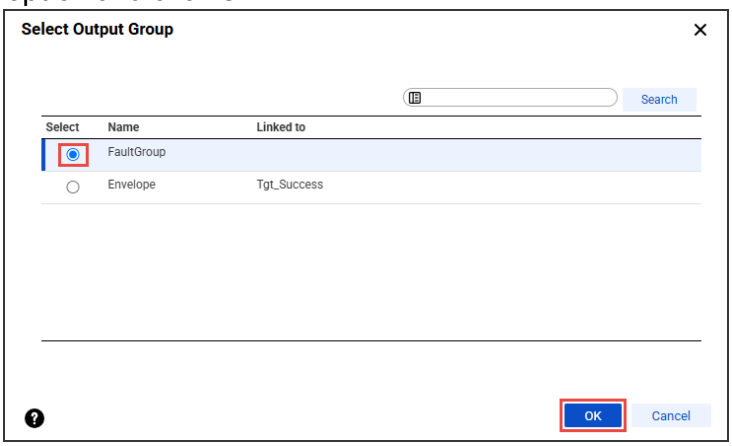
41. Go to the **Field Mapping** tab and set the Field map option to Automatic. Ensure the fields are mapped.



42. Add a second Target transformation to the mapping and set the name as **Tgt_Fault**.



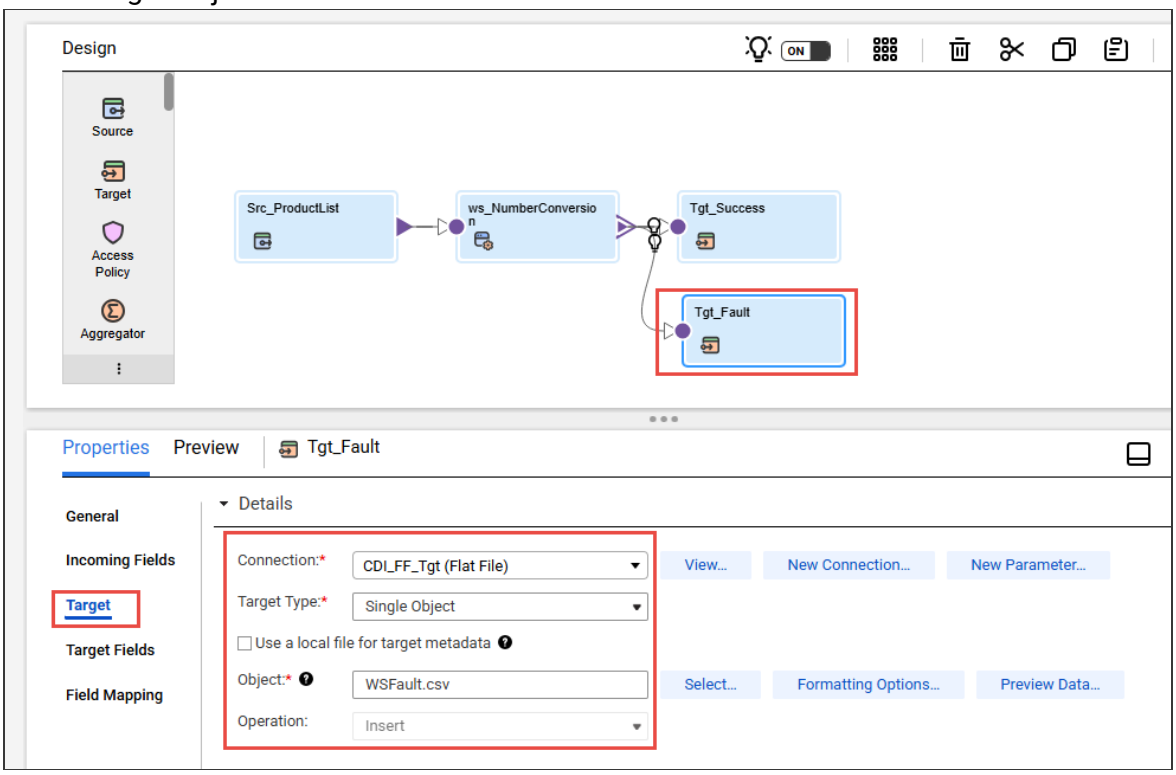
43. Link the **ws_NumberConversion** to **Tgt_Fault** and in the dialog choose the **FaultGroup** option and click **OK**.



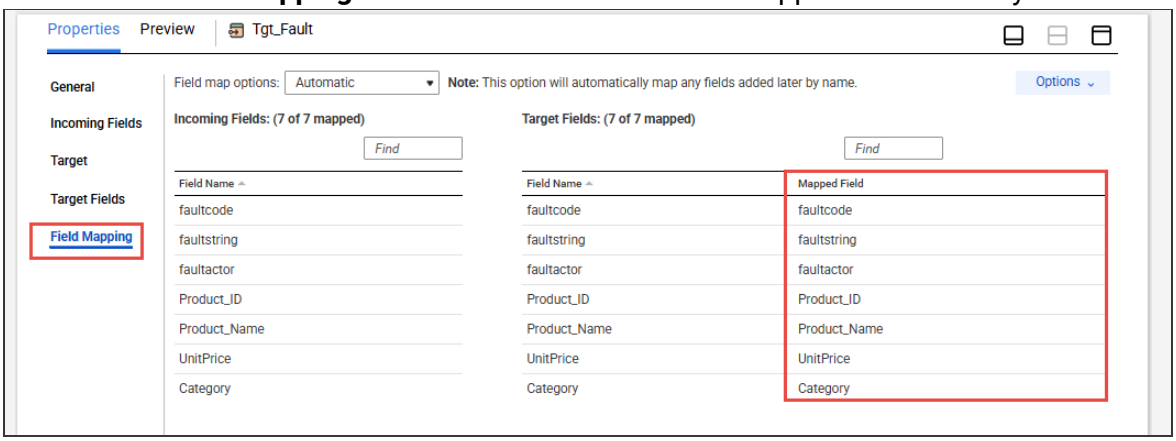
44. Selecting **Tgt_Fault** in the mapping, move to the **Target** tab in the properties.

45. From the Connection drop-down, select your **target Flat File** connection and click **Select** to select the target object.

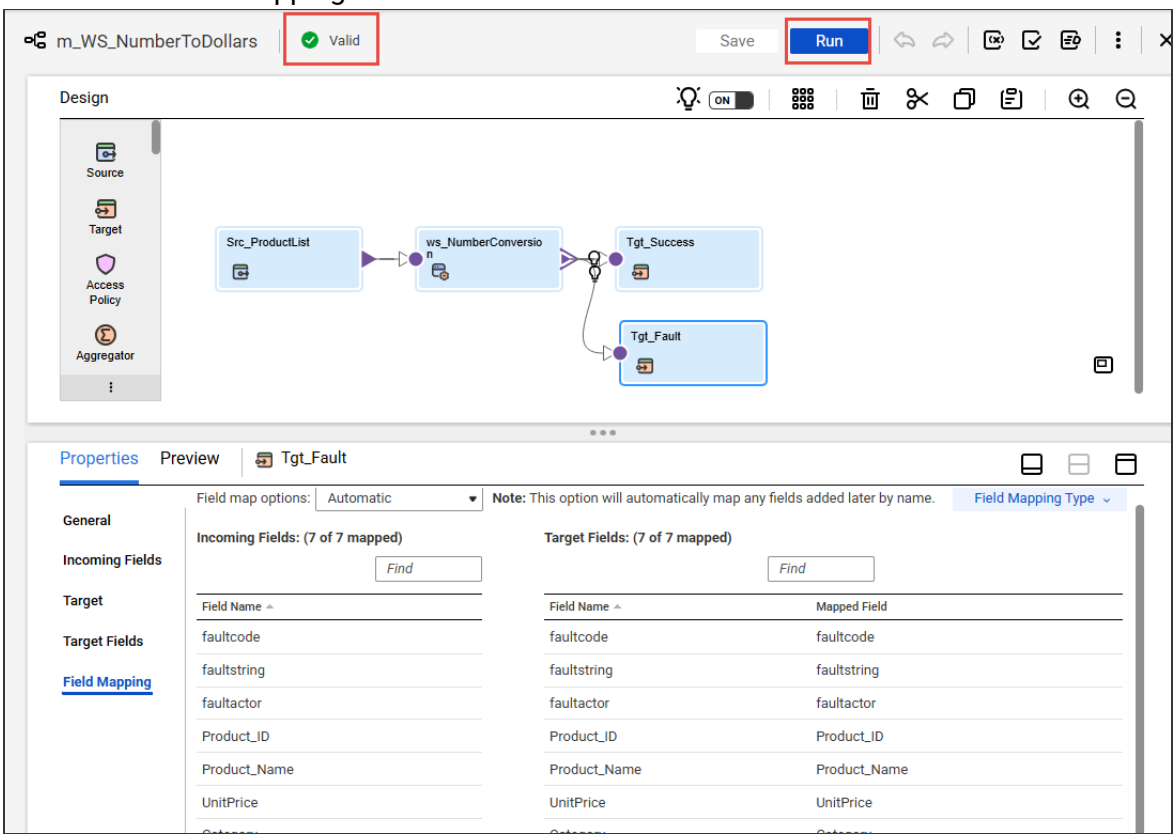
46. In the Target Object window select **WSFault.csv** and click **OK**.



47. Move to the **Field Mapping** tab and ensure the fields are mapped automatically.



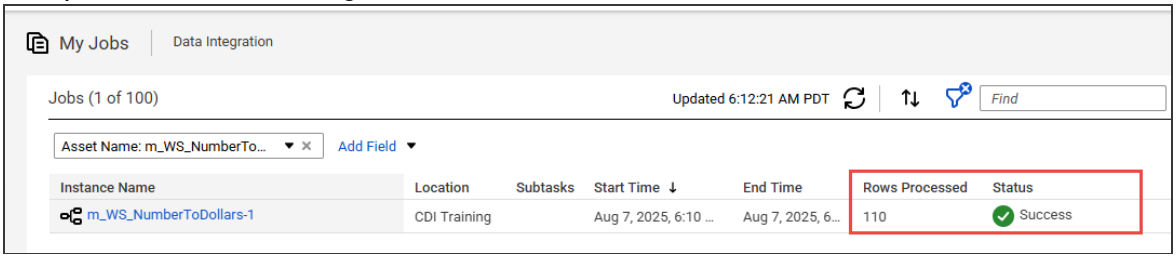
48. **Save** and **run** the mapping.



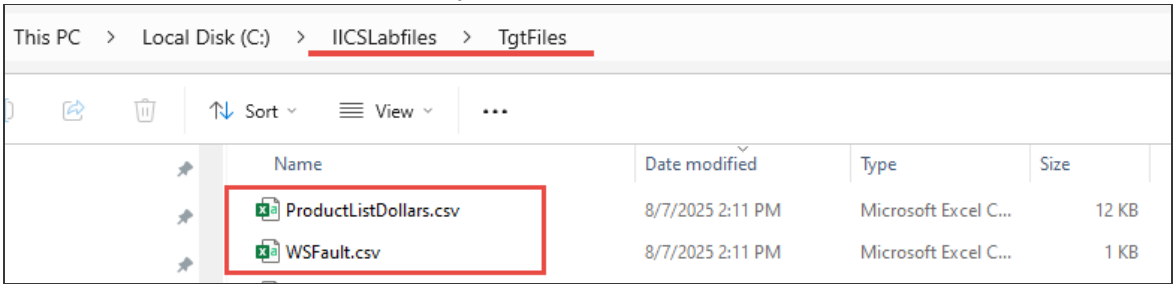
49. From the Runtime Environment drop-down, select **INFA-SERVER** and click **Run**.

Monitor the Mapping Execution Status

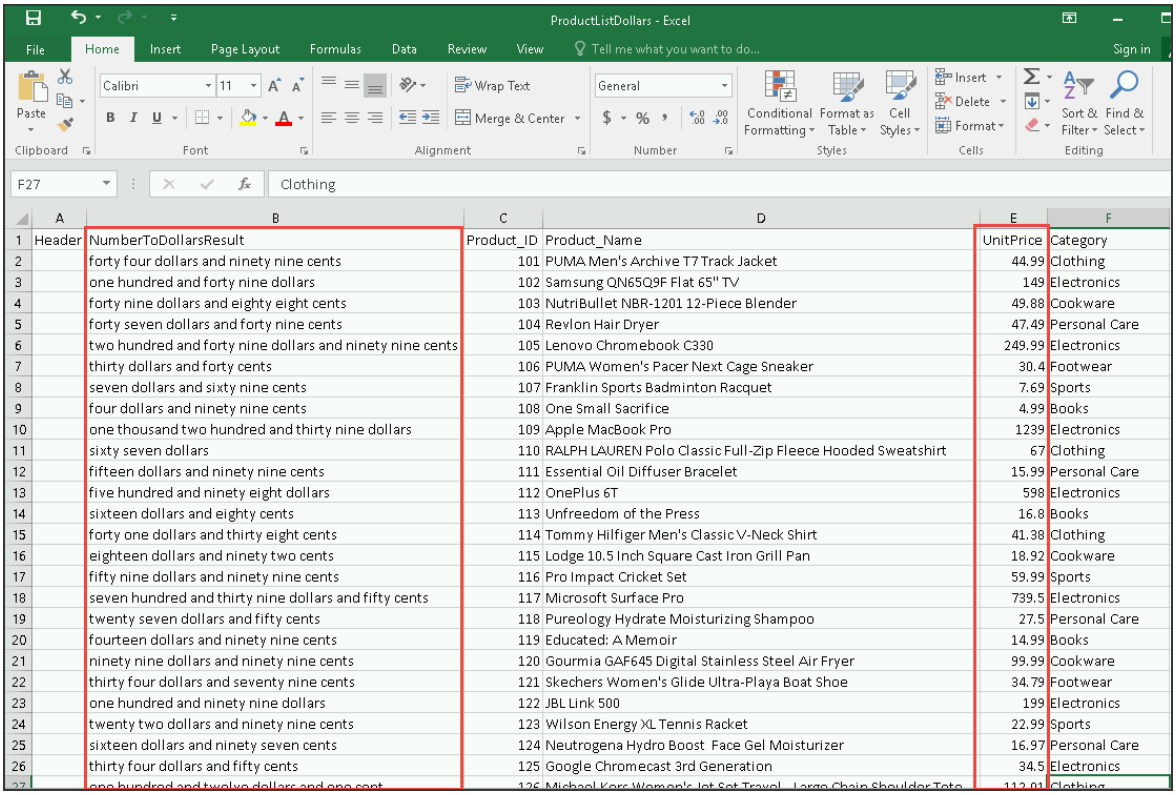
50. To monitor the mapping status, from the navigation pane, click **My Jobs**. When the task is complete, the status changes to **Success**.



51. Two output files will be updated in your target folder in the IICSLabFiles\TgtFiles folder.



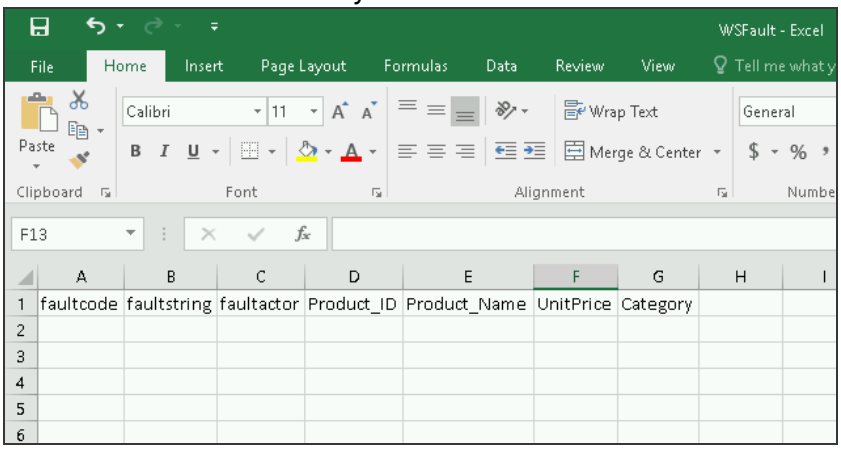
52. In the ProductListDollars file, the NumberToDollarsResult shows the unit price in words for each record.



ProductListDollars - Excel

	A	B	C	D	E	F
1	Header	NumberToDollarsResult	Product_ID	Product_Name	UnitPrice	Category
2		forty four dollars and ninety nine cents	101	PUMA Men's Archive T7 Track Jacket	44.99	Clothing
3		one hundred and forty nine dollars	102	Samsung QN65Q9F Flat 65" TV	149	Electronics
4		forty nine dollars and eighty eight cents	103	NutriBullet NBR-1201 12-Piece Blender	49.88	Cookware
5		forty seven dollars and forty nine cents	104	Revlon Hair Dryer	47.49	Personal Care
6		two hundred and forty nine dollars and ninety nine cents	105	Lenovo Chromebook C330	249.99	Electronics
7		thirty dollars and forty cents	106	PUMA Women's Pacer Next Cage Sneaker	30.4	Footwear
8		seven dollars and sixty nine cents	107	Franklin Sports Badminton Racquet	7.69	Sports
9		four dollars and ninety nine cents	108	One Small Sacrifice	4.99	Books
10		one thousand two hundred and thirty nine dollars	109	Apple MacBook Pro	1239	Electronics
11		sixty seven dollars	110	RALPH LAUREN Polo Classic Full-Zip Fleece Hooded Sweatshirt	67	Clothing
12		fifteen dollars and ninety nine cents	111	Essential Oil Diffuser Bracelet	15.99	Personal Care
13		five hundred and ninety eight dollars	112	OnePlus 6T	598	Electronics
14		sixteen dollars and eighty cents	113	Unfreedom of the Press	16.8	Books
15		forty one dollars and thirty eight cents	114	Tommy Hilfiger Men's Classic V-Neck Shirt	41.38	Clothing
16		eighteen dollars and ninety two cents	115	Lodge 10.5 Inch Square Cast Iron Grill Pan	18.92	Cookware
17		fifty nine dollars and ninety nine cents	116	Pro Impact Cricket Set	59.99	Sports
18		seven hundred and thirty nine dollars and fifty cents	117	Microsoft Surface Pro	739.5	Electronics
19		twenty seven dollars and fifty cents	118	Pureology Hydrate Moisturizing Shampoo	27.5	Personal Care
20		fourteen dollars and ninety nine cents	119	Educated: A Memoir	14.99	Books
21		ninety nine dollars and ninety nine cents	120	Gourmia GAF645 Digital Stainless Steel Air Fryer	99.99	Cookware
22		thirty four dollars and seventy nine cents	121	Skechers Women's Glide Ultra-Playa Boat Shoe	34.79	Footwear
23		one hundred and ninety nine dollars	122	JBL Link 500	199	Electronics
24		twenty two dollars and ninety nine cents	123	Wilson Energy XL Tennis Racket	22.99	Sports
25		sixteen dollars and ninety seven cents	124	Neutrogena Hydro Boost Face Gel Moisturizer	16.97	Personal Care
26		thirty four dollars and fifty cents	125	Google Chromecast 3rd Generation	34.5	Electronics

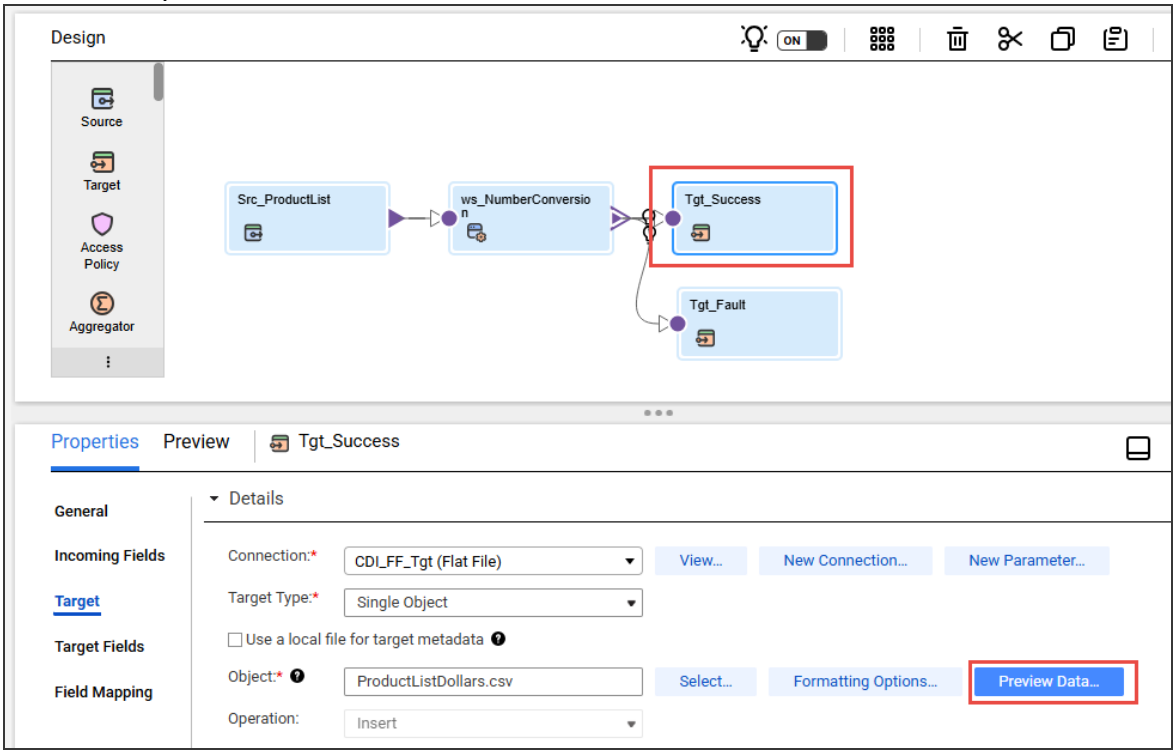
53. The WSFault file will list any faults that occurred.

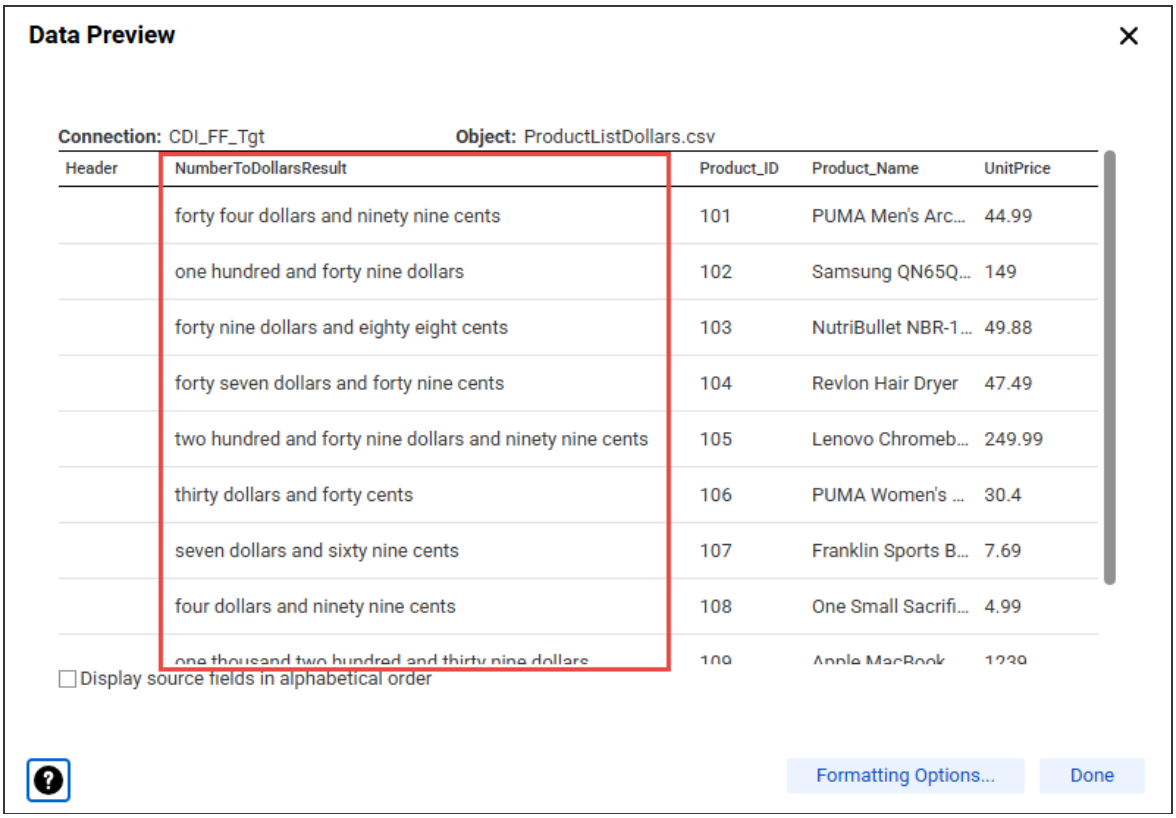


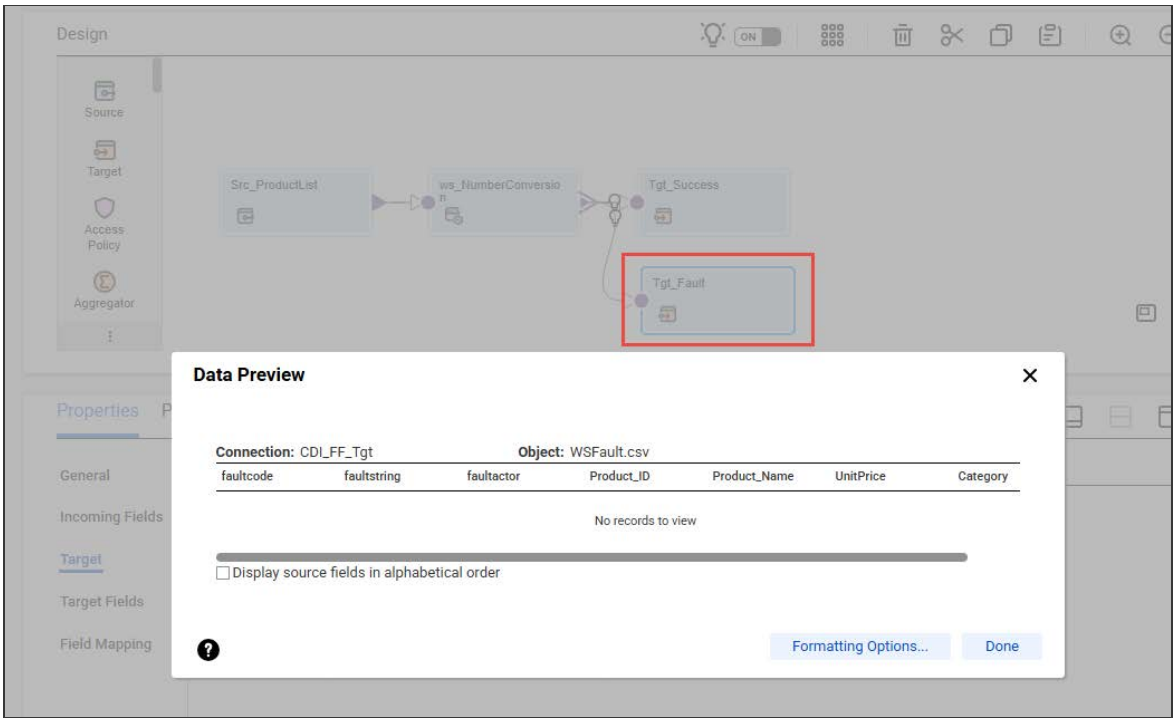
WSFault - Excel

	A	B	C	D	E	F	G	H	I
1	faultcode	faultstring	faultactor	Product_ID	Product_Name	UnitPrice	Category		
2									
3									
4									
5									
6									

54. To view the data on the **Target** tab click **Preview Data** for each, or move to the TgtFiles folder and open to review.







55. Click **Done** once complete and **close** the assets from the navigation pane.

This concludes the lab.

Module 15: IDMC REST APIs

Lab 15-1: Create a Mapping Using a REST V2 Connector to Retrieve Weather Information

Overview:

REST is a web standards-based architecture that uses HTTP Protocol for data communication. These commonly used APIs allow you to create web-based applications.

This lab focuses on using a REST V2 Connector to retrieve current weather information using the OpenWeather API. The lab demonstrates how to test the REST connection, fetch weather data in JSON format, and write the results to a flat file.

Objectives:

- Review a REST connection using the REST V2 Connector
- Retrieve weather data from the OpenWeather API
- Write the JSON response to a flat file

Duration:

30 minutes

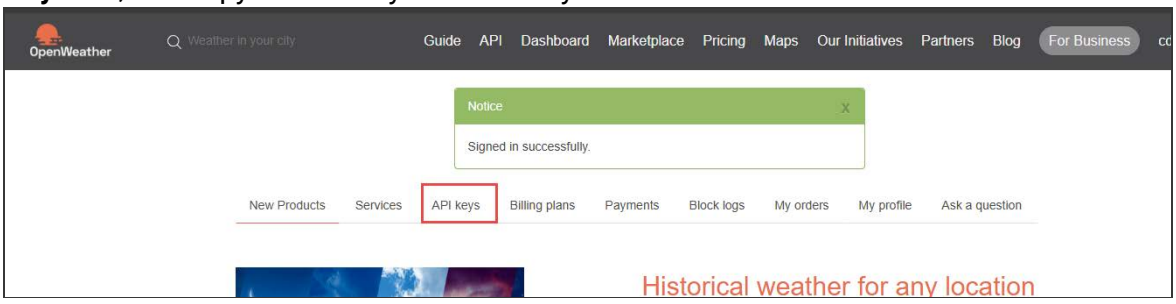
Tasks:

Sign Up to Access OpenWeatherAPI and Access the Unique API Key

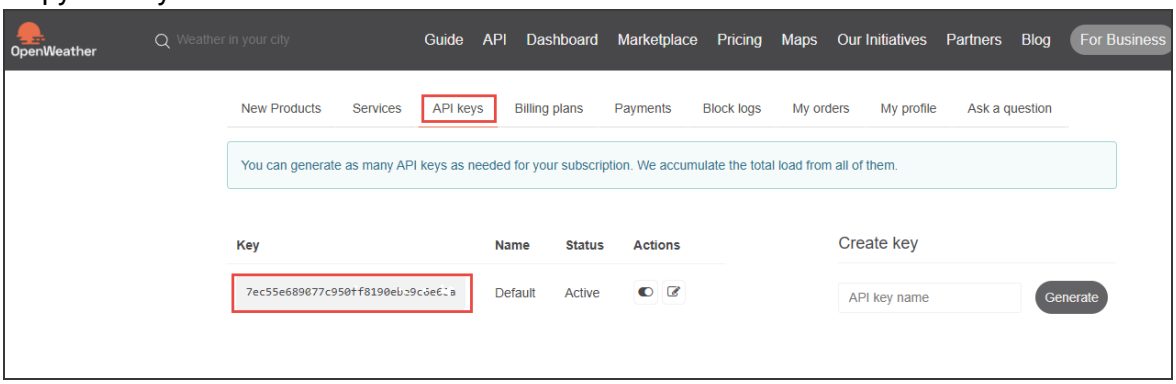
1. Open a web browser and enter the following URL in a new tab: https://home.openweathermap.org/users/sign_up

2. Create a new account in OpenWeather using the link provided. After you create an account, you can access your unique API key.

3. To access your unique API Key, log into your open weather account and select the **API Keys** tab, and copy the API key from the Key field.



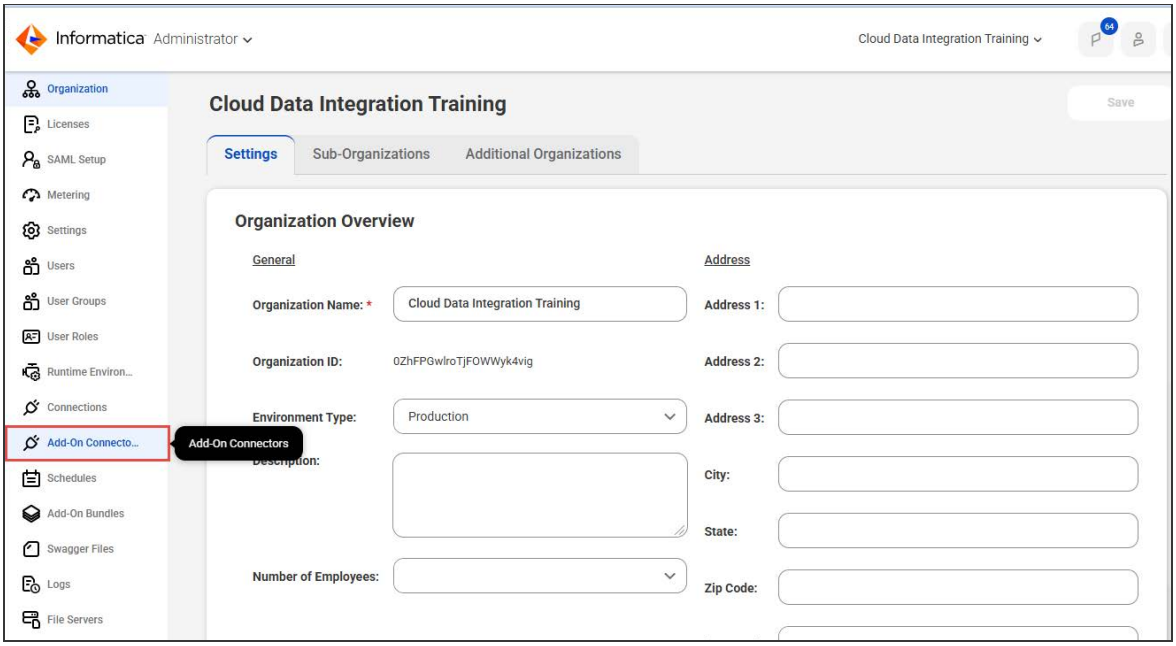
4. Copy the key:



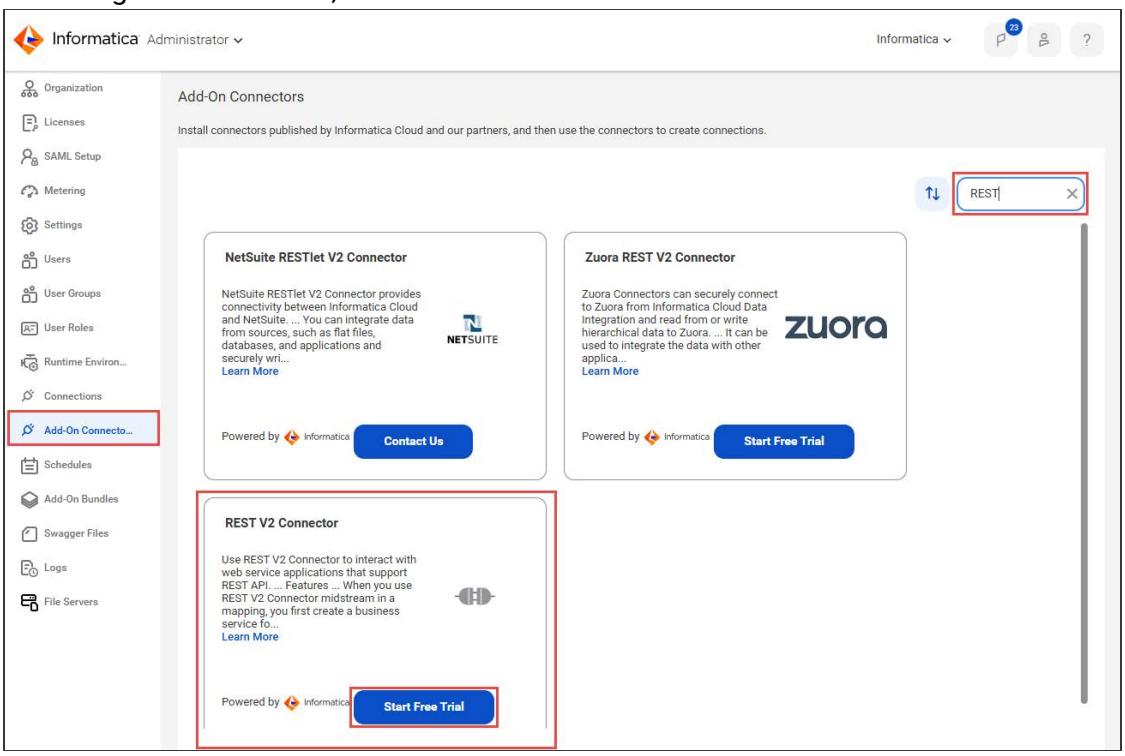
5. Paste the API key in a notepad and save the notepad file on your system.

Enable the REST V2 Connector in the Administrator

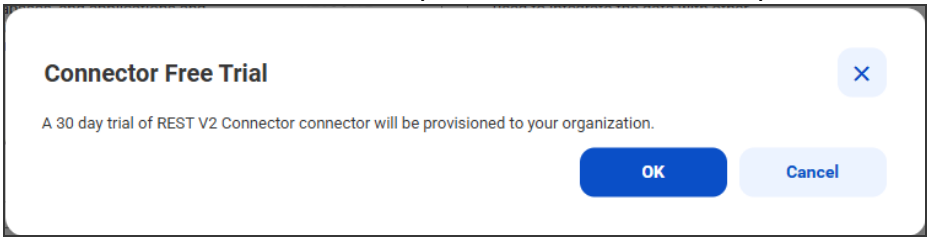
6. In the **Administrator** service, from the navigation pane to the left, select **Add-On Connectors**.



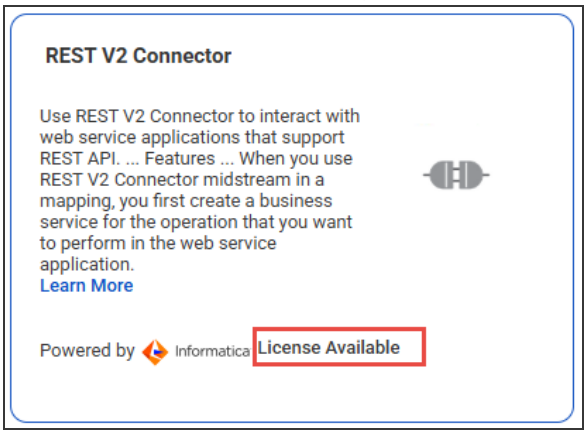
7. To the right in the **Search**, enter **REST** and locate the REST V2 Connector.



8. Locate the **REST V2 Connector** and check if the REST V2 connector is available for the Org. If the connector is not available press **Start Free Trial** and press **OK** on the dialog.

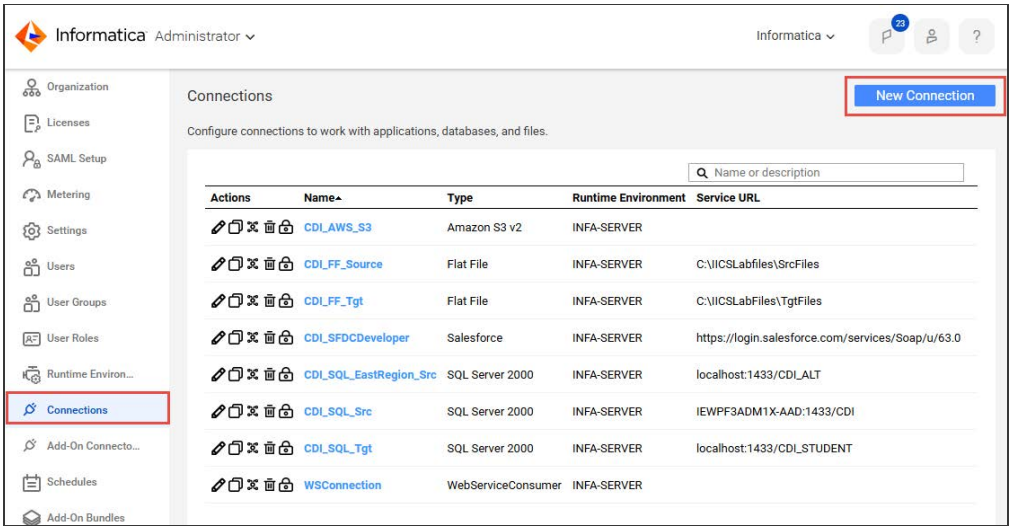


9. If the connector is available, its status shows as '**License Available**'.



Configure a REST V2 connector

10. Move to the **Connections** tab and choose **New Connection**.

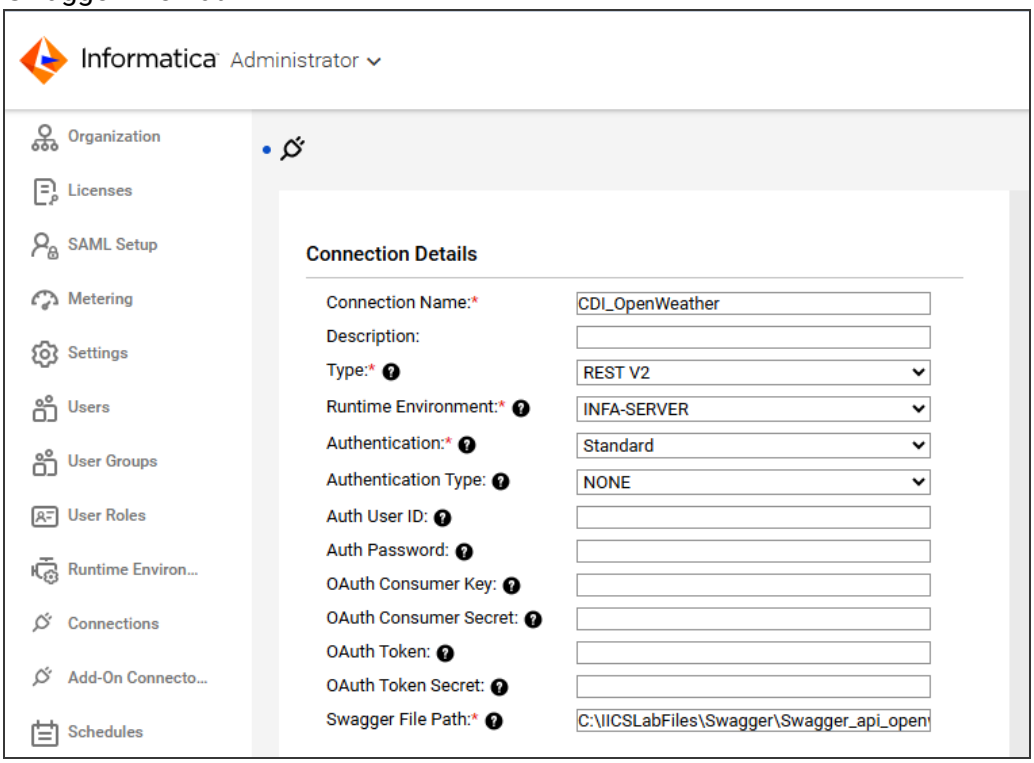


11. Set the name as **CDI_OpenWeather** and the type as **RESTV2.**

12. Set the Runtime Environment to INFA-SERVER.

13. Set Authentication to Standard and leave Authentication Type as NONE.

14. Enter **C:\IICSLabFiles\Swagger\Swagger_api_openweathermap_org_072248.json** as the Swagger File Path.

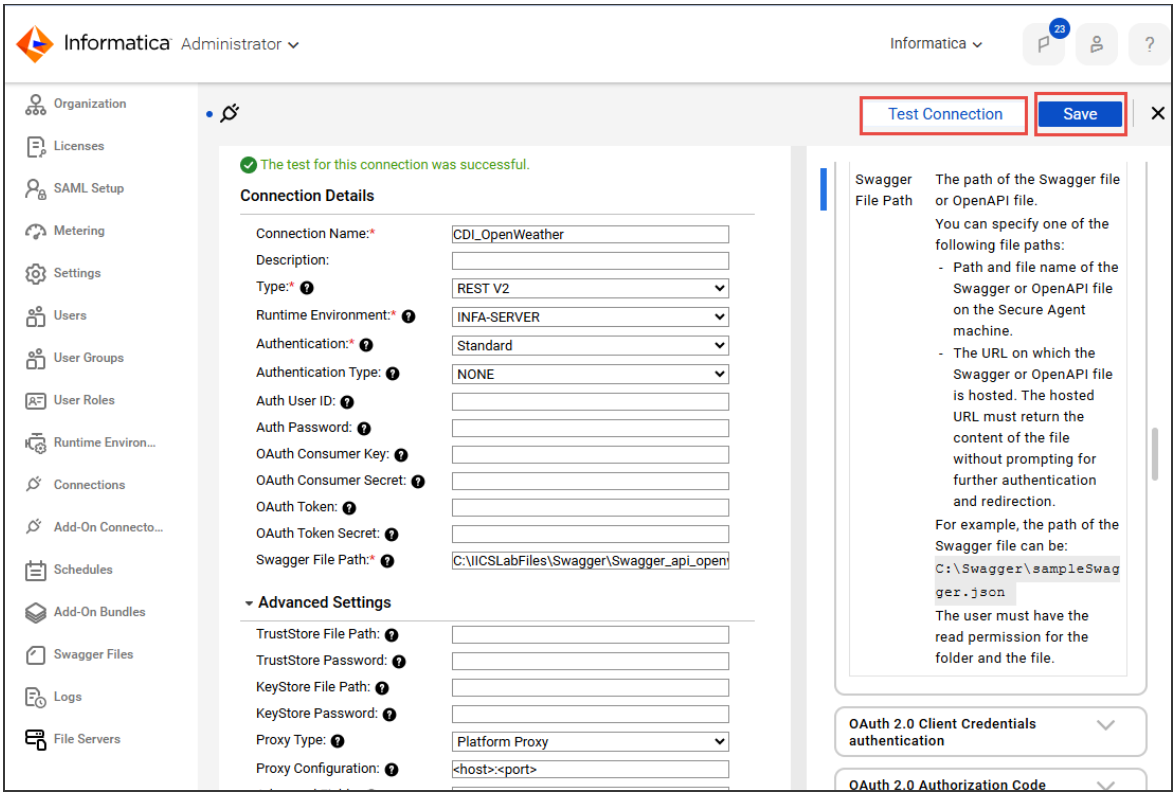


A Swagger file is a comprehensive document detailing the structure and behavior of RESTful APIs. Typically formatted in JSON or YAML, it encompasses all necessary components for describing and interacting with an API, such as:

- **Paths:** API endpoints or routes.
- **Operations:** Available methods at each path (e.g., GET, POST, PUT, DELETE).
- **Parameters:** Inputs to operations that may include query parameters, path parameters, headers, or request bodies.
- **Responses:** Expected responses from operations, including status codes and response bodies.
- **Security:** Authentication methods required for API access.
- **Definitions:** Data models and schemas for request and response bodies.

15. Expand the **Advanced Settings** and ensure the Proxy Type is selected as **Platform Proxy**.

16. **Save** and **Test** the connection to ensure it is successful.



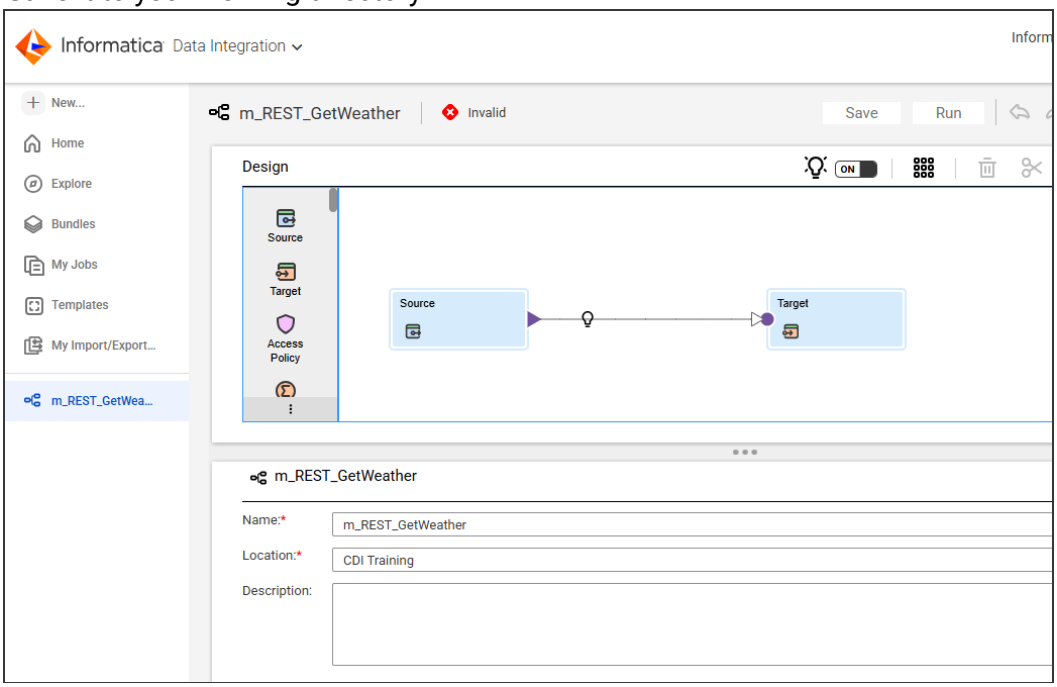
17. **Close** the connection. This connection should be used in the mapping.

Create a Mapping to Access OpenWeather and Return the Weather for the Requested City

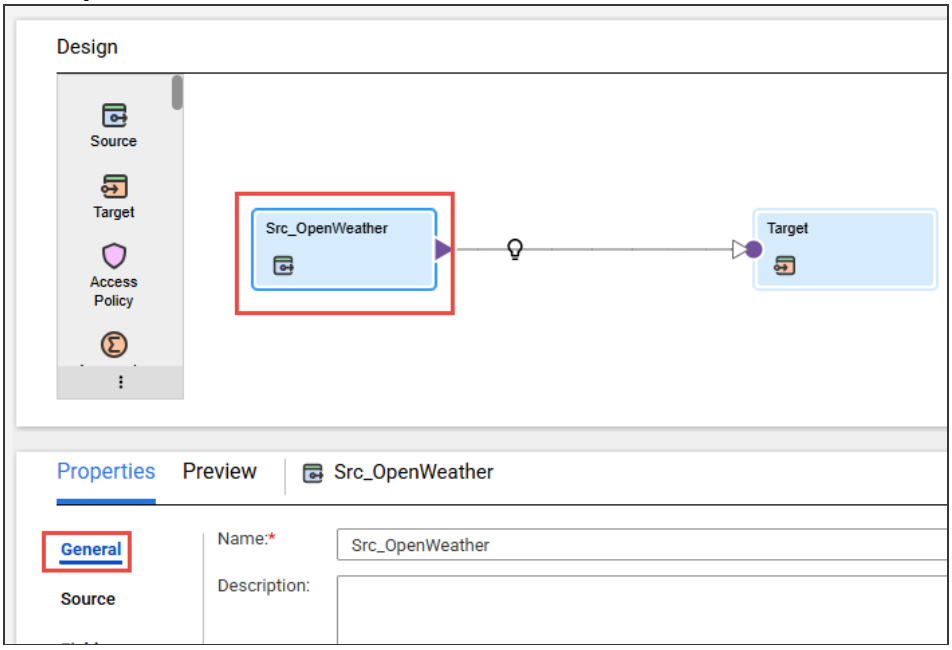
18. Using **My Services**, switch to the **Data Integration** service.

19. Create a new mapping and in the Name field, enter **m_REST_GetWeather**.

20. **Save** it to your working directory.



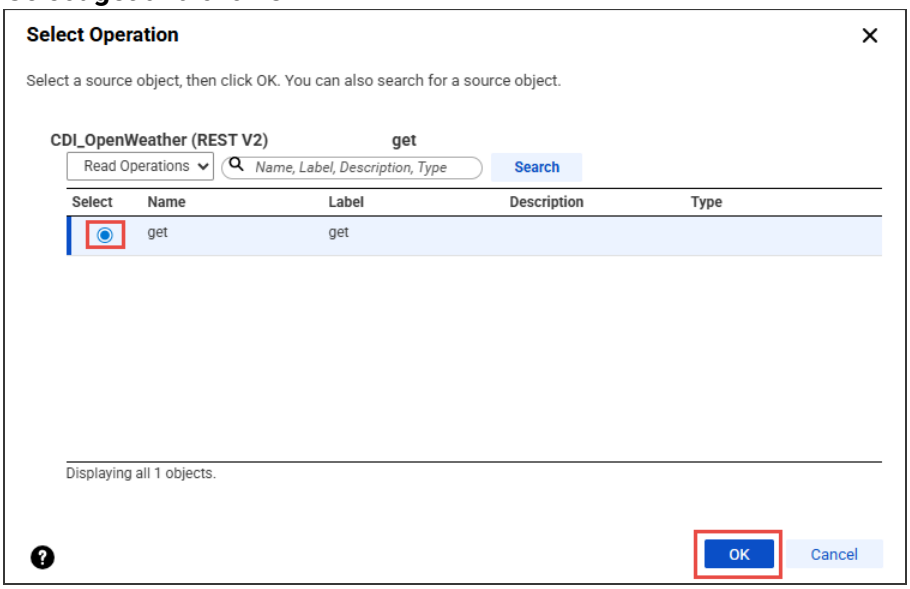
21. Select the **Source** transformation and on the **General** tab, set the Name as **Src_OpenWeather**.



22. From the properties pane, click **Source** and from the Connection drop-down, select **CDI_OpenWeather (REST V2)**.

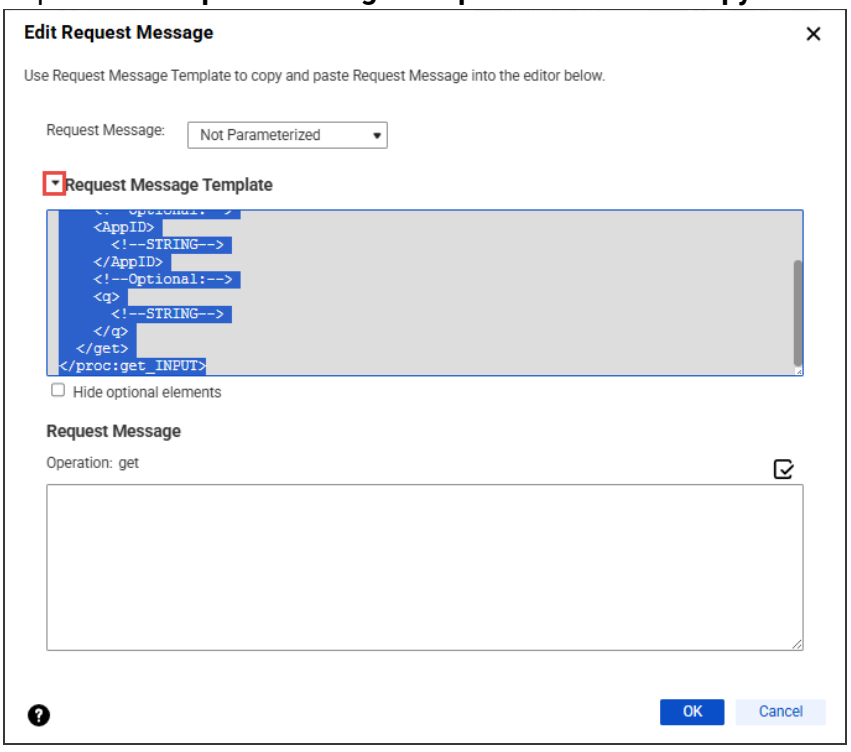
23. For the **Operation** field, click **Select**.

24. Select **get** and click **OK**.

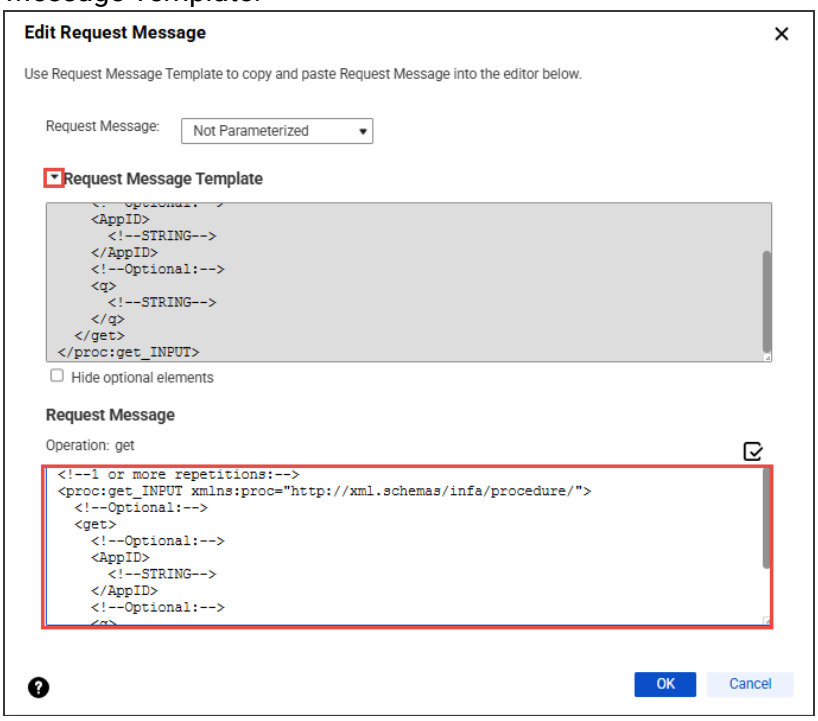


25. Expand the **Query Options** section and click **Configure**.

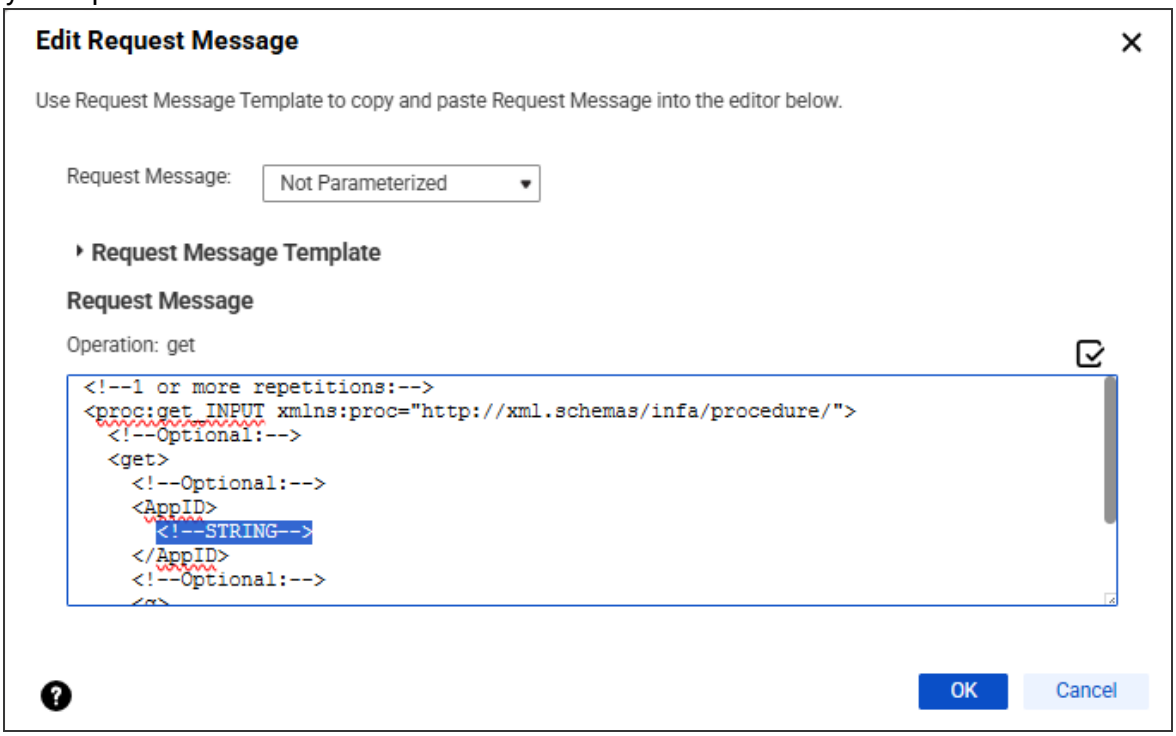
26. Expand the **Request Message Template** section and **copy** all the contents from the field.

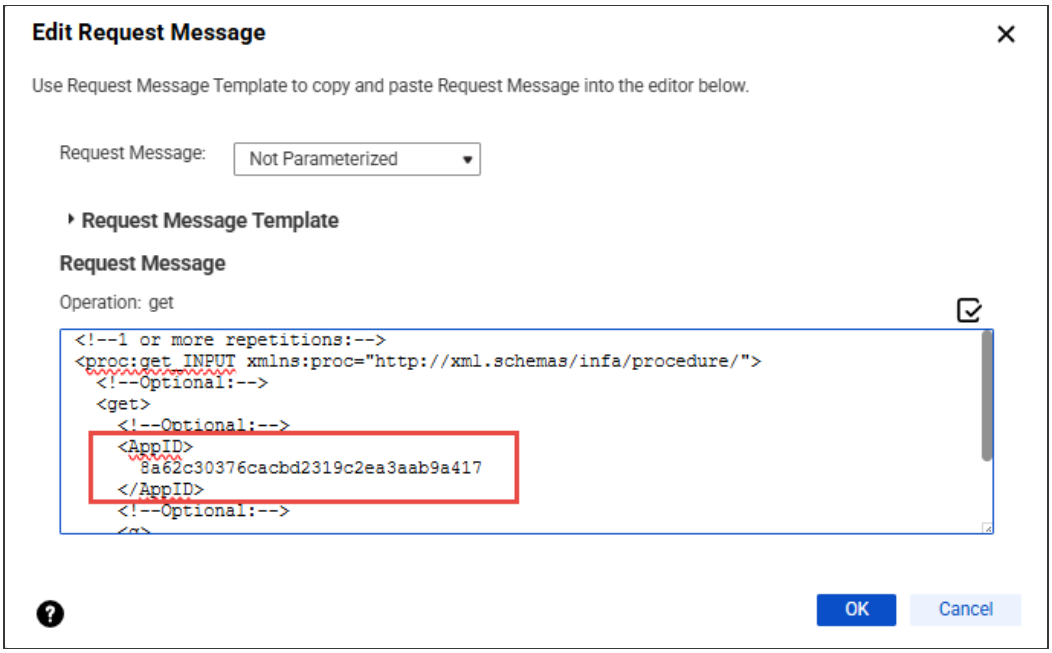


27. **Paste** the copied contents into the **Request Message** field and collapse the Request Message Template.

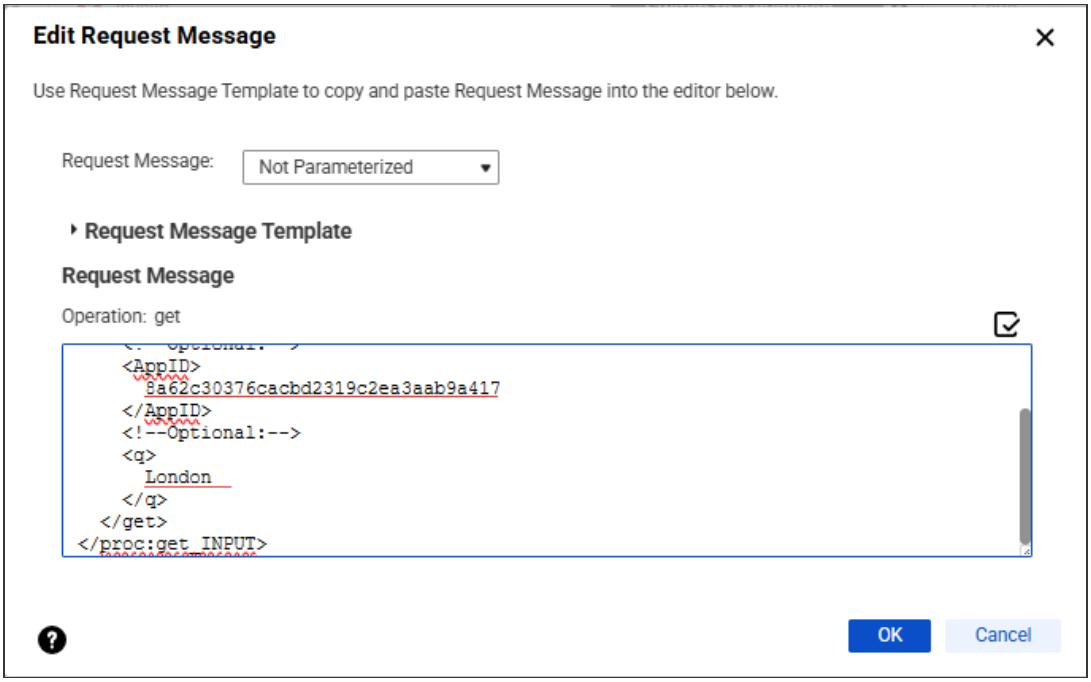


28. In the **Request Message** field, in between **<AppID> </AppID>**, enter the unique API Key that you copied earlier.

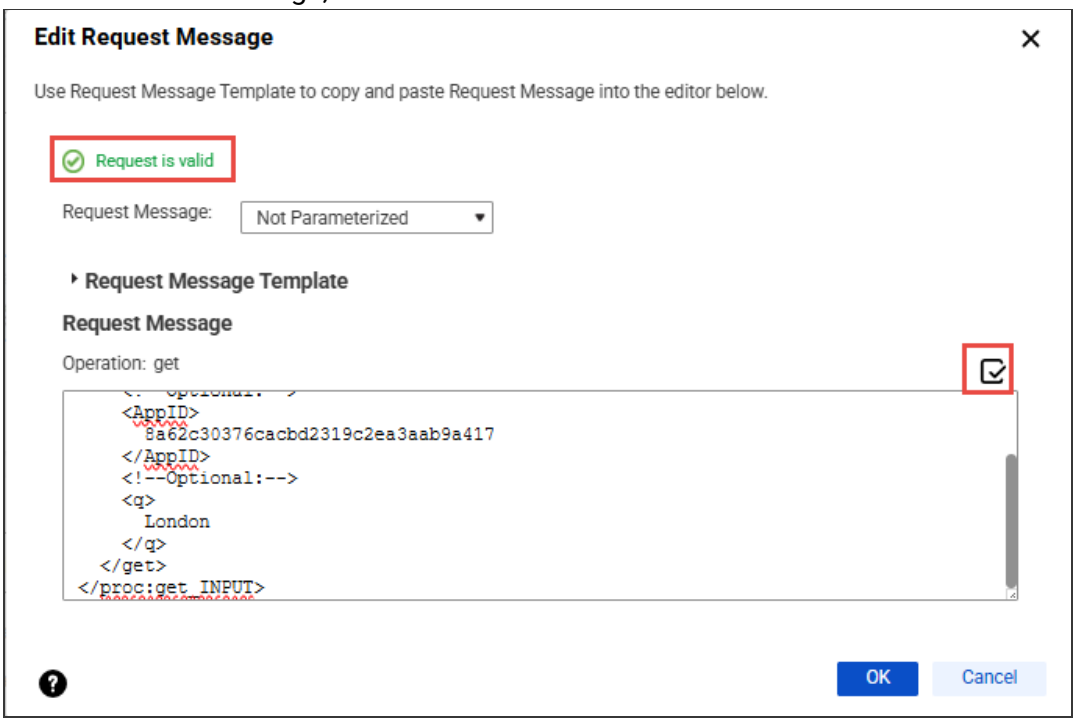




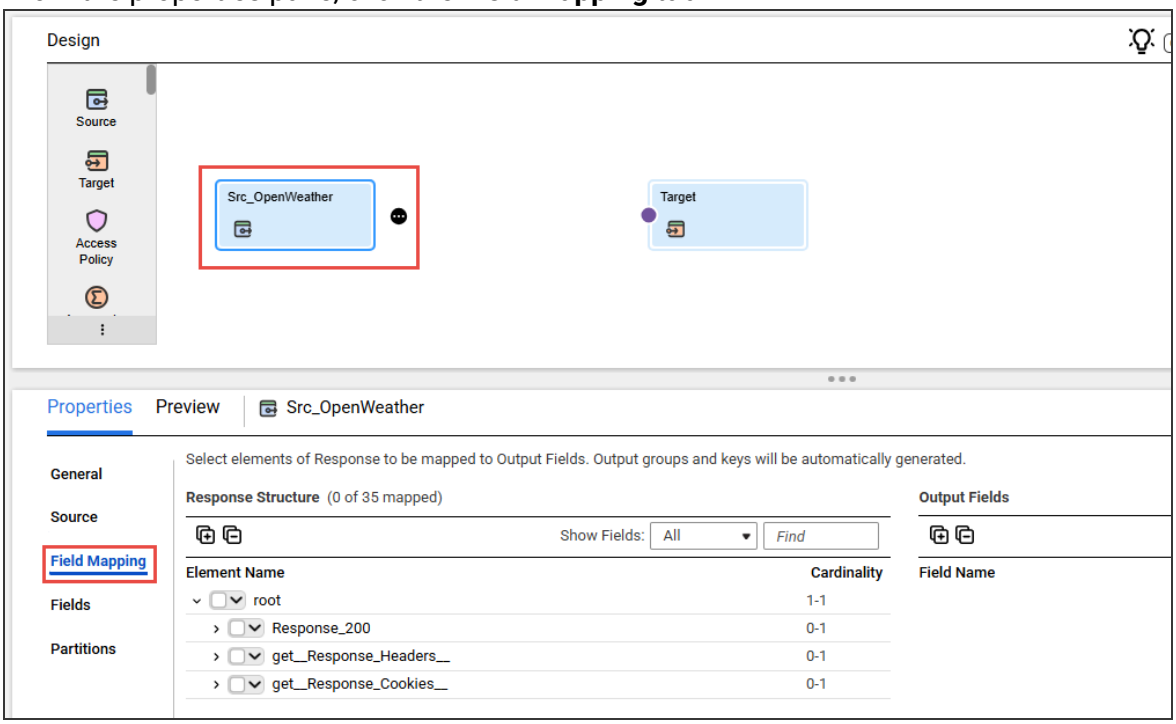
29. In the Request Message field, in between **<q> </q>**, enter **London**.
Example:
<AppID>
8a62c30376cacbd2319c2ea3aab9a417
</AppID>
<q>
London
</q>



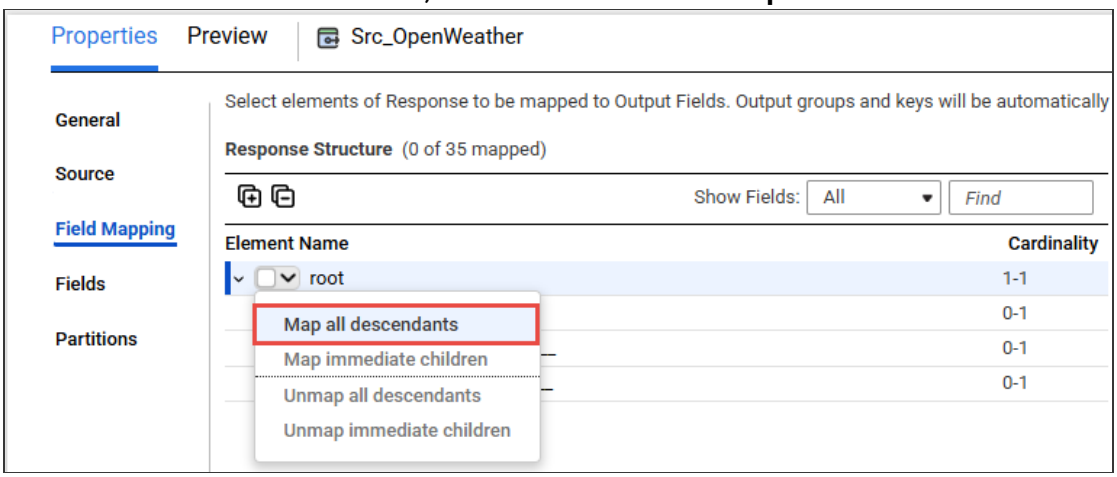
30. To validate the message, click  and click **OK**.



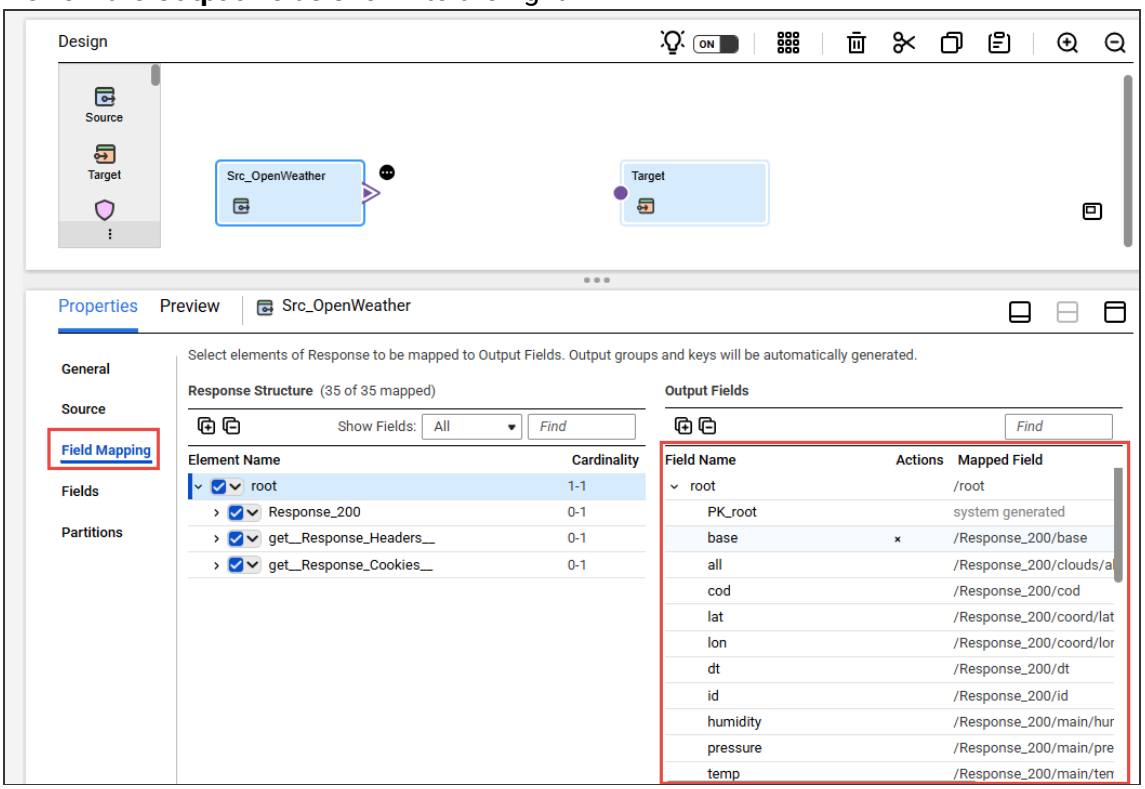
31. From the properties pane, click the **Field Mapping** tab.



32. From the **Element Name** section, select **root** and select **Map all descendants**.

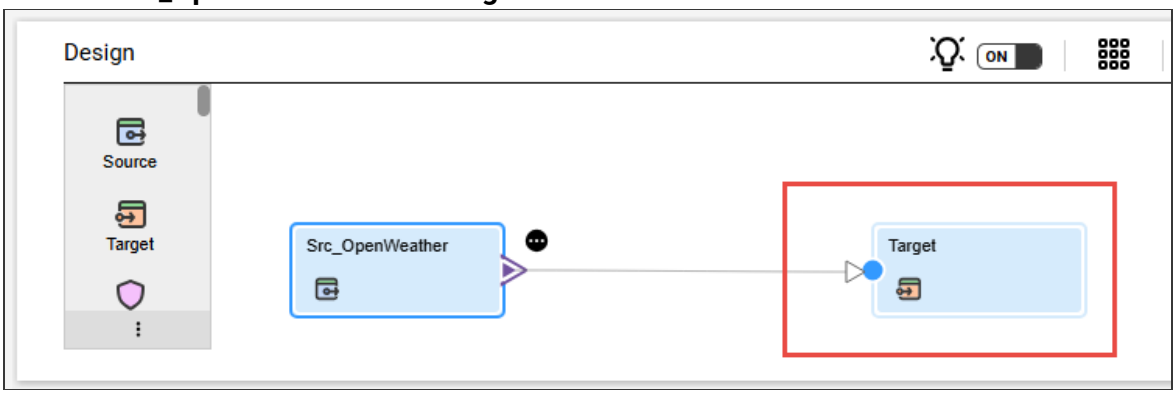


33. Review the **Output fields** shown to the right.



34. **Save** the mapping.

35. Link the **Src_OpenWeather** to the **Target**.



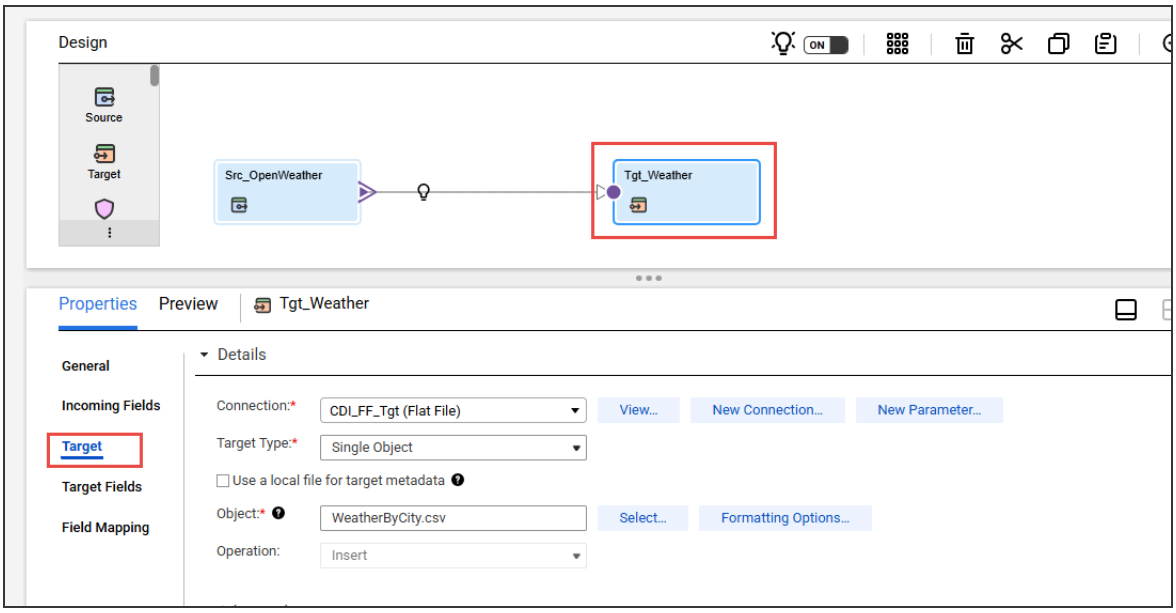
36. Select **root** and click **OK**.

37. Select the Target and on the **General** tab set the Name as **Tgt_Weather**.

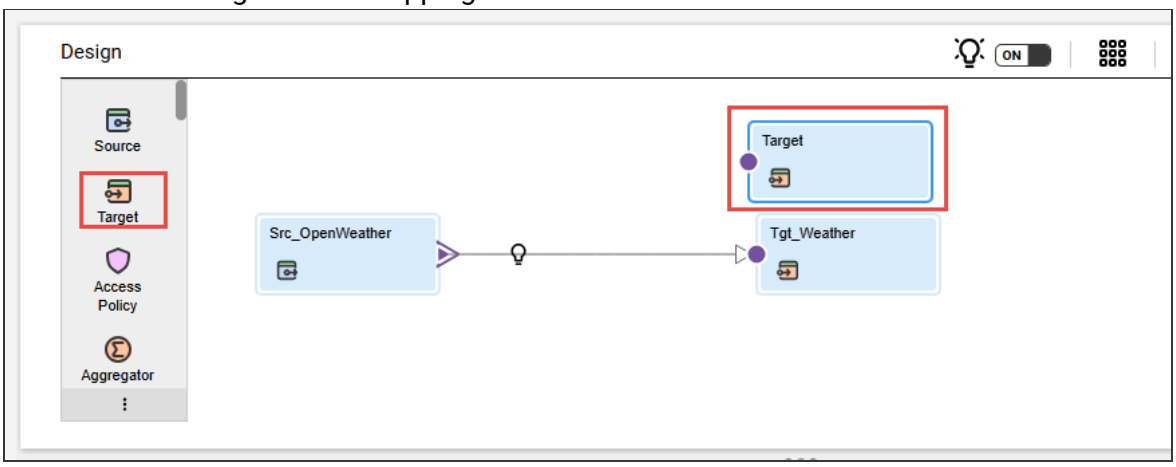
38. Move to the **Target** tab and from the Connection drop-down, select your **target Flat File** connection.

39. Click **Select** and in the Target Object window, select **Create New at Runtime** and enter **WeatherByCity.csv** as the Static File Name.

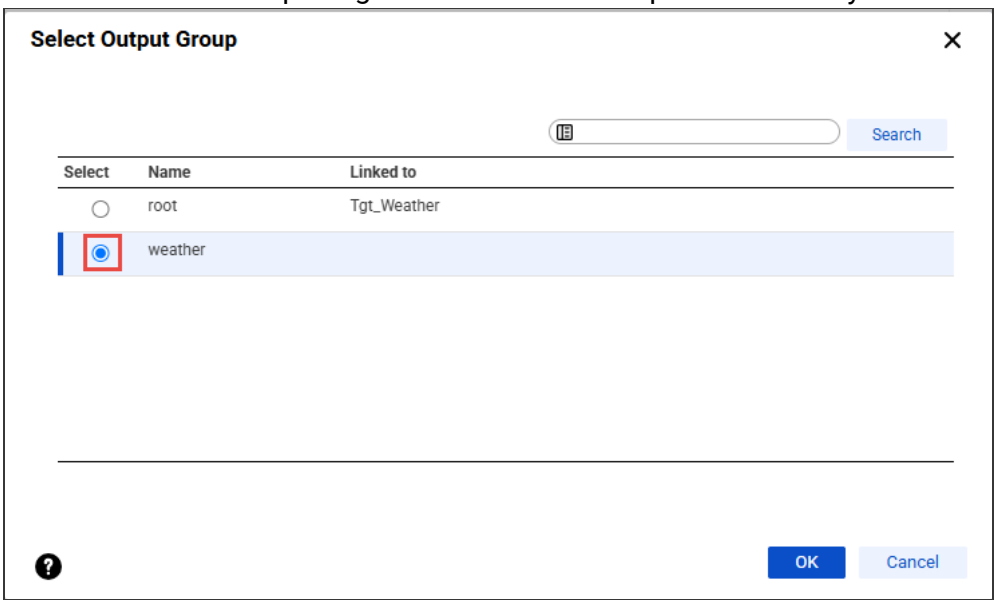
40. Click **OK**.



41. Add a second Target to the mapping.



42. Connect **Src_OpenWeather** to the new Target and this time choose the option **weather** and click **OK**. This will output a general weather description for the city.

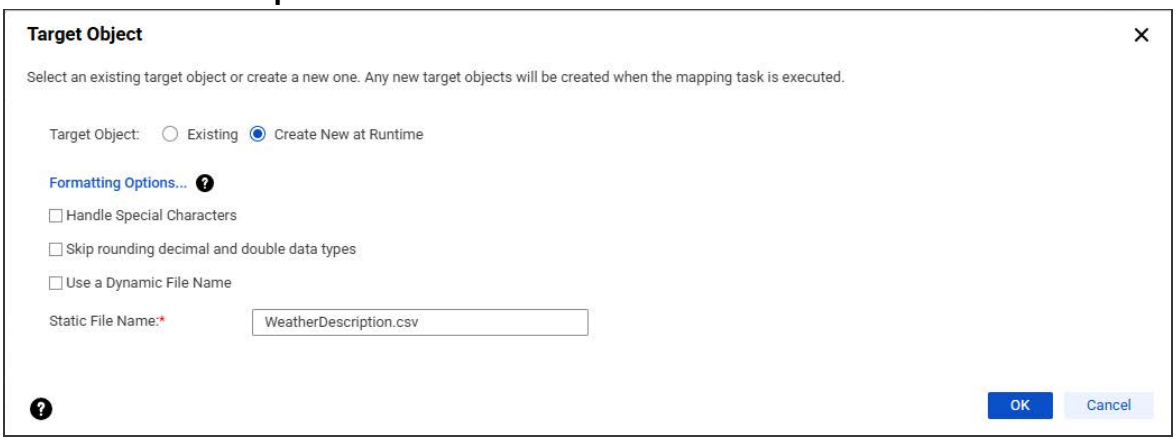


43. In the properties, on the General tab, set the name as **Tgt_WeatherDesc.**

44. Move to the **Target** tab and from the Connection drop-down, select your **target Flat File** connection.

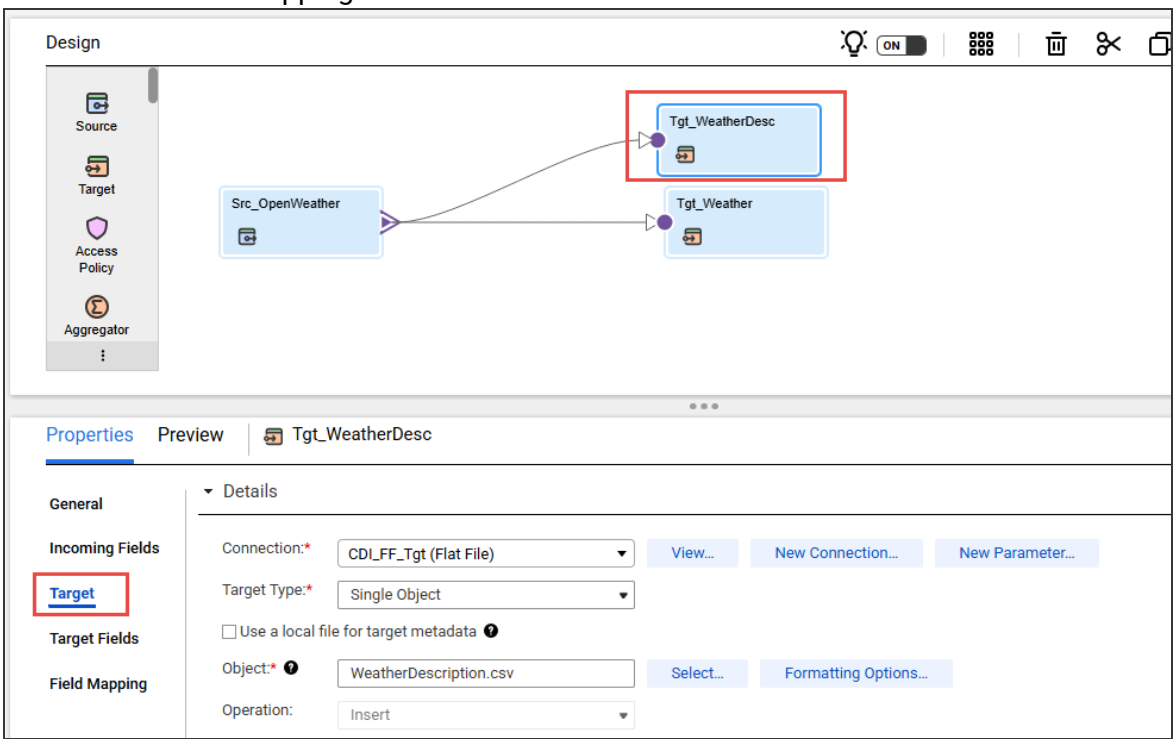
45. To select the target object, from the Object field, click **Select** and in the Target Object window, select **Create New at Runtime**.

46. Enter **WeatherDescription.csv** as the Static File Name.



47. Click **OK.**

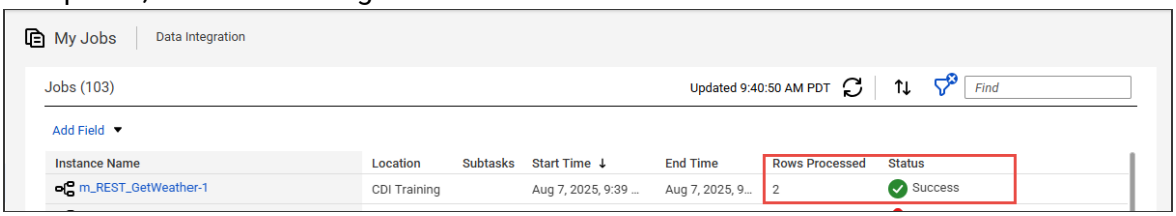
48. **Save** and **Run** the mapping.



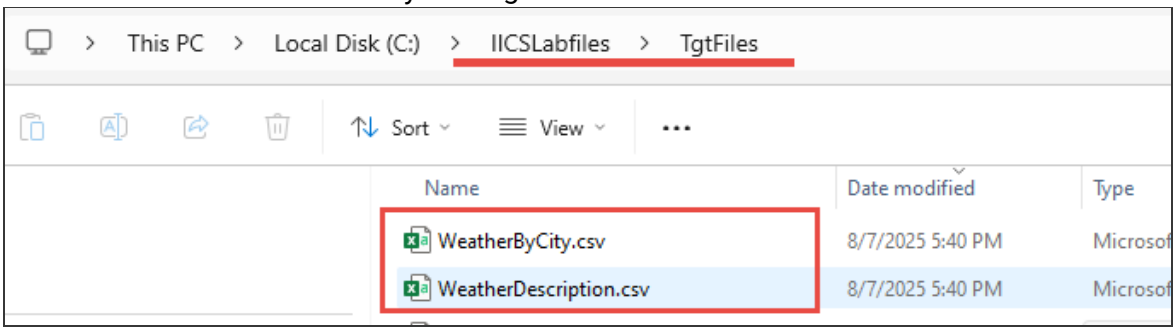
49. From the Runtime Environment drop-down, select INFA-SERVER and click **Run**.

Monitor the Mapping Execution Status

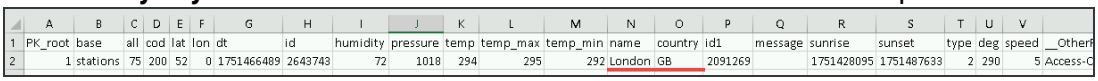
50. To monitor the mapping status, from the navigation pane, click **My Jobs**. When the task completes, the status changes to **Success**.



Two csv files will be written to your target folder.

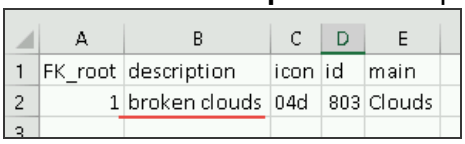


- **WeatherByCity** will contain weather information about London as requested.



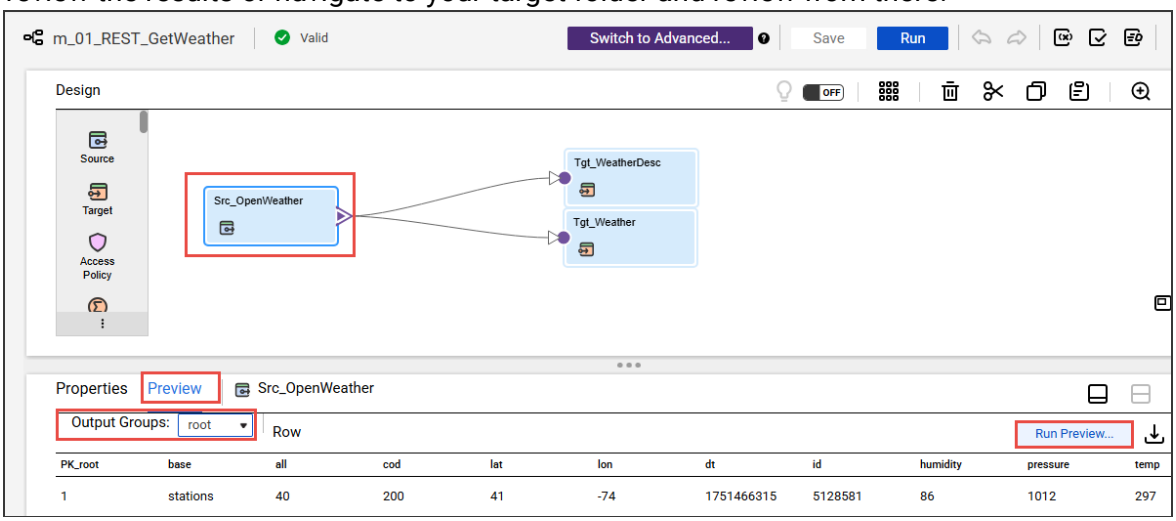
	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	
1	PK_root	base	all	cod	lat	lon	dt	id	humidity	pressure	temp	temp_max	temp_min	name	country	id1	message	sunrise	sunset	type	deg	speed	__Other
2	1	stations	75	200	52	0	1751466489	2643743	72	1018	294	295	292	London	GB	2091269		1751428095	1751487633	2	290	5	Access-C

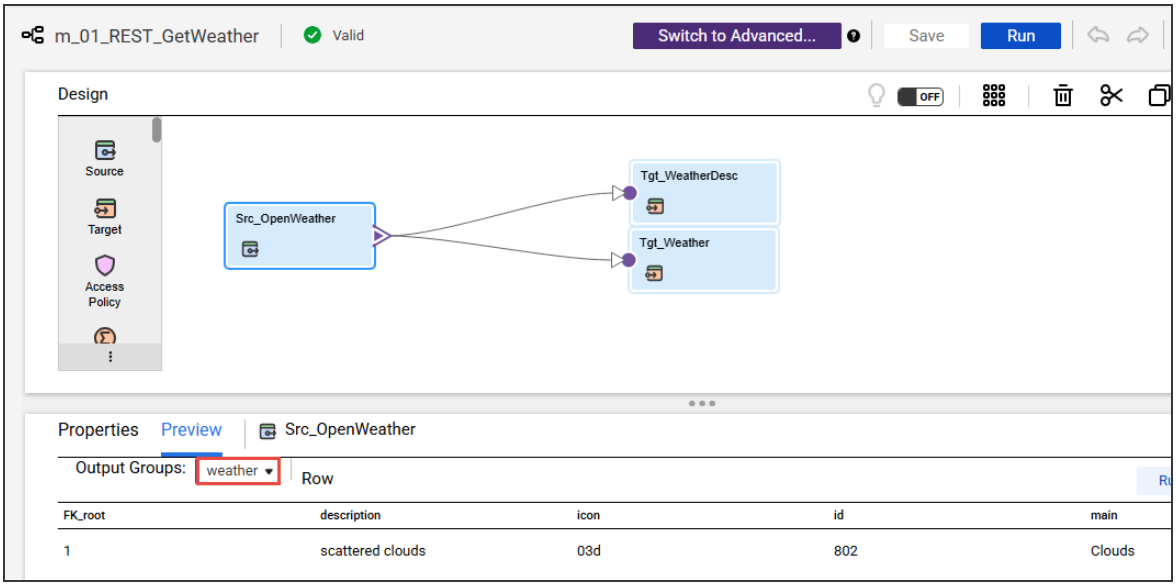
- The **Weather Description** file will provide a brief weather description for the city.



	A	B	C	D	E
1	FK_root	description	icon	id	main
2	1	broken clouds	04d	803	Clouds
3					

51. Either, select the **source** and run a data Preview. Select the different Output Groups to review the results or navigate to your target folder and review from there.





52. **Close** the asset from the navigation pane.

This concludes the lab.

Lab 15-2: Use Informatica CDI to login to the Platform API and Run a Job

Overview:

This lab focuses on utilizing Cloud Data Integration to interact with the Informatica Platform API through REST V2 connections. We will review predefined API connections, define business services for logging into the platform and running a task, and execute these tasks using web service transformations in a mapping.

Objectives:

- Define the required Business Services to login to Informatica and run a job
- Define the required REST V2 Connections
- Configure a mapping with web service transformations calling the business services

Duration:

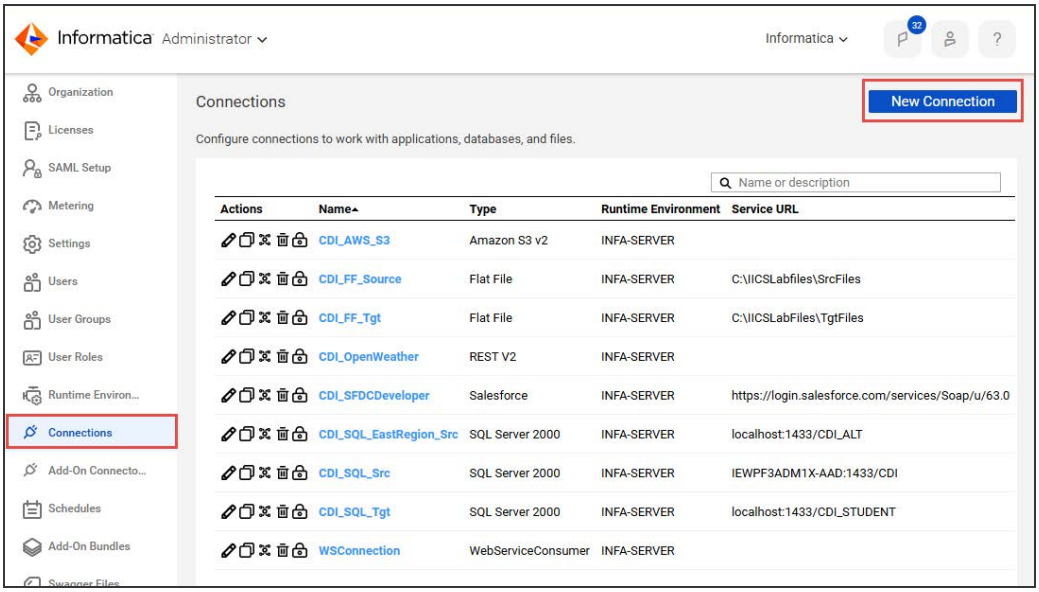
60 minutes

Tasks:

Define the REST V2 Connection

1. Move to the **Administrator** Service and open the **Connections** tab.

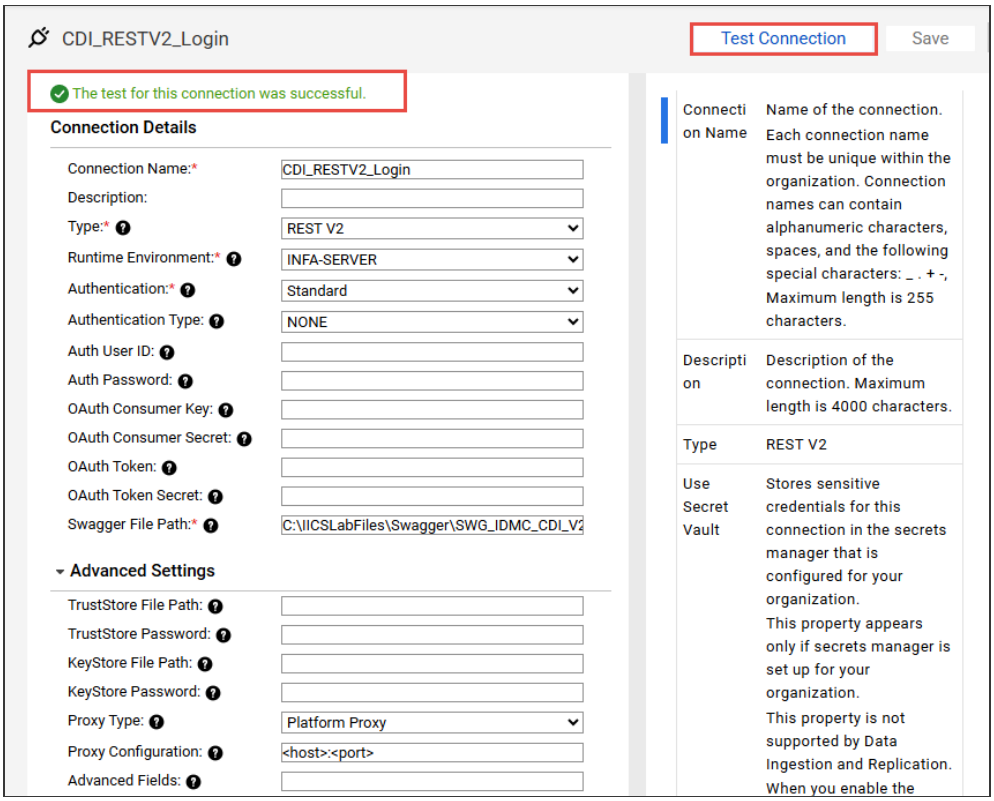
2. Create a **New Connection**.



3. Configure the following properties:

Connection Name:	CDI_RESTV2_Login
Type:	REST V2
Runtime Environment:	INFA-SERVER
Authentication	Standard
Authentication Type	None
Swagger File Path:	C:\IICSLabFiles\Swagger\SWG_IDMC_CDI_V2_LOG-IN_v2.json
Proxy Type	Platform Proxy

4. **Save** and **test** the connection.



The connection is pointing to a swagger file in the training folder. The swagger file is a definitions file that describes the web service and how to interact with it. They can be created in the Administrator service.

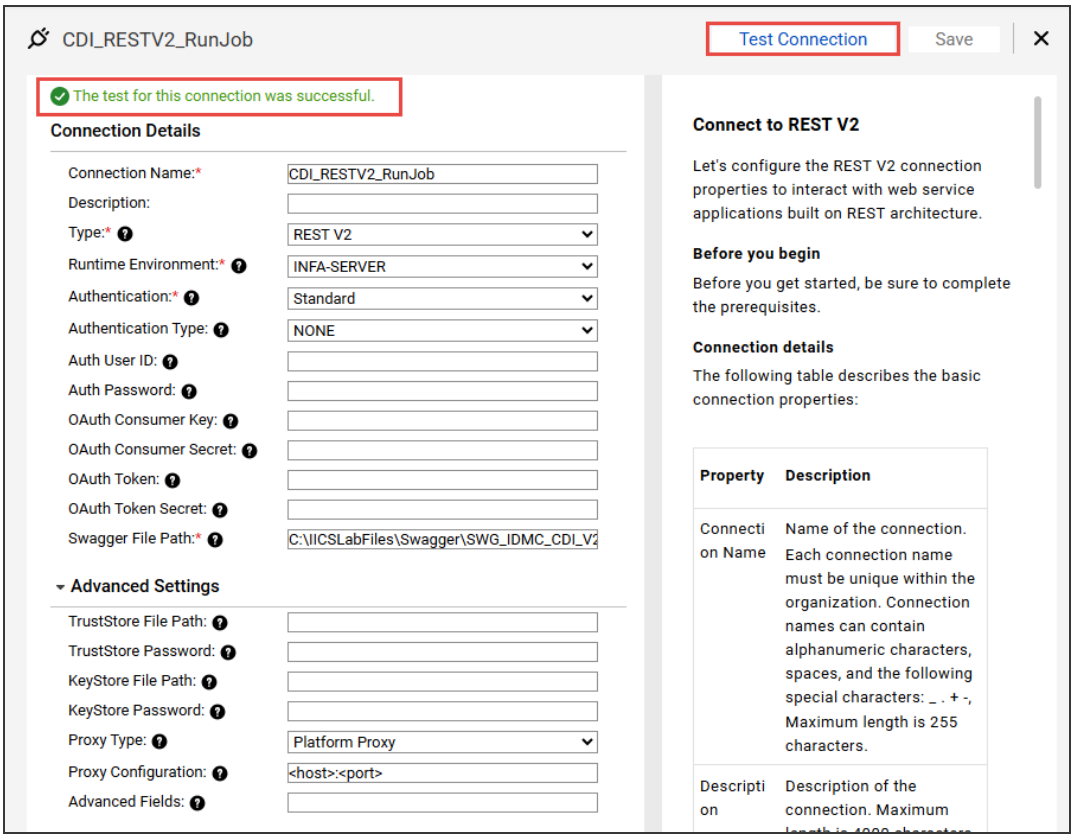
The Auth User ID and Password are not required here.

Define the RunJob Connection

5. Create a **New Connection** and define the following properties:

Connection Name:	CDI_RESTV2_RunJob
Type:	REST V2
Runtime Environment:	INFA-SERVER
Authentication	Standard
Authentication Type	None
Swagger File Path:	C:\IICSLabFiles\Swagger\SWG_IDMC_CDI_V2_RUN_JOBa.json
Proxy Type	Platform Proxy

6. **Save** and **Test** the connection.



The connection is pointing to a swagger file in the training folder. The swagger file is a definitions file that describes the web service and how to interact with it. They can be created in the Administrator service.

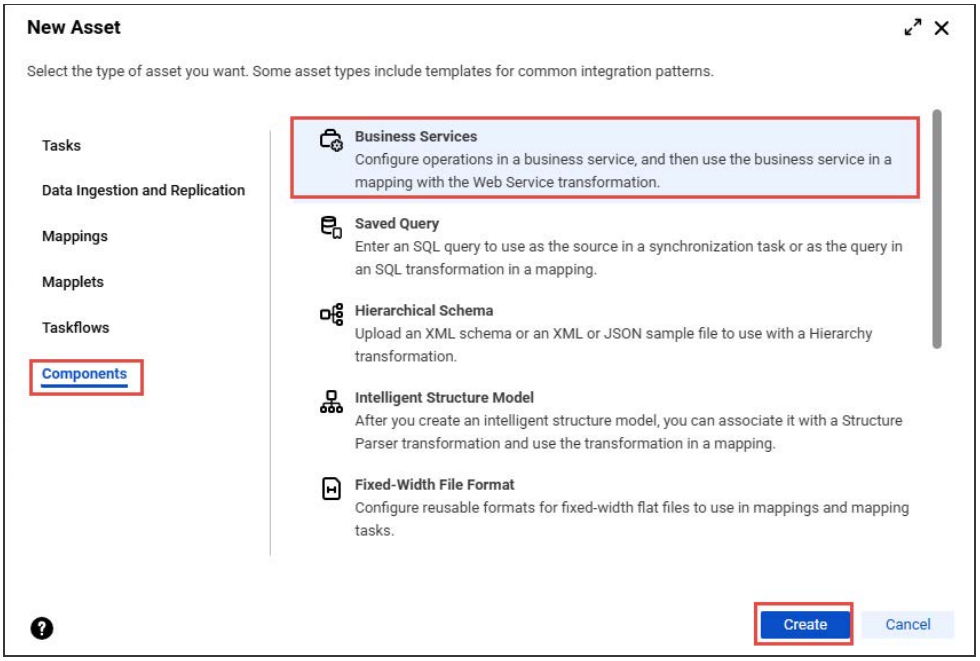
The Auth User ID and Password are not required here.

7. Close any open assets.

Create the Login Business Service

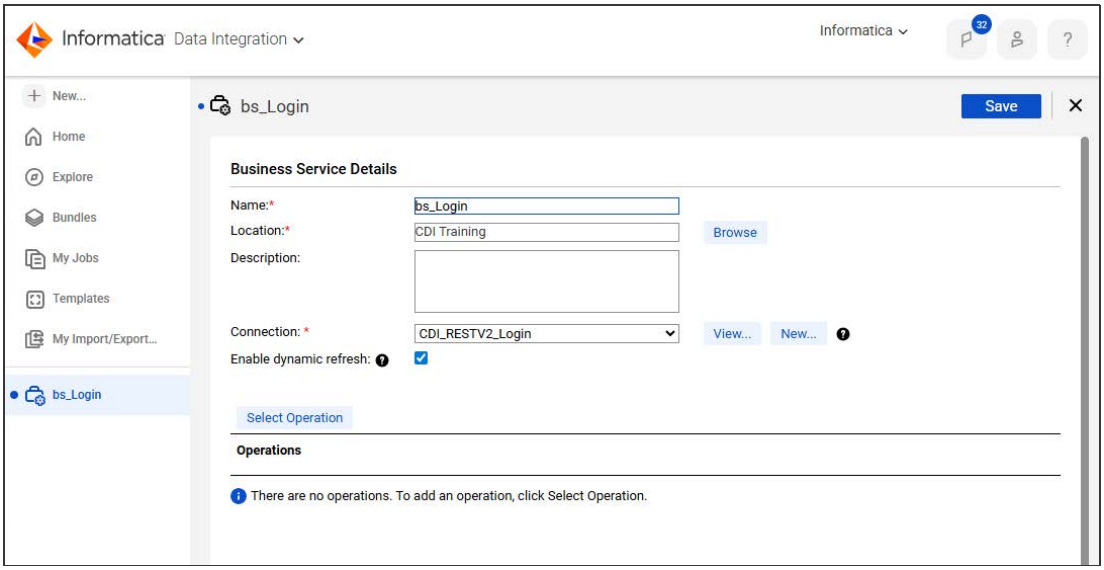
This service can be used in a Web Service transformation in a mapping.

8. Return to the **Data Integration** Service, click **New** > **Components** > **Business Services** and click **Create**.



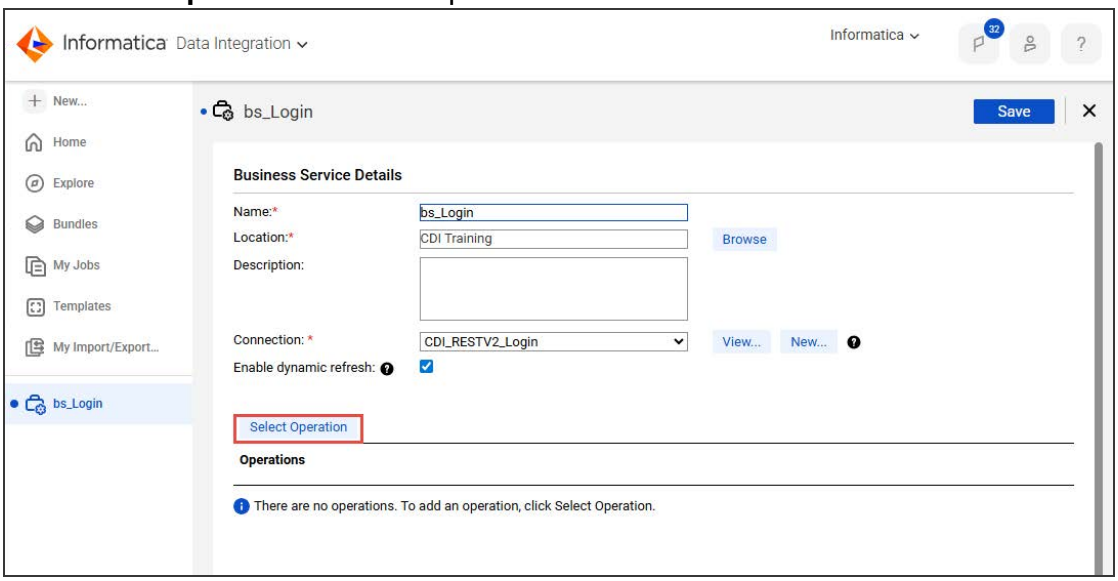
9. Set the following properties:

Name:	bs_Login
Location:	Your student folder
Connection:	CDI_RESTV2_Login
Enable dynamic refresh:	Checked

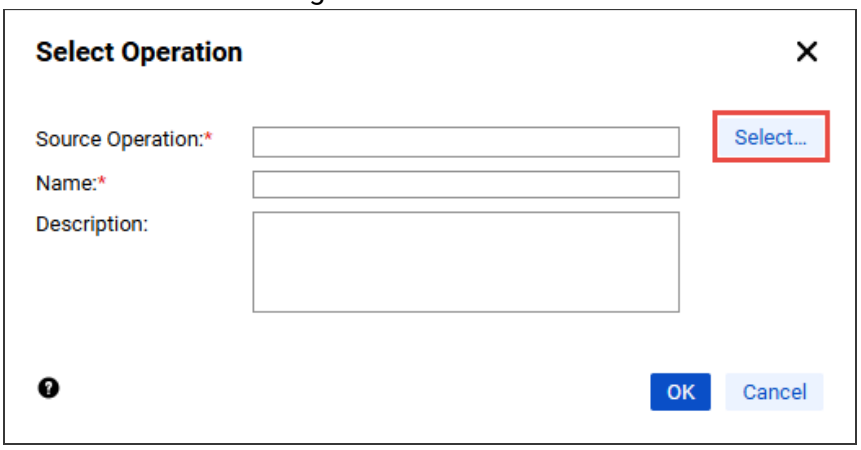


Enable dynamic refresh when checked, will refresh the connection metadata every time you run the task.

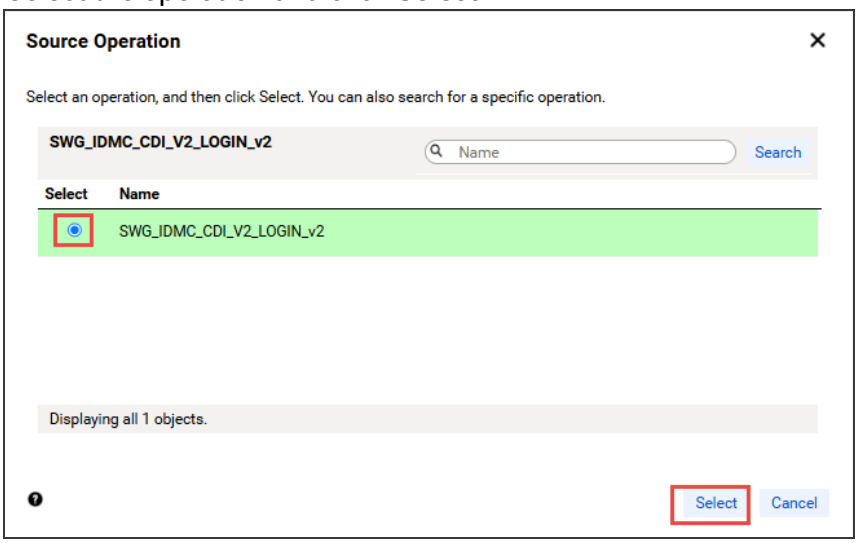
10. Click **Select Operation** to add an operation.



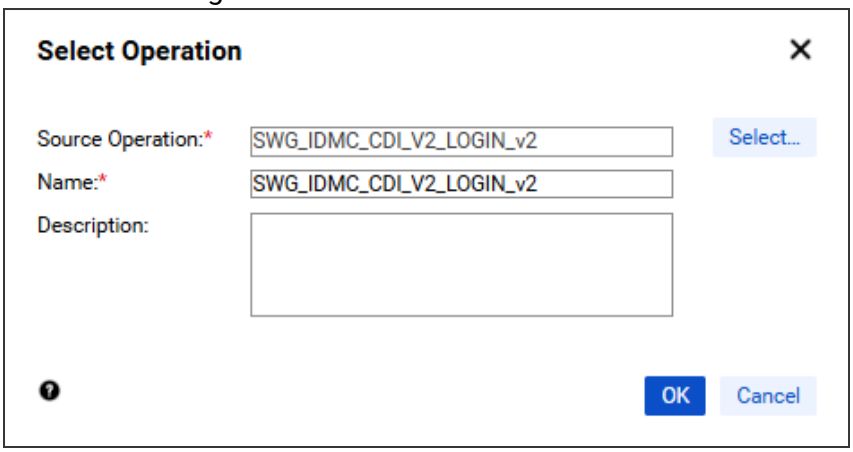
11. Click **Select** in the dialog.

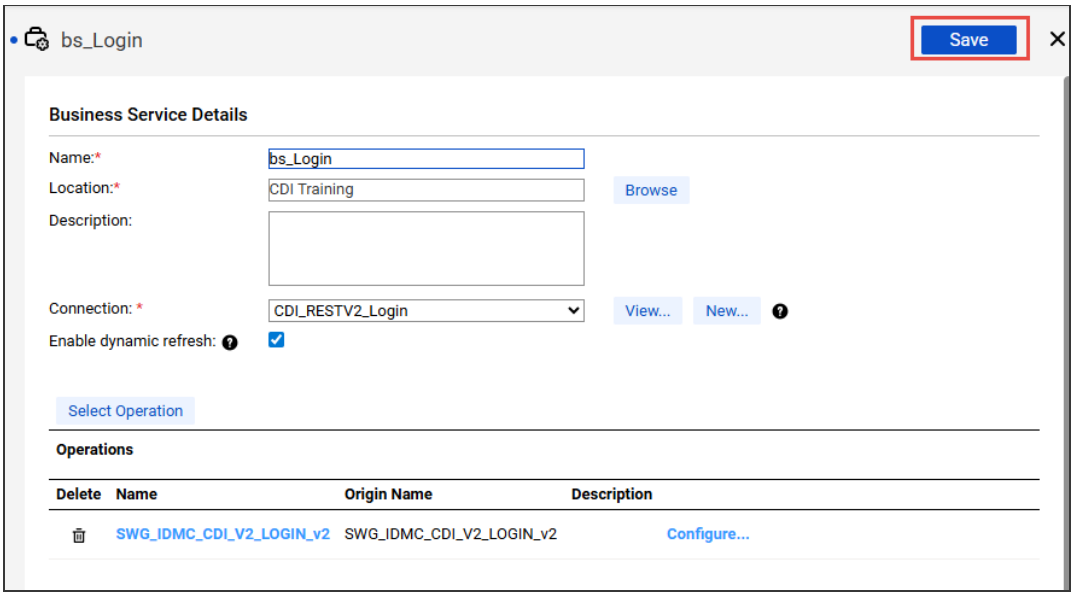


12. Select the operation and click **Select.**



13. Then click **OK** again and **Save** the business service.



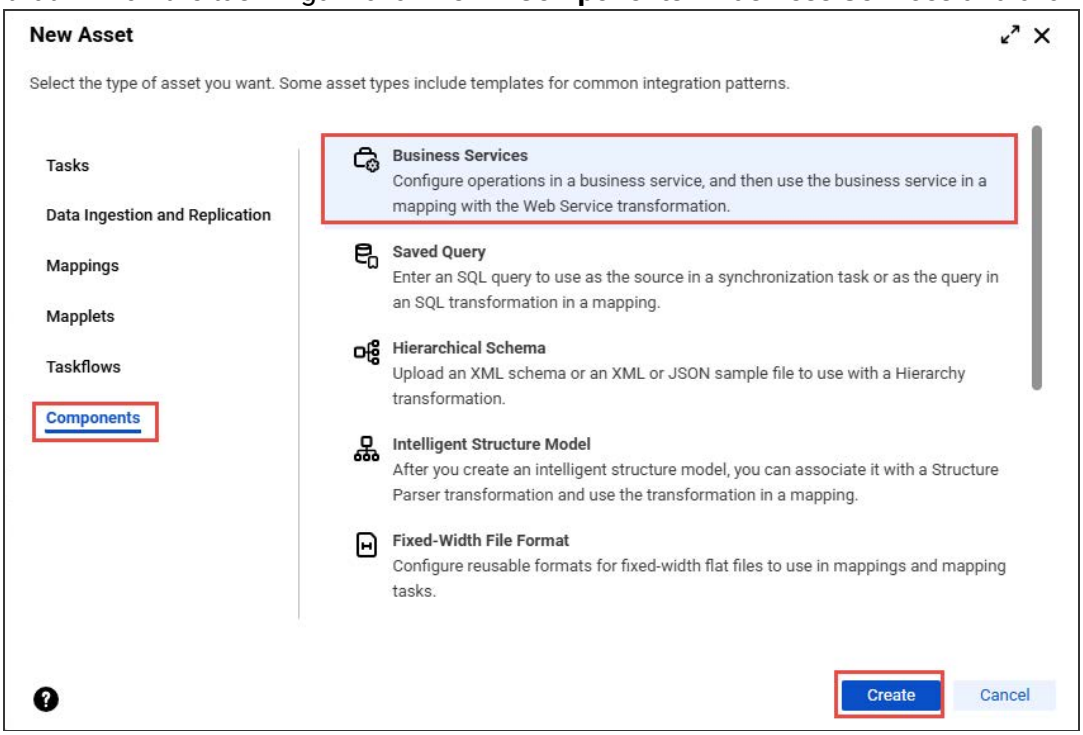


14. **Close** the business service.

Define the Run Job Business Service

This service can be used in a Web Service transformation in a mapping.

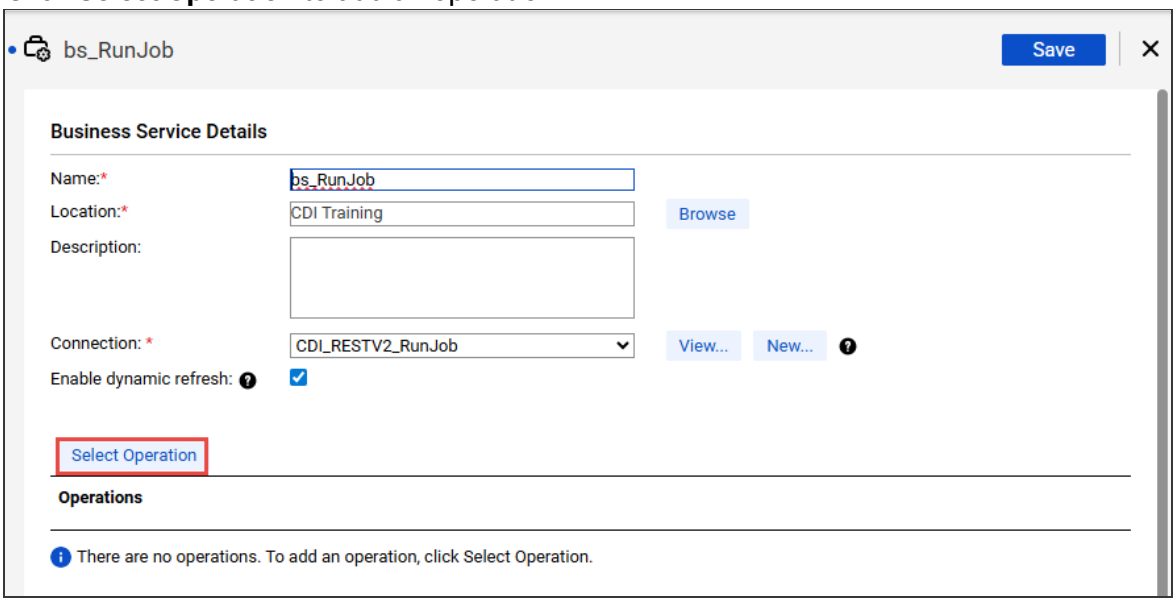
15. We have defined the login business service, now we need to define the business service that will run the task. Again click **New > Components > Business Services** and click **Create**.



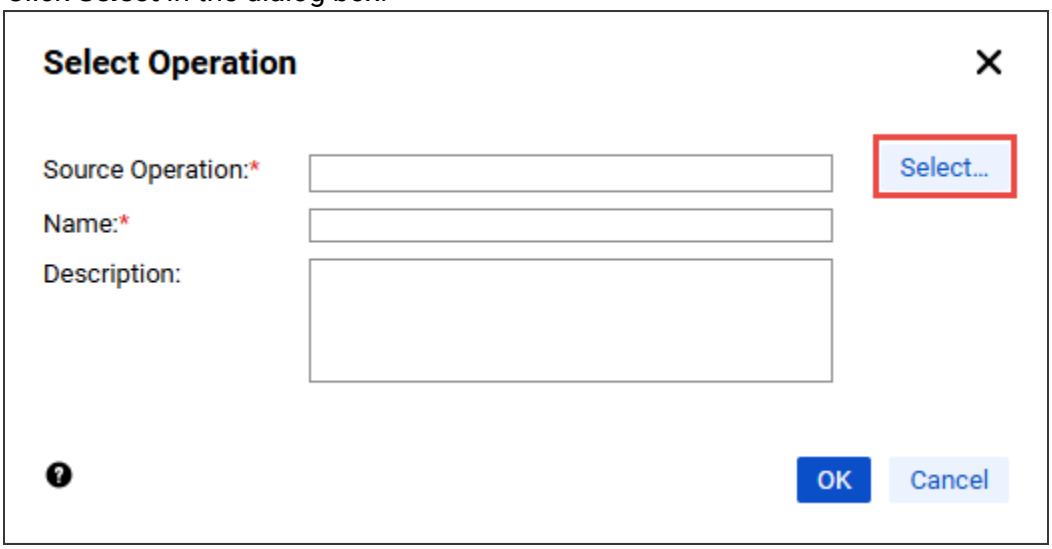
16. Set the following properties:

Name:	bs_RunJob
Location:	Your student folder
Connection:	CDI_RESTV2_RunJob
Enable dynamic refresh:	Checked

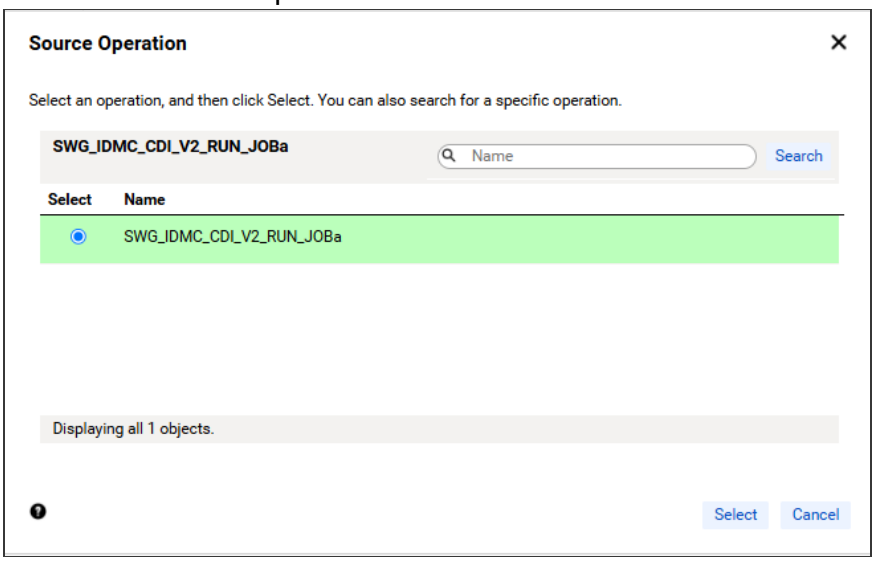
17. Click **Select Operation** to add an operation.



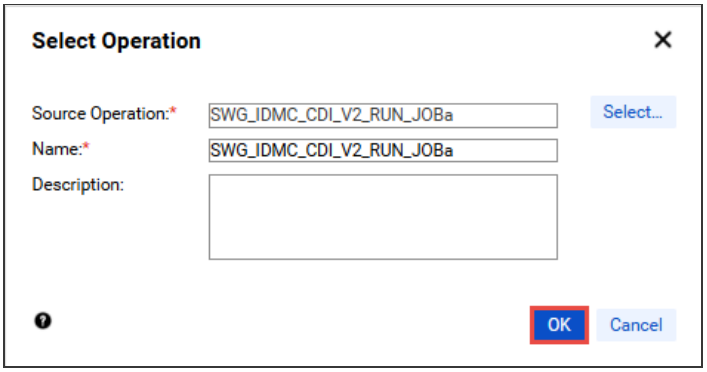
18. Click **Select** in the dialog box.

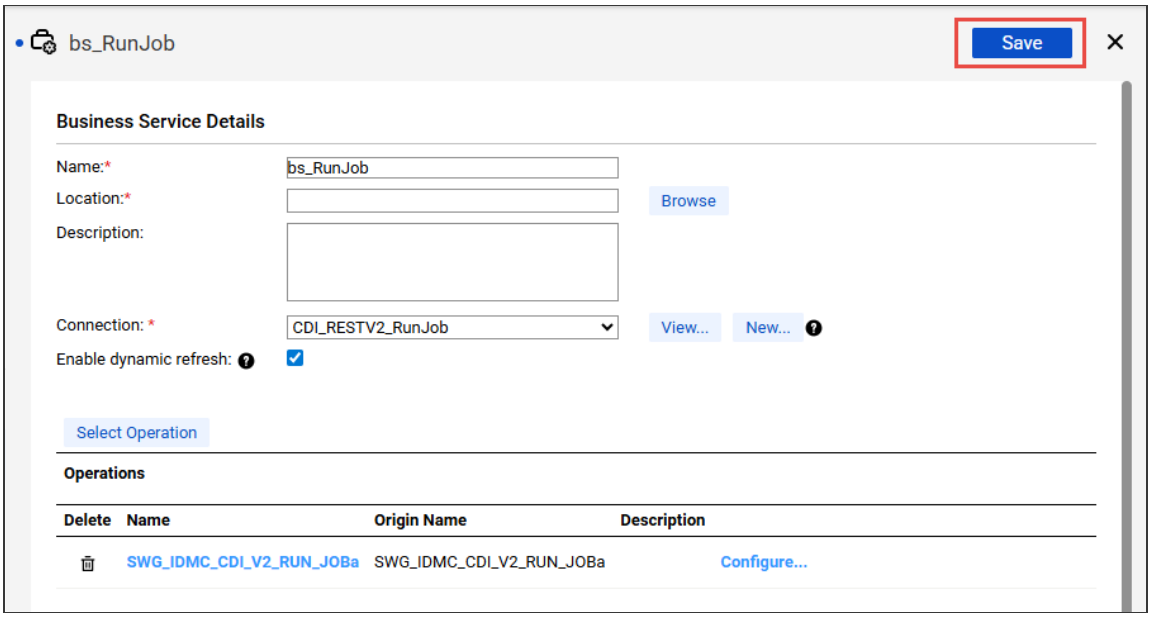


19. Select the **Run Job** operation and click **Select.**



20. Then click **OK** and **Save** the business service.

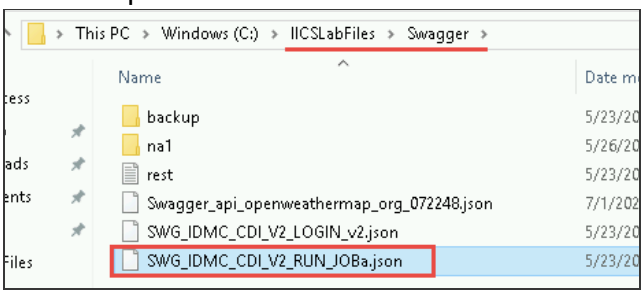


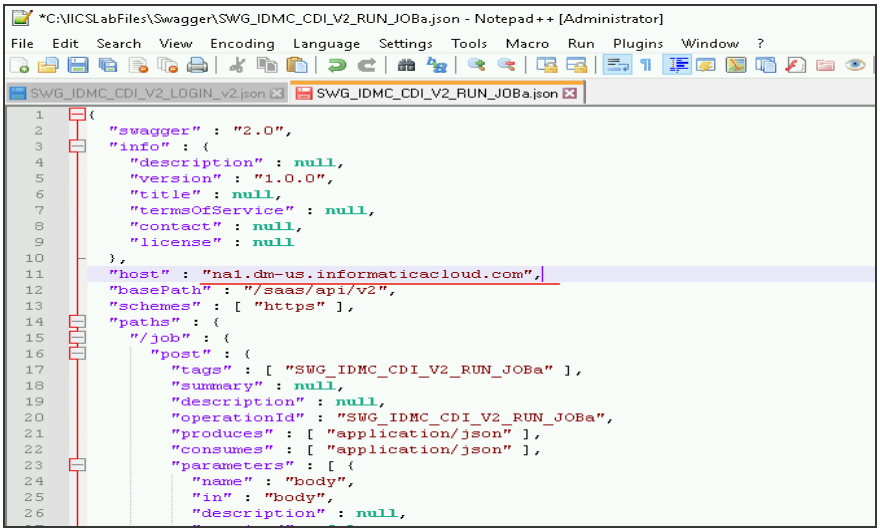


21. **Close** the business service.

If there is an issue selecting the operation for the ***run*** job, check the swagger file in the **IICSLabFiles > Swagger folder.** Ensure the host is set to the pod where your org has been defined. This is usw5 by default but can be updated if required.

For example:





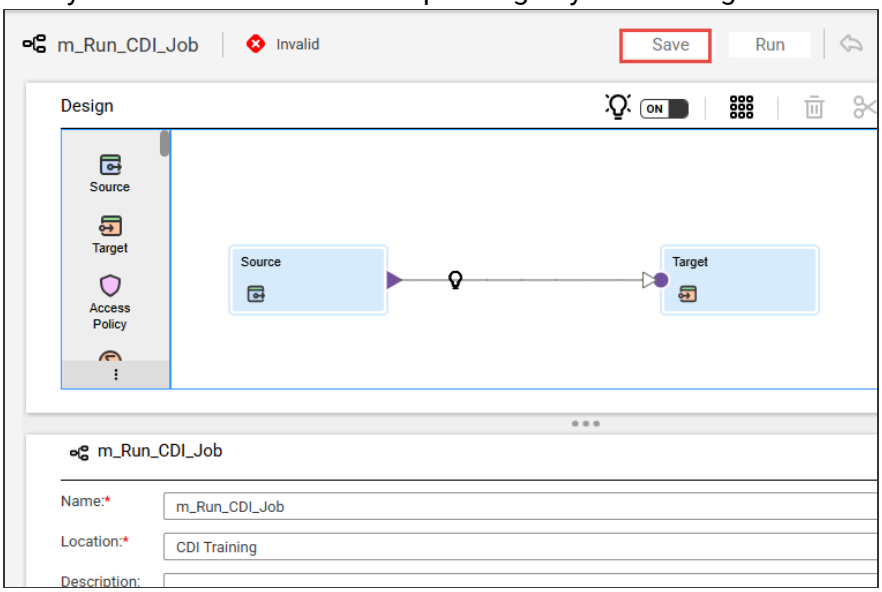
You can identify your pod using the https web address used to access your org - for example: https://***use4***.dm-us.informaticacloud.com/

Create a Mapping that will provide Login Credentials, Log into the Platform API and Run a Job

22. In the **Data Integration** service create a **New** > **Mappings > Mapping**.

23. In the Name field, enter **m_Run_CDI_Job**.

24. Verify that the asset Location is pointing to your working folder.



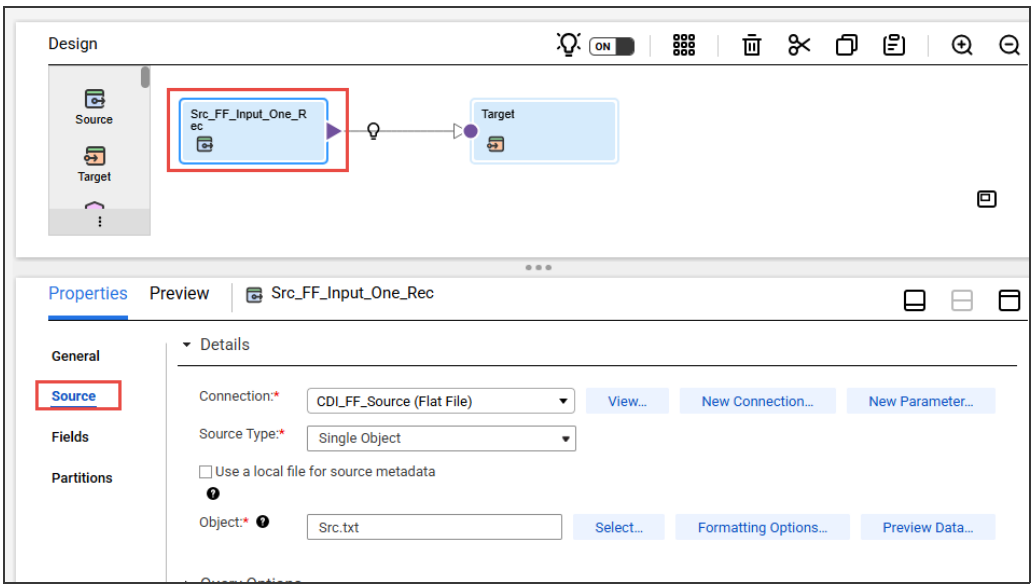
25. To configure the source, from the mapping canvas, click the **Source** transformation.

26. On the **General** tab of the Source properties, enter the Name as **Src_FF_Input_One_Rec**

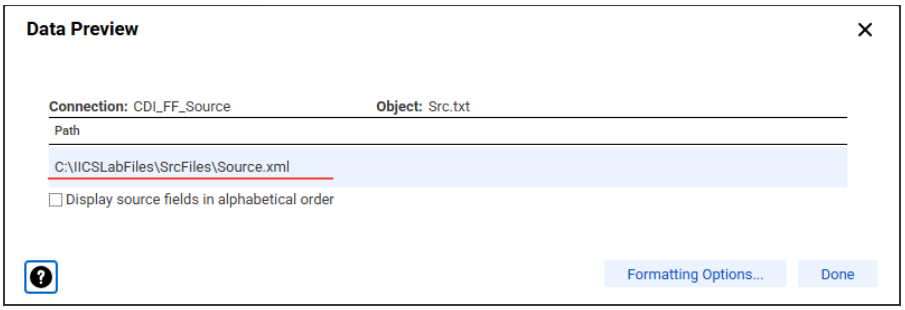
27. From the properties pane, click **Source** and from the Connection drop-down, select **CDI_FF_Source**

28. From the Object field, select **Src.txt** and save the mapping.

The mapping will not run if we don't put at least one record through it. The virtue of Src.txt as an input is that it has one record. We do not need the data inside Src.txt, just the existence of a record.



29. Click **Preview Data** to review the data in the source table. We can see that it consists of one record which is needed to put into the pipeline to start the process. We will not use the data in the file.

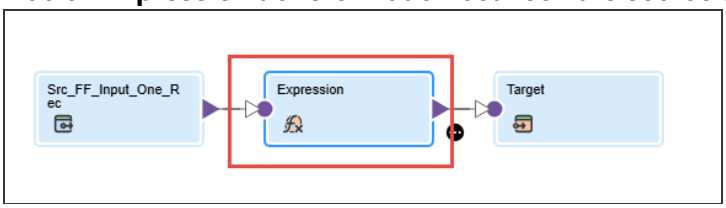


30. Click **Done** once complete.

Define an Expression to Hold the Login Credentials

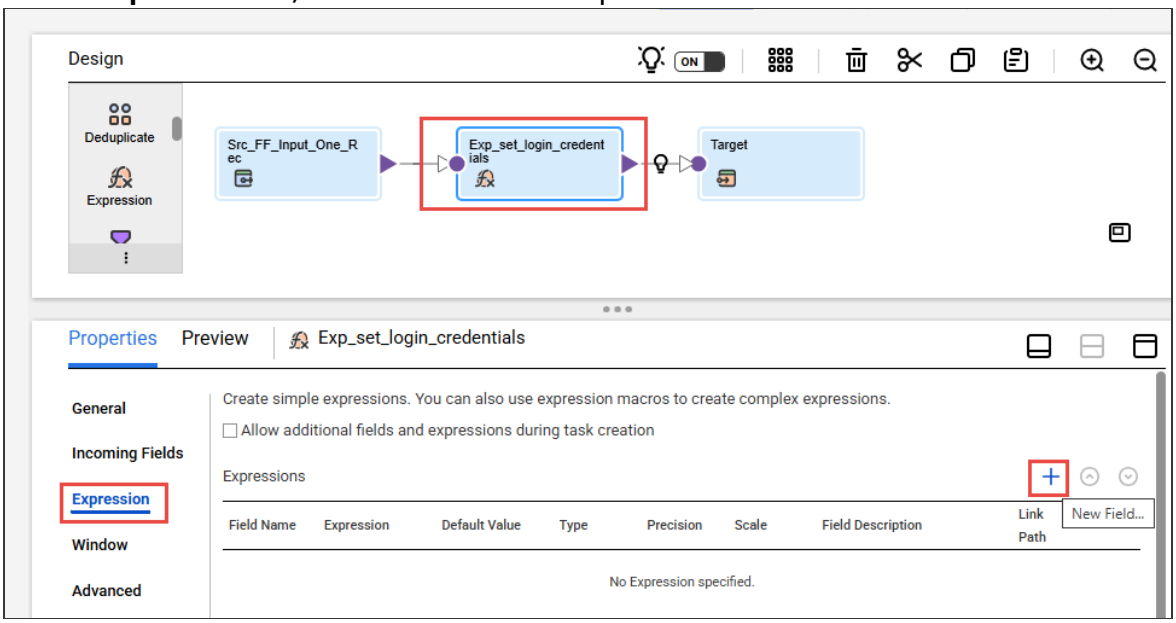
For the lab, we will configure an expression transformation to hold the username and password. This information will be passed on to the Web service transformation that will log into Informatica and run the job specified.

31. Add an **Expression** transformation between the source and the target transformation.



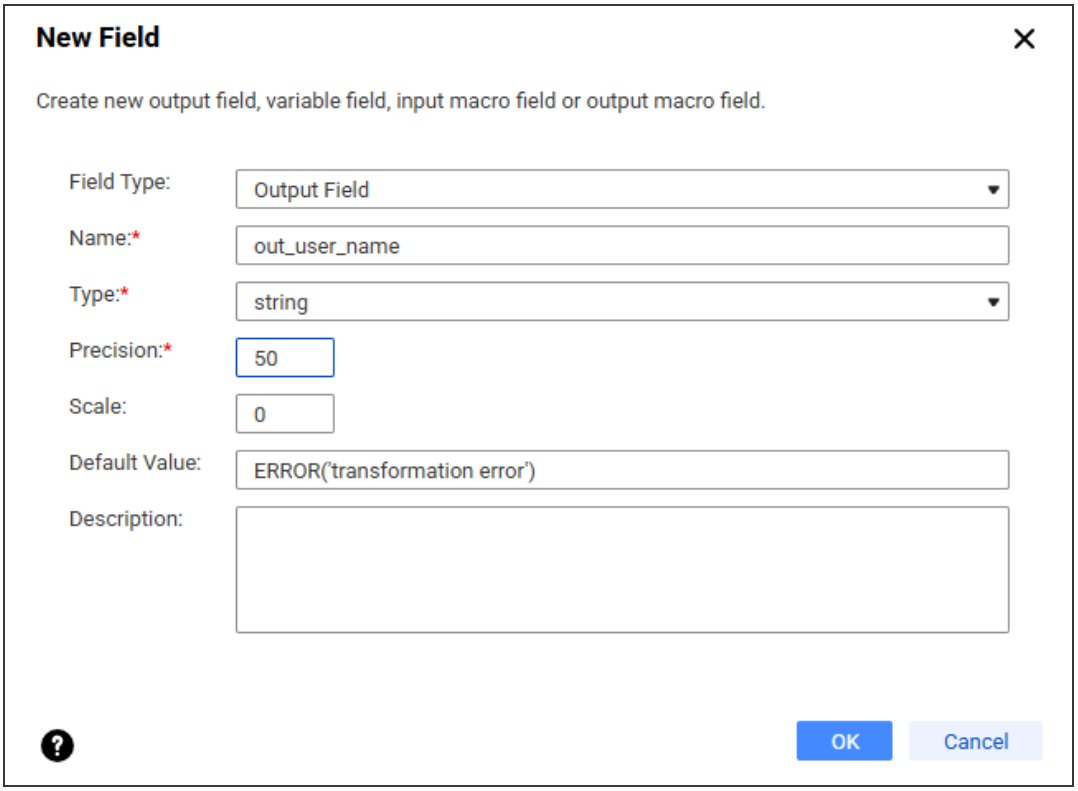
32. On the Properties > **General** tab, set the Name as **Exp_set_login_credentials.**

33. On the **Expression** tab, click **+** to add a new expression.

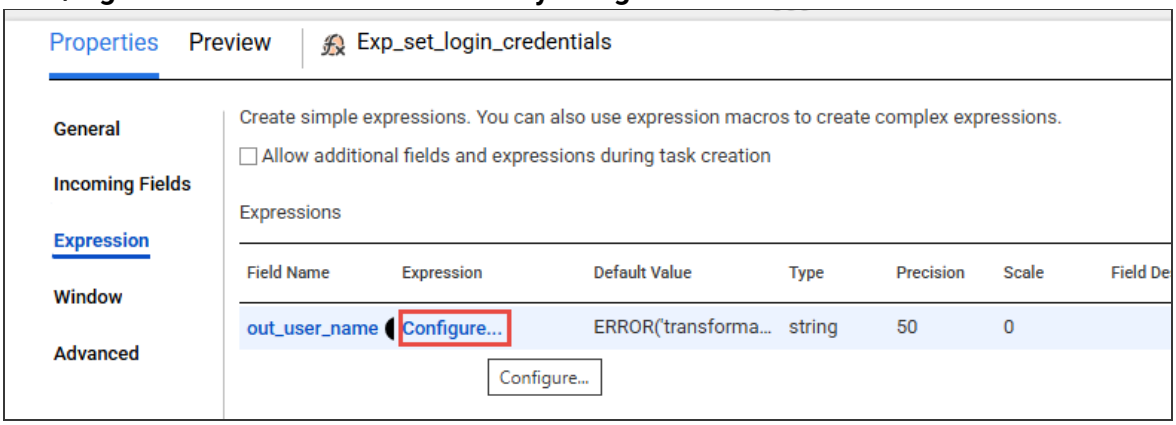


34. Set the following properties:

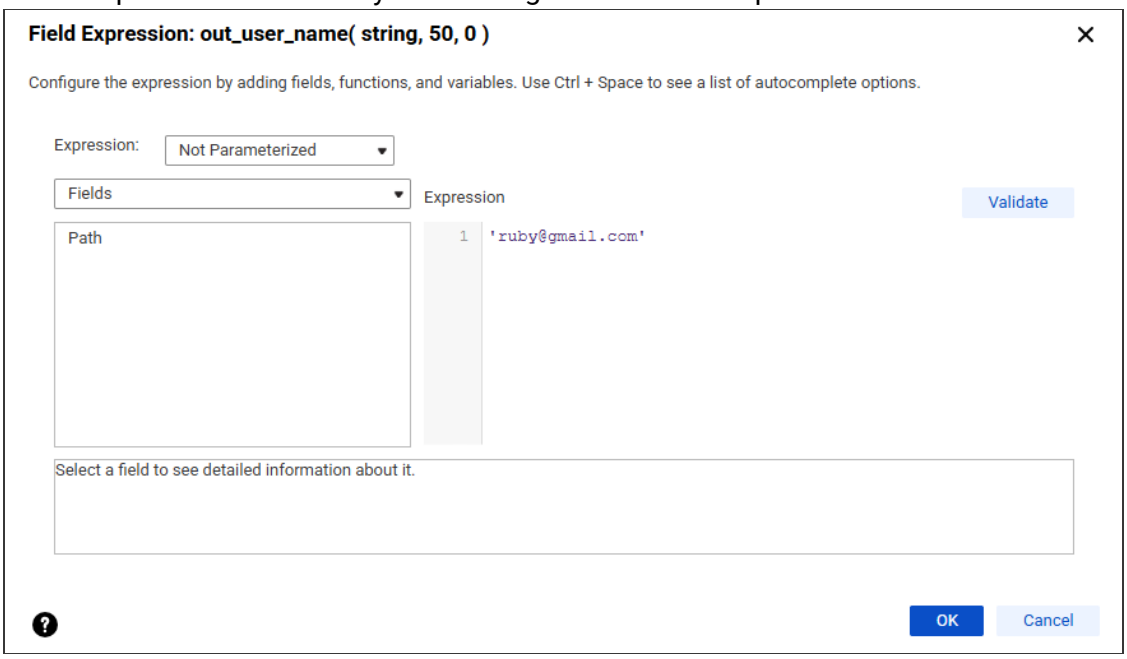
Field Type:	Output Field
Name:	out_user_name
Type:	String
Precision:	50



35. Click **OK** and click **Configure** to define the expression. For this example we will set our **user/login names. This is the name that you log into Informatica Cloud with**.



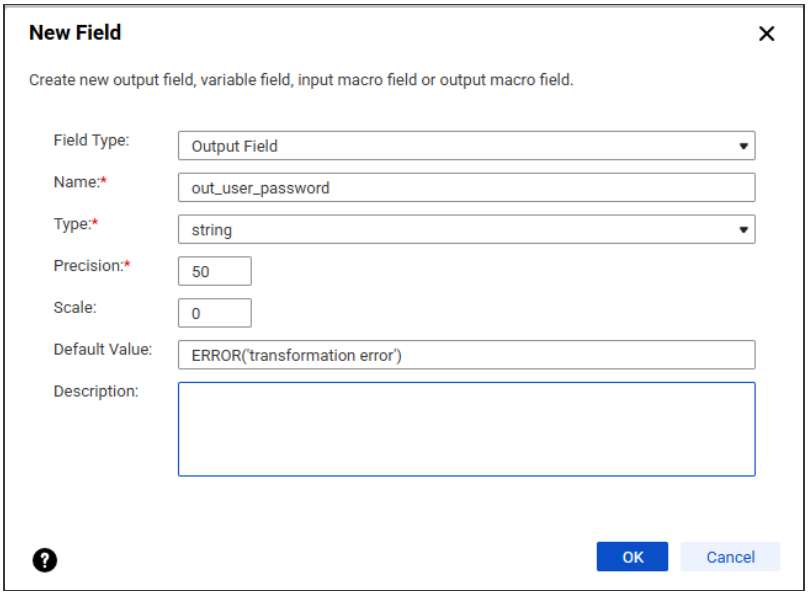
36. In the expression area enter your user login in" – for example:



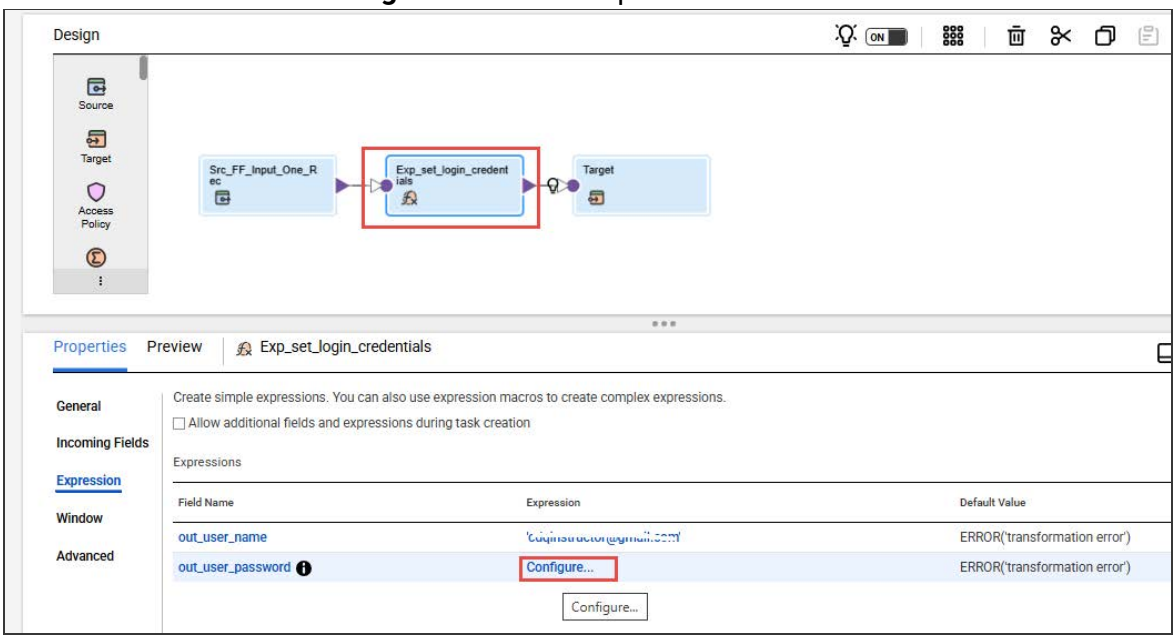
37. Click **OK**.

38. On the **Expression** tab add another output field. Set the following properties:

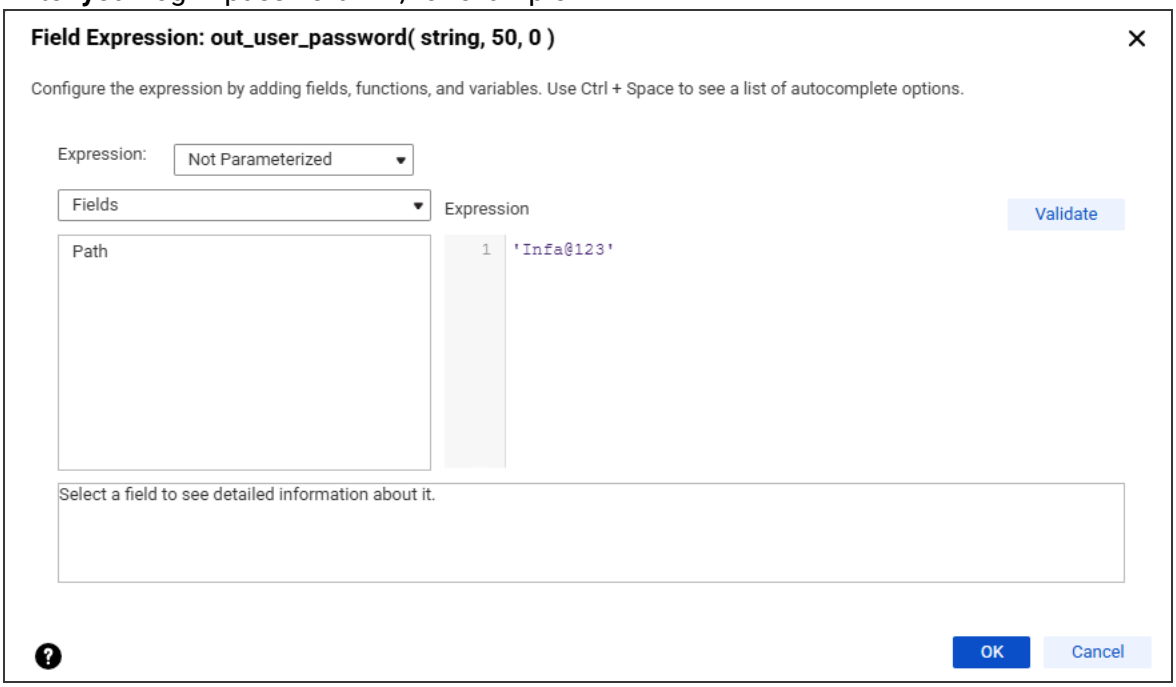
Field Type:	Output Field
Name:	out_user_password
Type:	String
Precision:	50



39. Click **OK** and then click **Configure** to define the password.



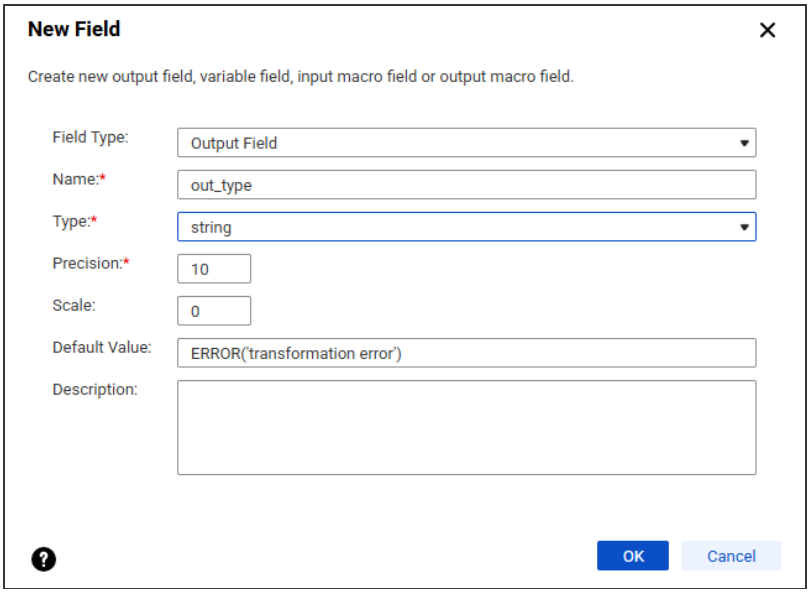
40. Enter **your** log in password in ”, for example:



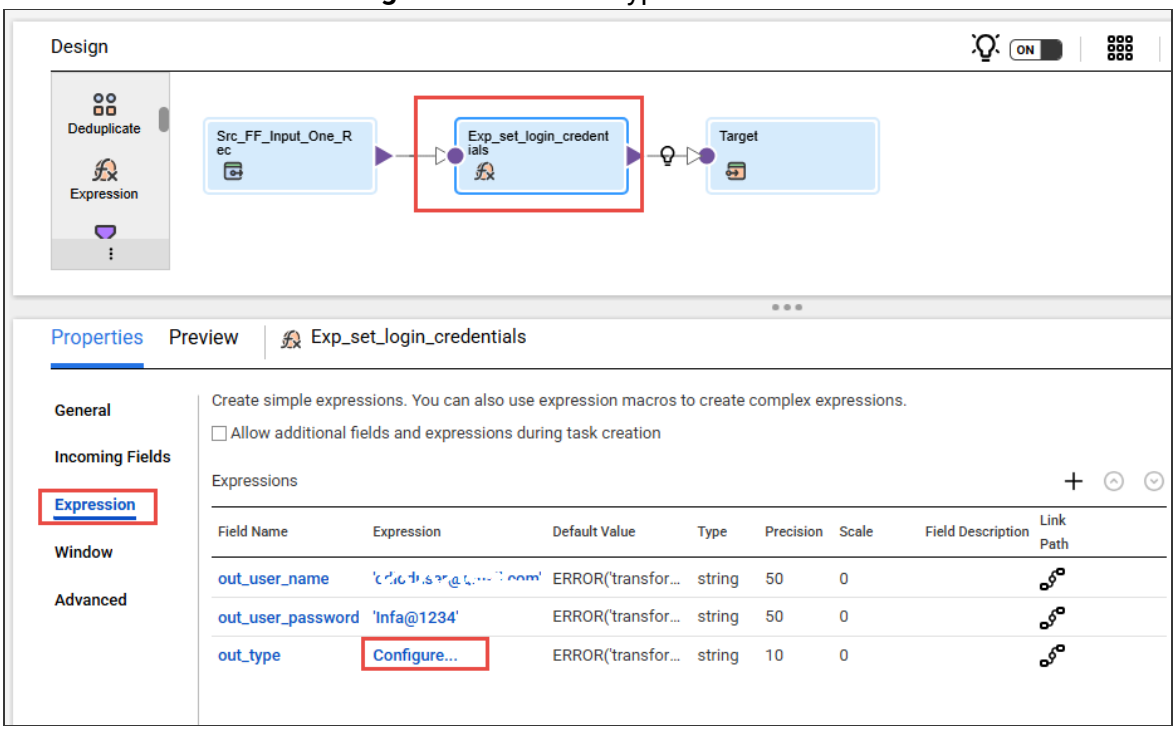
41. Click **OK**.

42. On the **Expression** tab add another output field. Set the following properties:

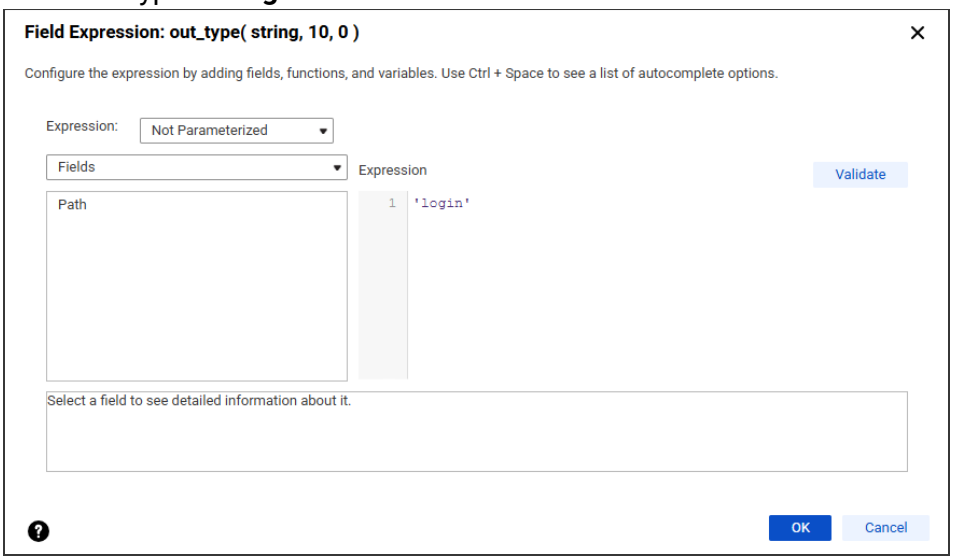
Field Type:	Output Field
Name:	out_type
Type:	String
Precision:	10



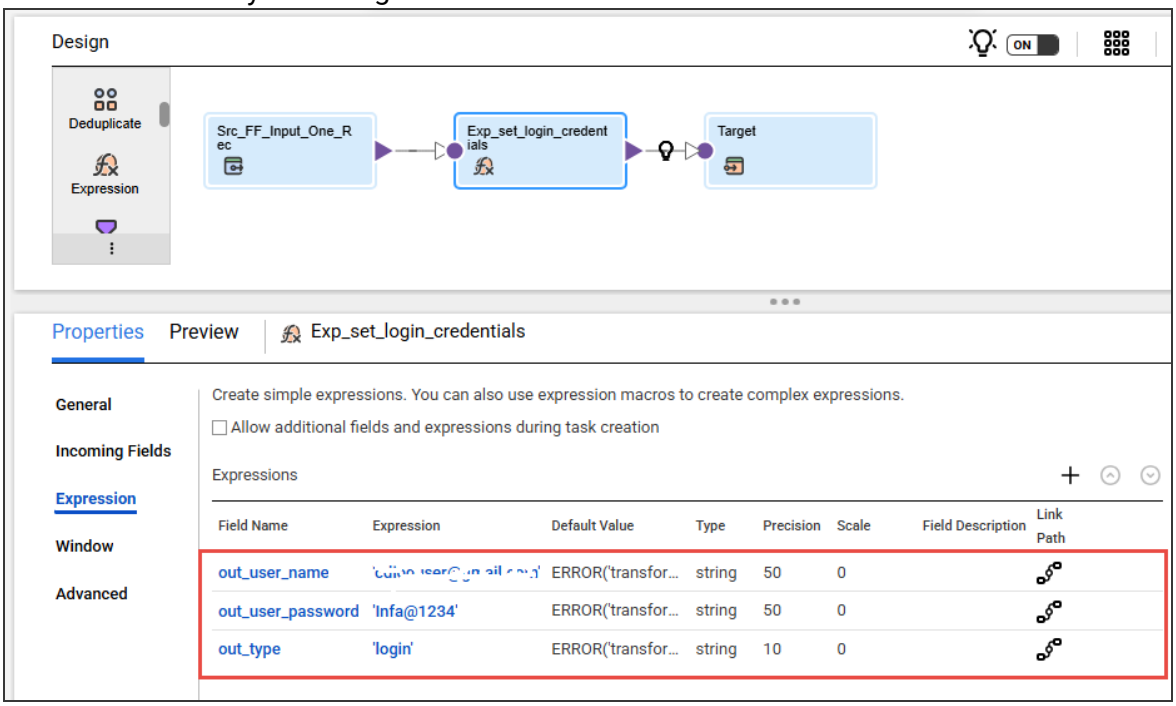
43. Click **OK** and then click **Configure** to define the type.



44. Enter the type as **'login'**



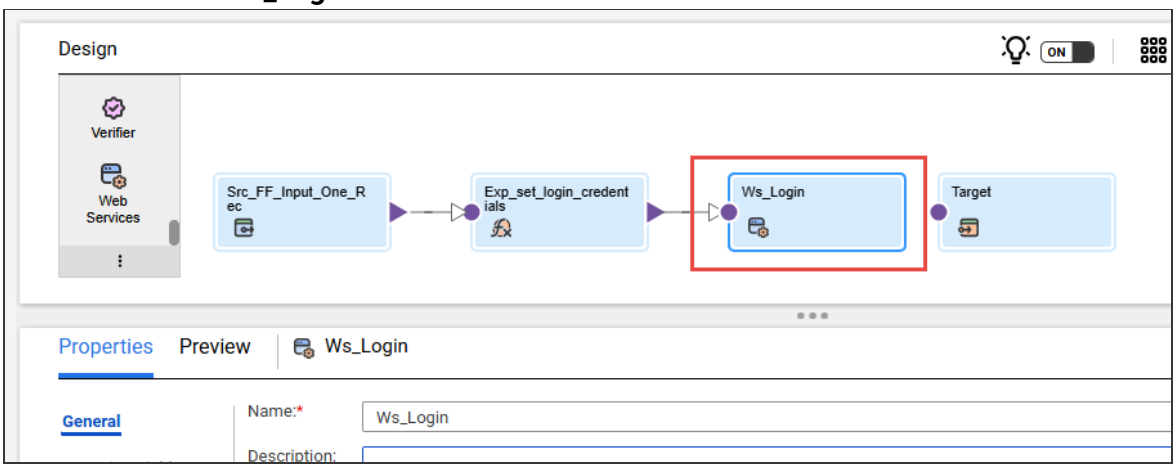
45. Click **OK** and **save** your changes.



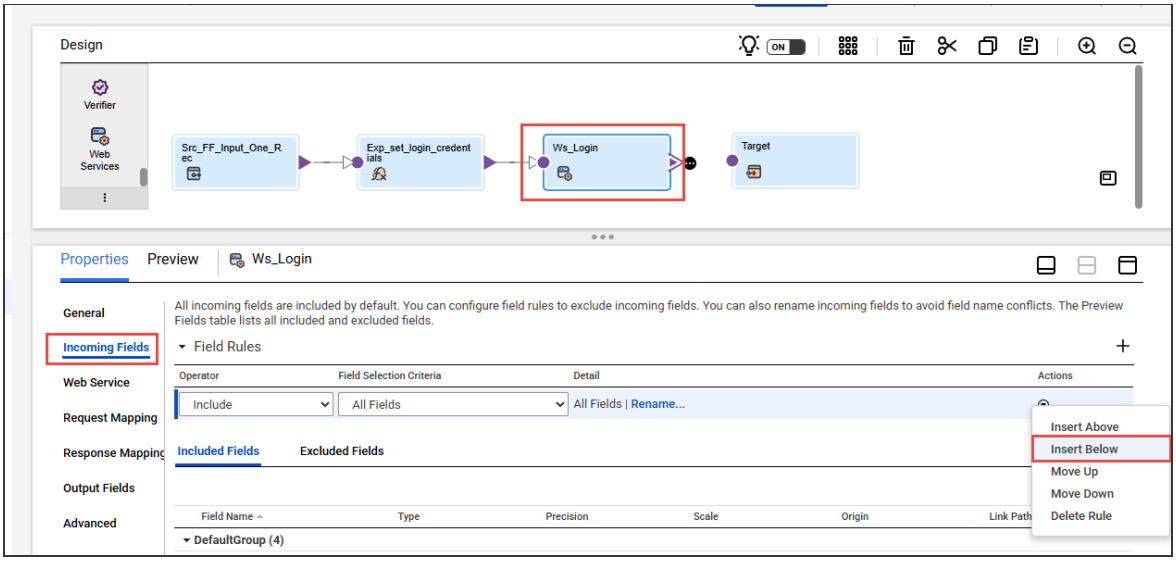
Configure a Web Service Transformation to Use the Business Service for Login with Credentials from the Expression Transformation

46. Add a **web services** transformation to the mapping between the Exp_set_login_credentials and the Target.

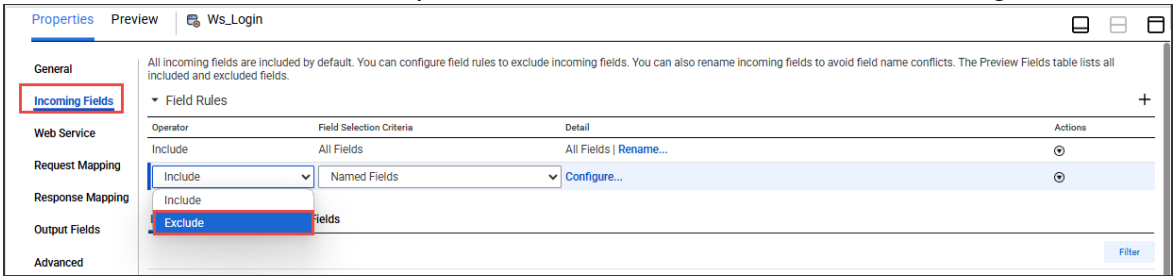
47. Set the name as **Ws_Login**.



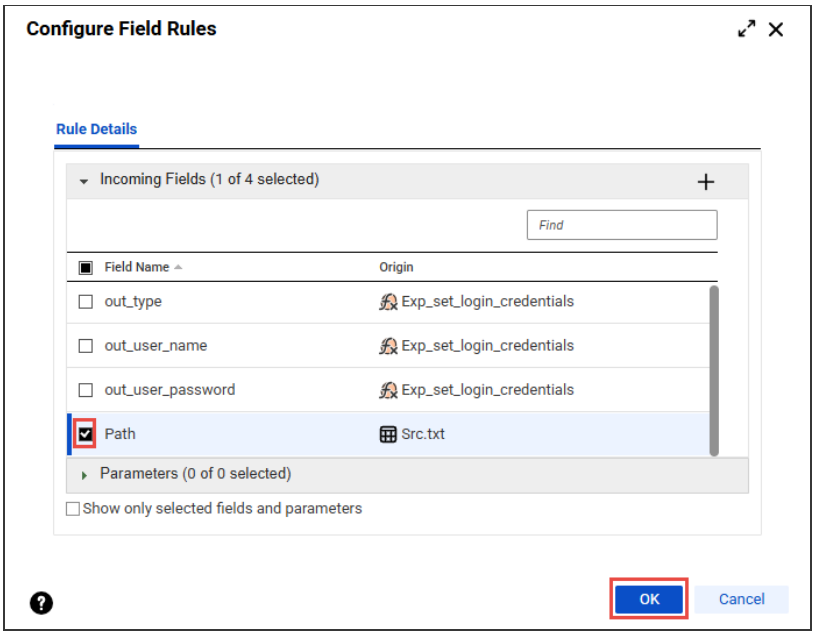
48. On the **Incoming Fields** tab we want to exclude a field. Click on the row under **Field Rules** and from the **Actions** menu, choose **Insert Below**.

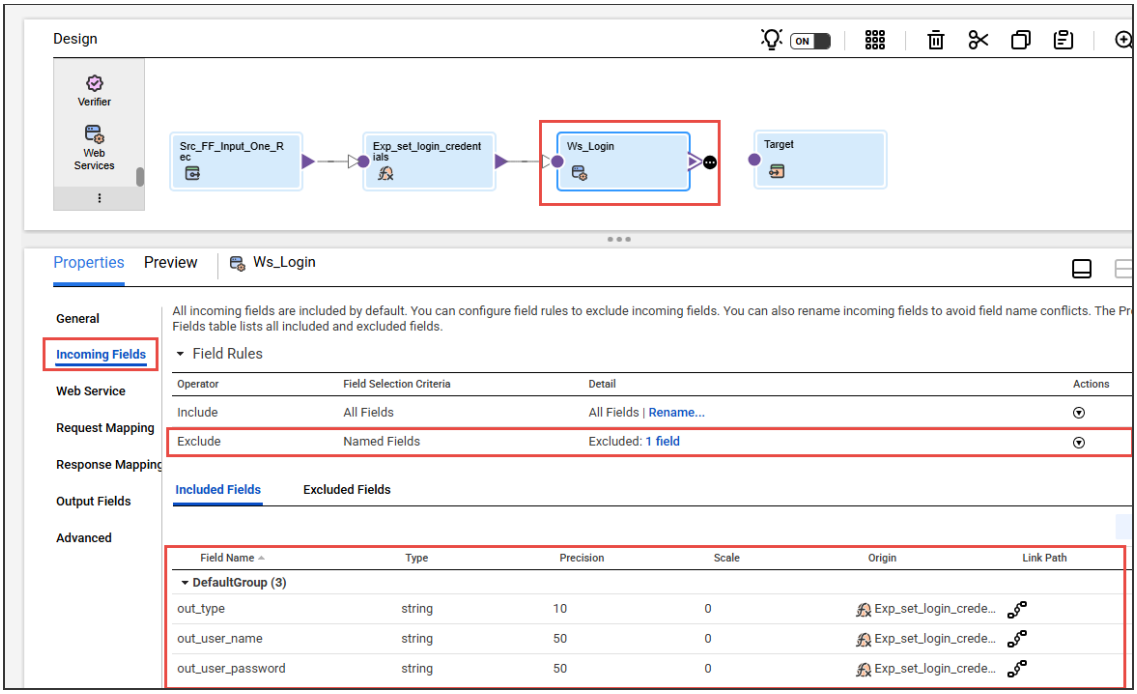


49. Select the new row and under Operator, choose **Exclude** and then click **Configure..**

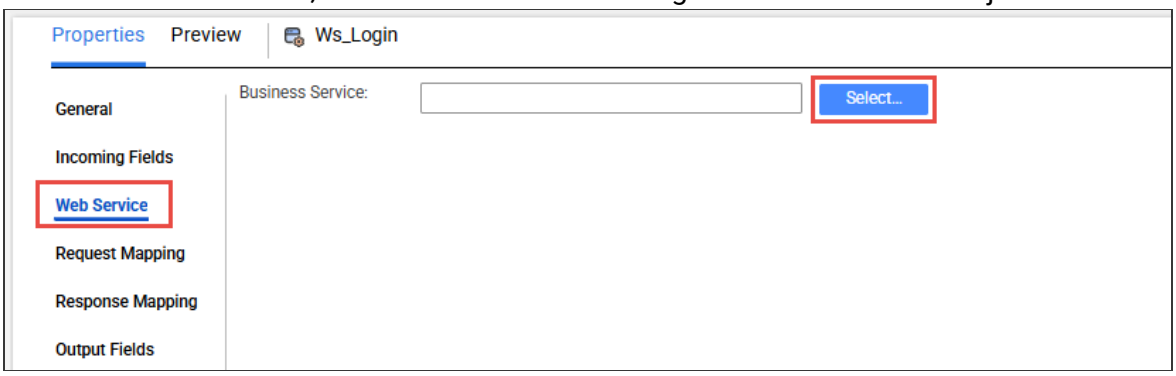


50. Exclude the **Path** field from the source. We do not need this. Click **OK**.



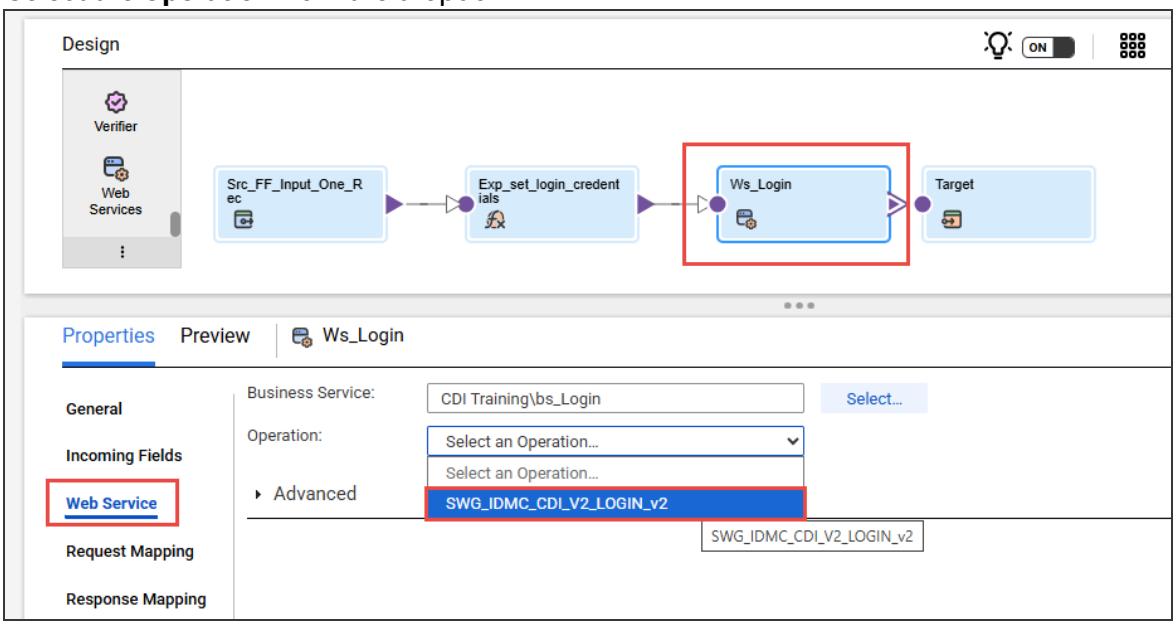


51. On the **Web Service** tab, click **Select** to select the login business service we just defined.

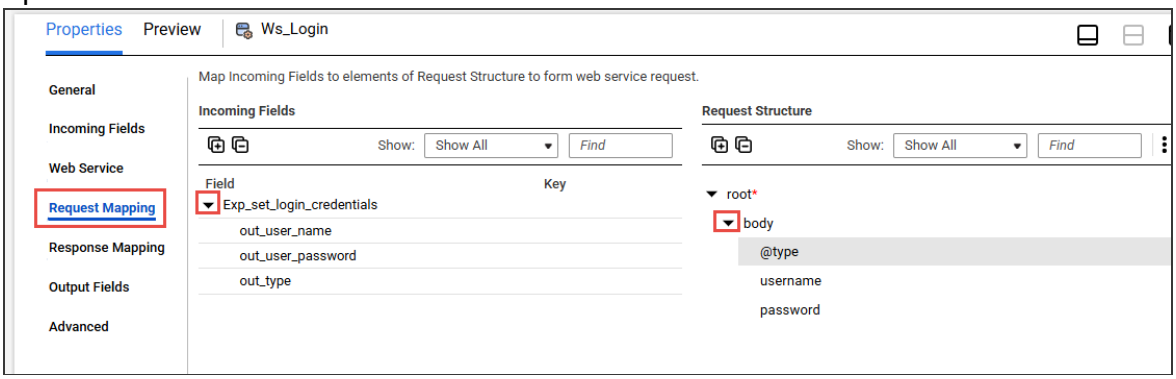


52. Navigate to your folder and select the **bs_Login** business service defined and click **Select**.

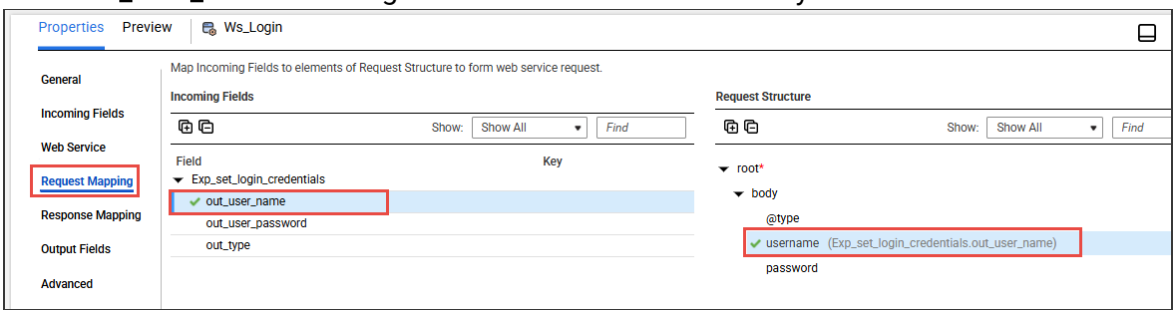
53. Select the **Operation** from the dropdown.



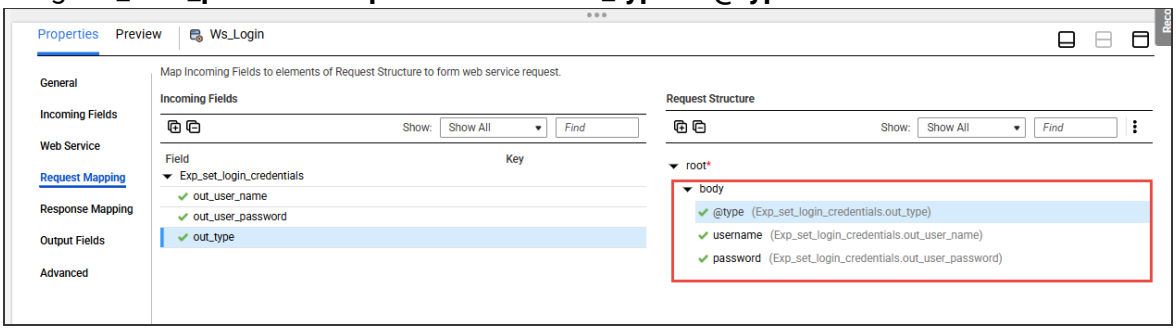
54. On the **Request Mapping** tab, expand the **Incoming Fields** and the **body** in the **Request Structure**. We can pass the login information to the web service for the business service operation.



55. Click **out_user_name** and drag it over to **username** under body.

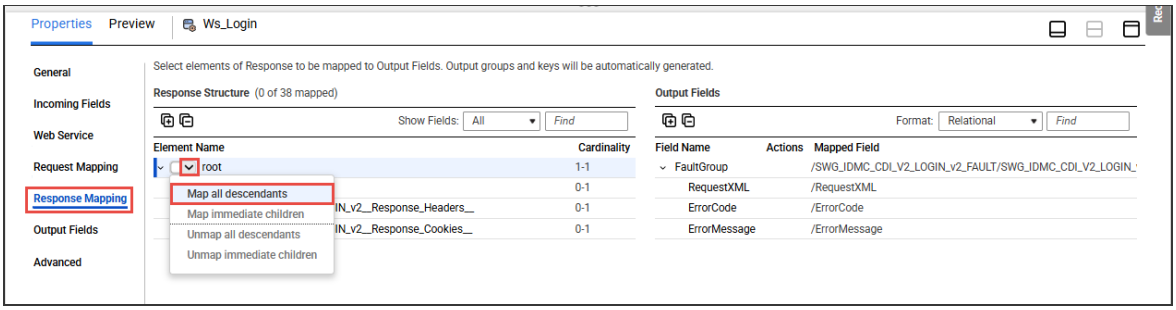


56. Drag **out_user_password** to **password** and **out_type** to **@type**.

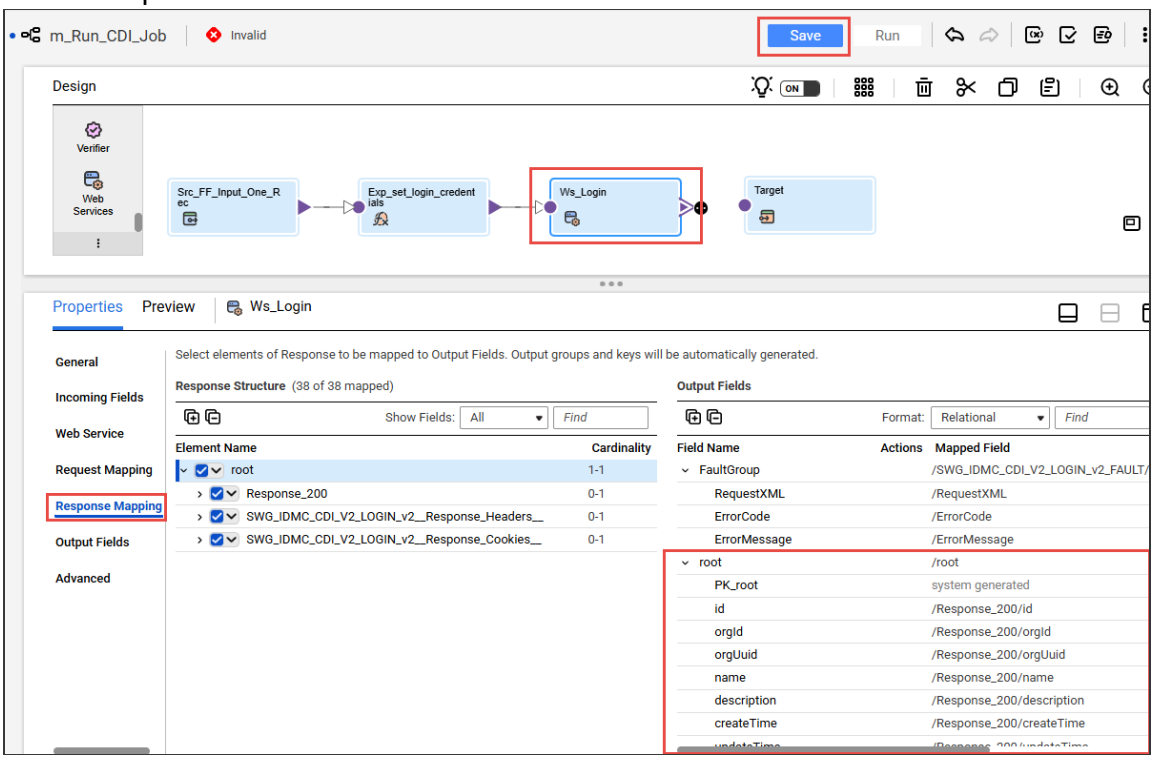


57. Move to the **Response Mapping** tab. This is where we can choose/restrict the response fields we receive back.

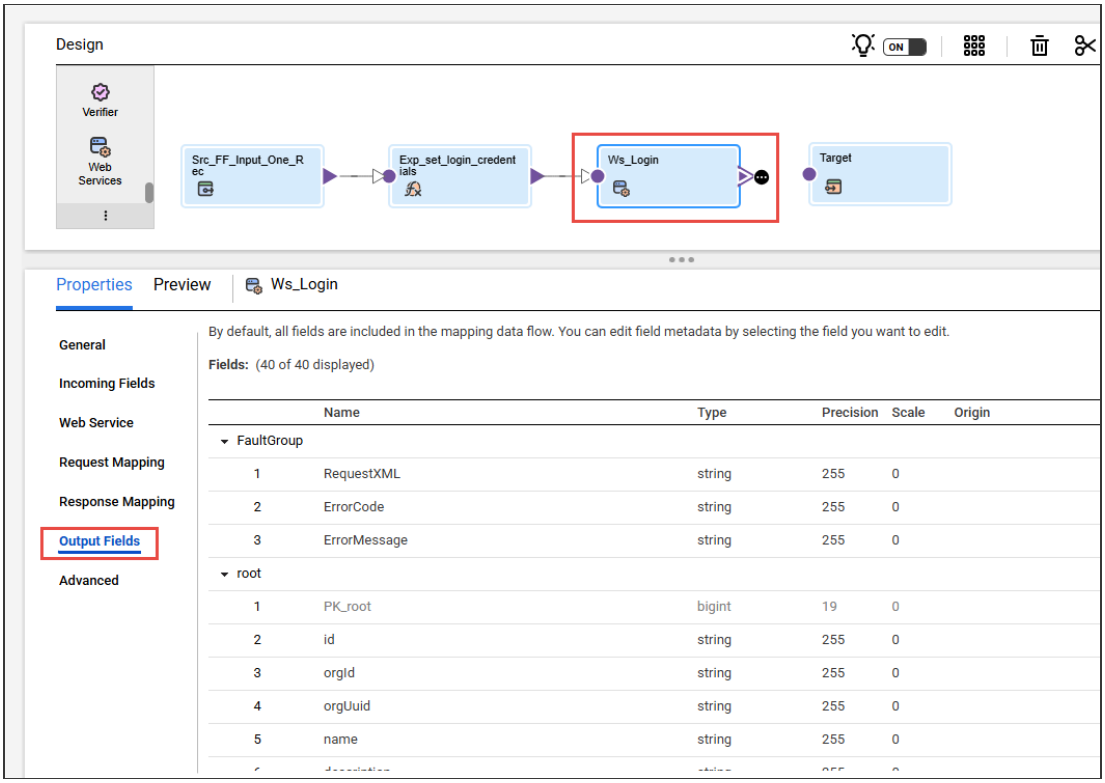
58. Under the Element Name settings, click on the root dropdown and choose to **Map all descendents.**



59. **Save** the updates.

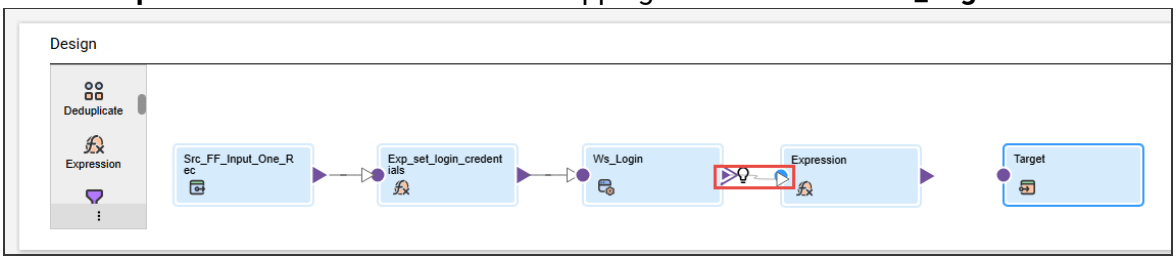


60. Click the **Output Fields** tab and review the output groups and fields.

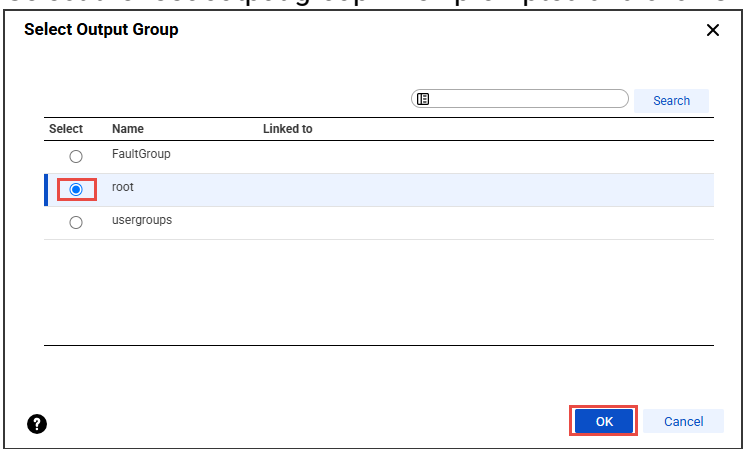


Configure an Expression Transformation to Define the Job Information

61. Add an **Expression** transformation to the mapping and connect the **ws_Login** to it.

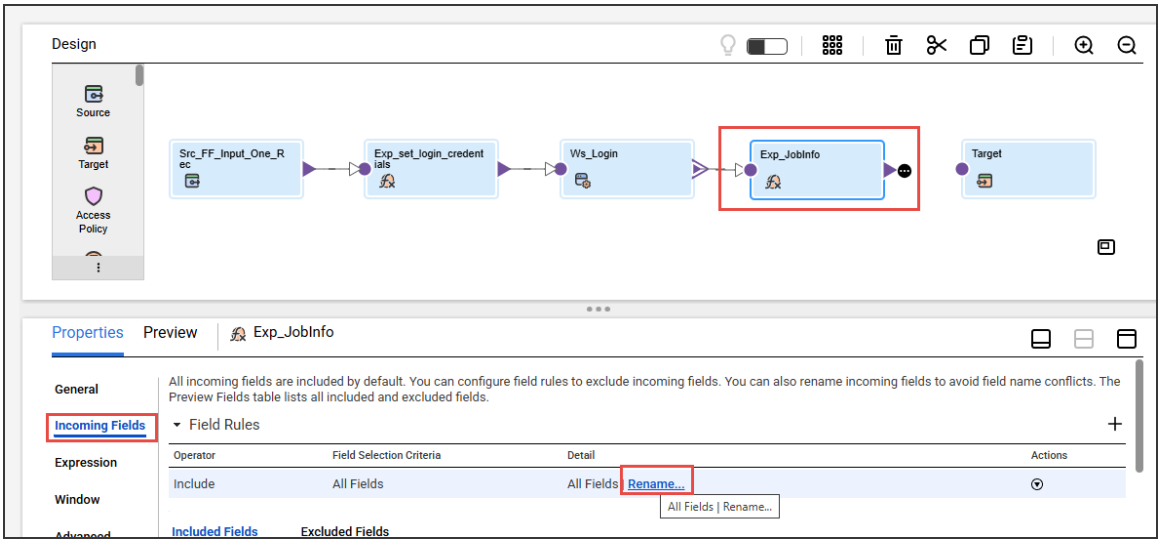


62. Select the **root** output group when prompted and click **OK**.



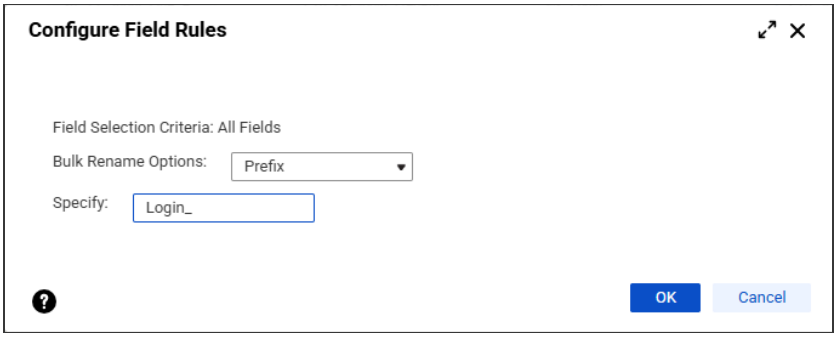
63. Set the name as **Exp_JobInfo.**

64. Move to the **Incoming Fields** tab and click **Rename** to rename all incoming fields. This will prevent conflicting name issues and makes it easier to understand where the fields have come from.

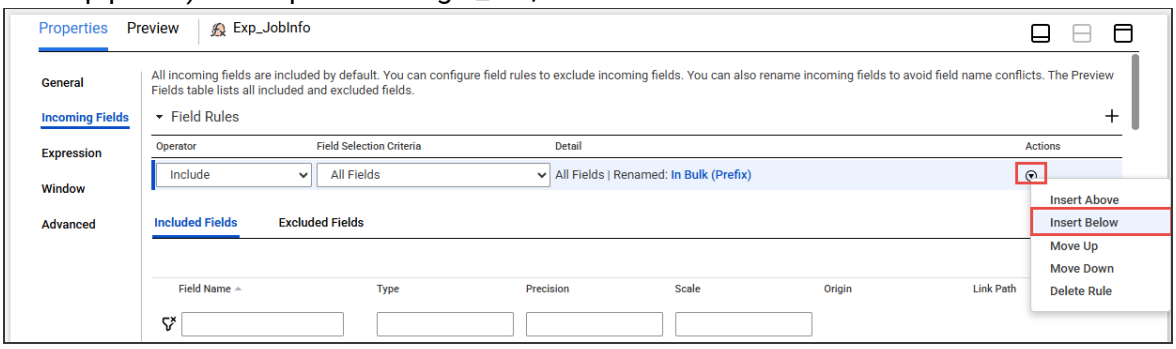


65. Bulk rename all incoming fields with a Login_ prefix by selecting **Prefix** from the Bulk Rename Options and enter **Login_** in the Specify field.

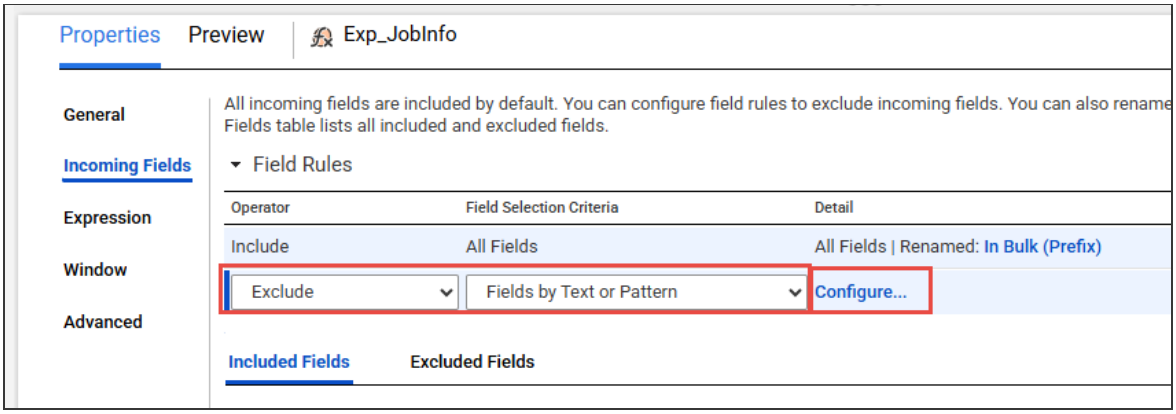
66. Then click **OK**.



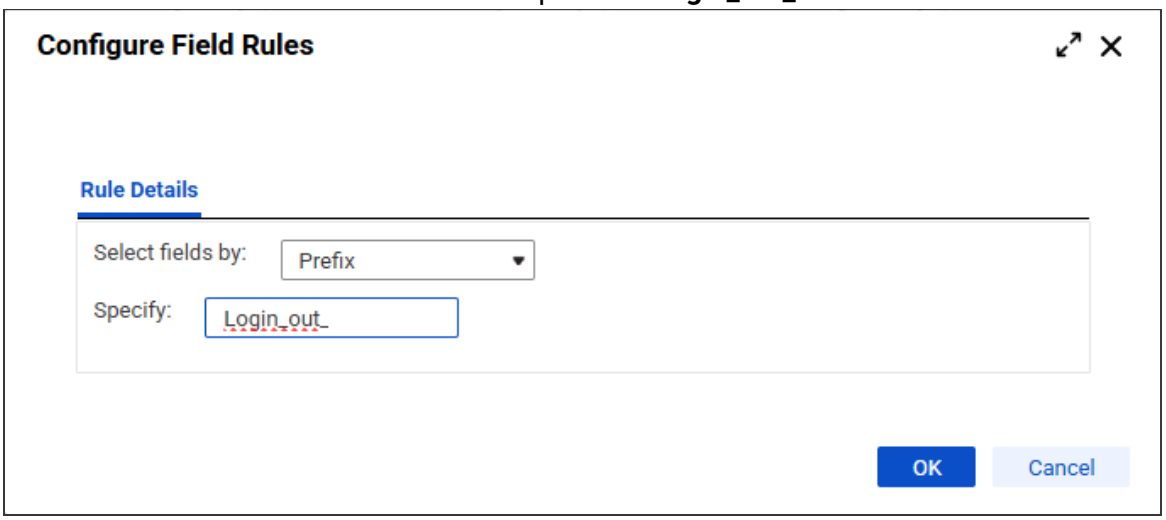
67. We want to use an Incoming fields exclude rule to eliminate all the output fields from the Login web service (they are not relevant to us and it limits the fields heading downstream in the pipeline) with a prefix of Login_out, so **add another Field Rule** below the Include one.



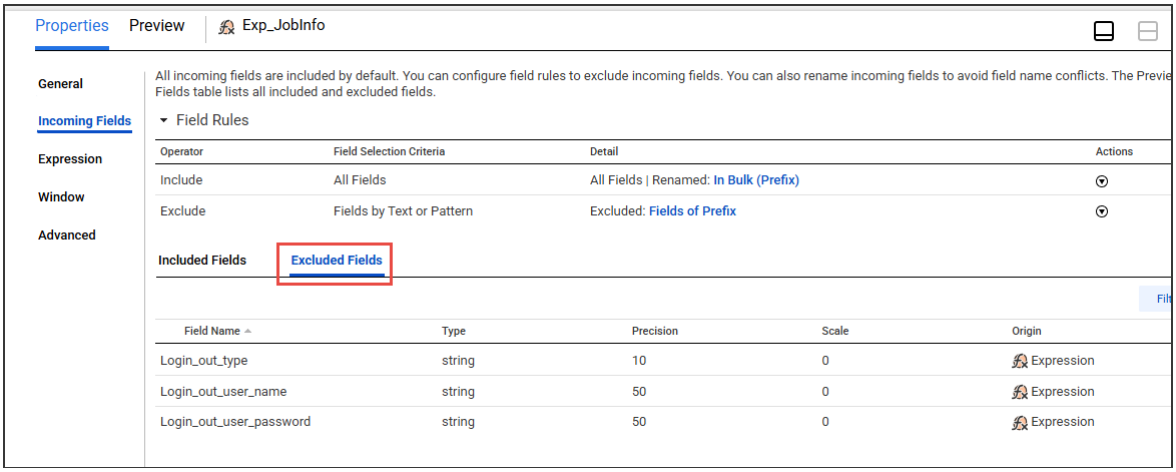
68. Set the Operator to **Exclude** and from the Field Selection Criteria, choose **Fields by Text or Pattern** from the dropdown and click **Configure** ….



69. Add the field rule to exclude fields with a prefix of **Login_out_** and click **OK.**

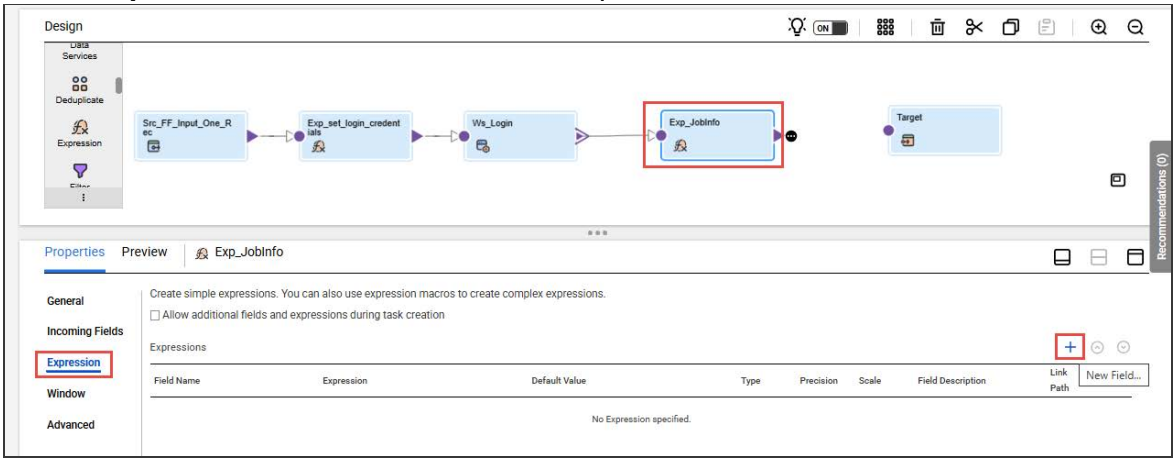


70. The excluded fields can be seen on the **Excluded Fields** tab below.



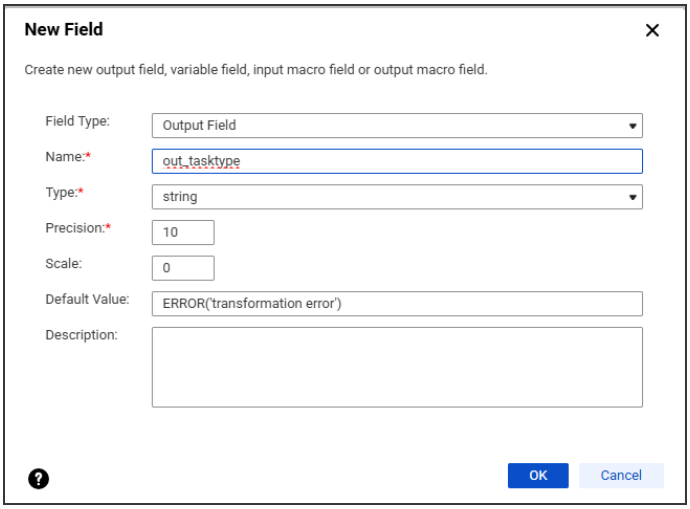
Define the Job Settings

71. On the **Expression** tab, click **+** to add a new expression.



72. Set the following properties:

Field Type:	Output Field
Name:	out_tasktype
Type:	String
Precision:	10



73. Click **OK** and click **Configure** to define the expression. For this example we will set the task type.

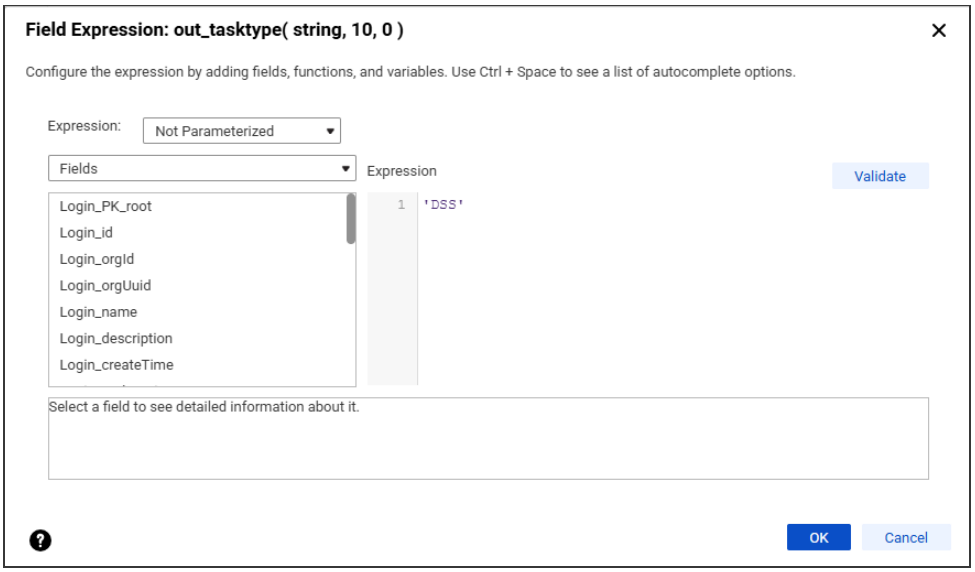
74. In the expression area enter **'DSS'** to run a Data Synchronization task. (For more information check the REST API Reference Guide – see examples below)

```
/api/v2/job
```

The following table describes the attributes to use in a job object:

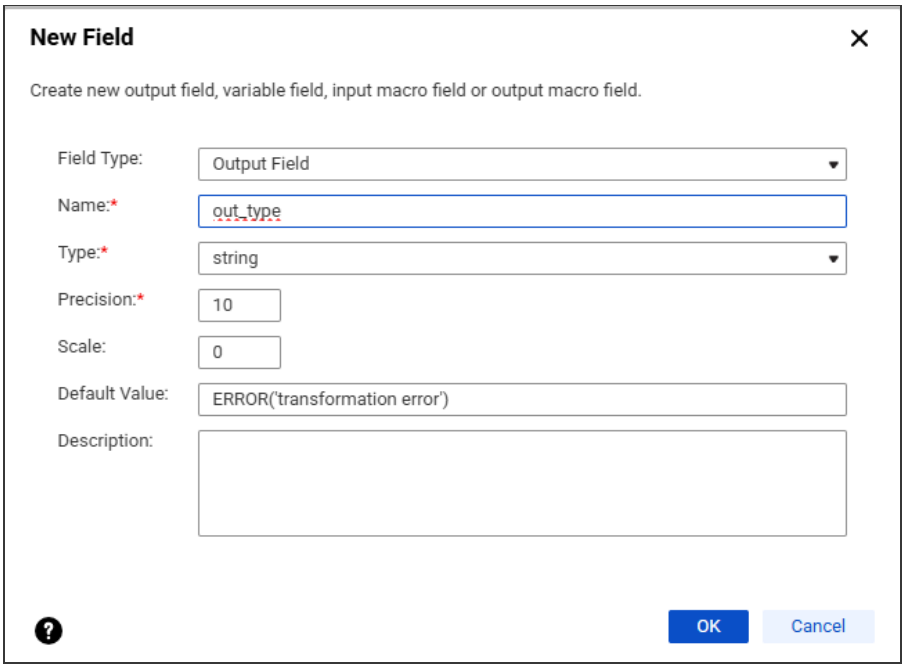
Field	Type	Required	Description
taskId	String	Required if taskName or taskFederatedId isn't included.	Task or linear taskflow ID. Use taskId or taskName in the URI. You can include this task ID when the task is located in the Default folder.
taskFederatedId	String	Required if the task isn't located in the Default folder.	Global unique identifier for the task, which includes the folder path to the task.
taskName	String	Required if taskId or taskFederatedId isn't included.	Task or linear taskflow name. Use taskId or taskName in the URI.
taskType	String	Yes	The type of task. For Data Integration, use one of the following codes: • DMASK. Masking task. • DRS. Replication task. • DSS. Synchronization task. • MTT. Mapping task. • PCS. PowerCenter task. • WORKFLOW. Linear taskflow.
callbackURL	String	No	A valid, publicly available URL. The service posts the job status to the callbackURL.
runtime		No	Attribute that defines runtime properties.
parameterFileName	String	No	Parameter file name.
parameterFileDir	String	No	Parameter file directory on the Secure Agent machine.

75. Click **OK.**

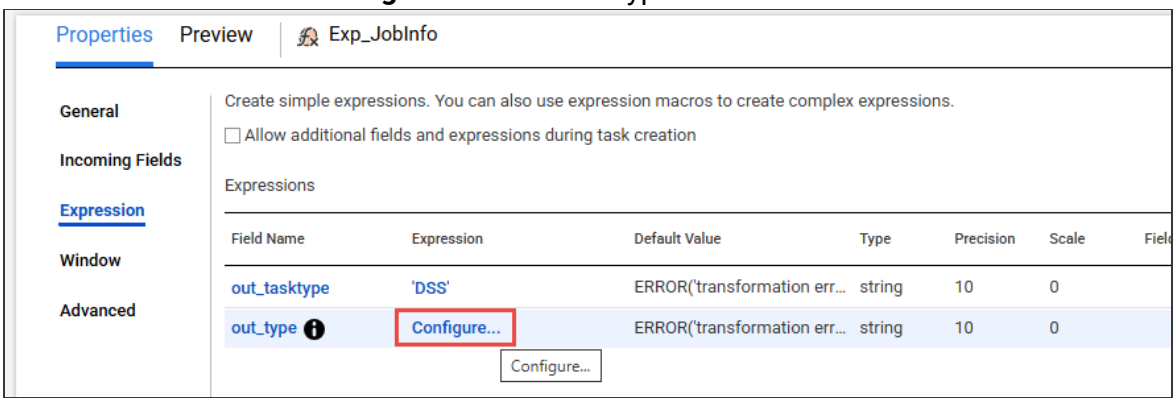


76. On the **Expression** tab add another output field.Set the following properties:

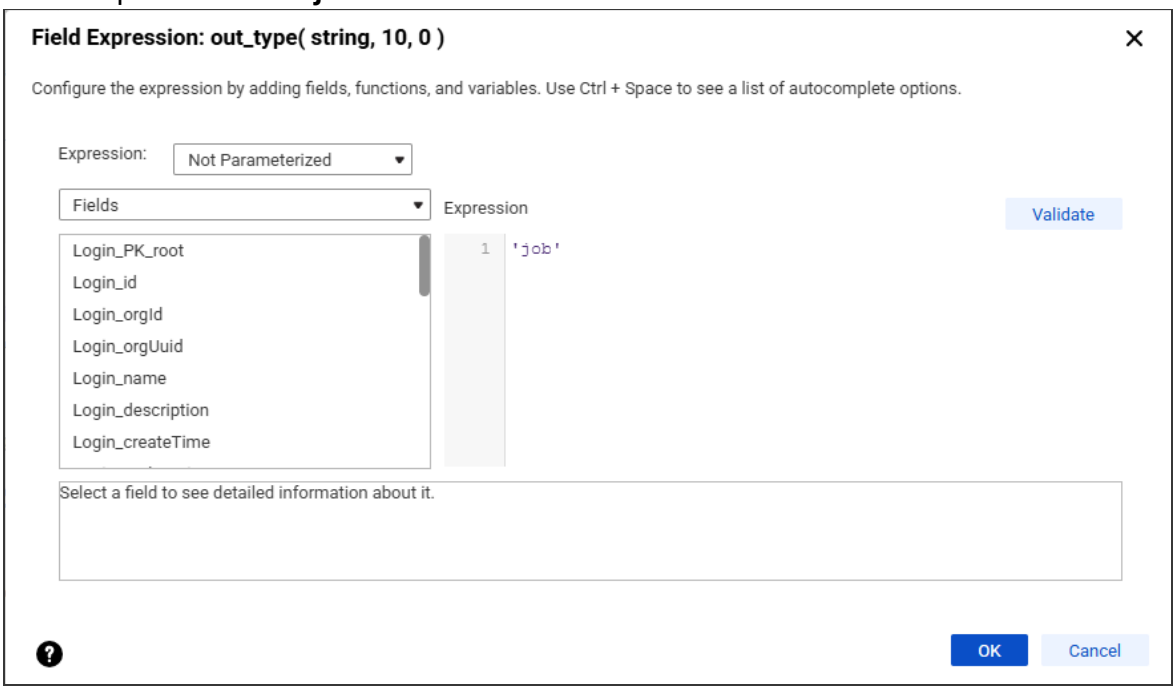
Field Type:	Output Field
Name:	out_type
Type:	String
Precision:	10



77. Click **OK** and then click **Configure** to define the type.



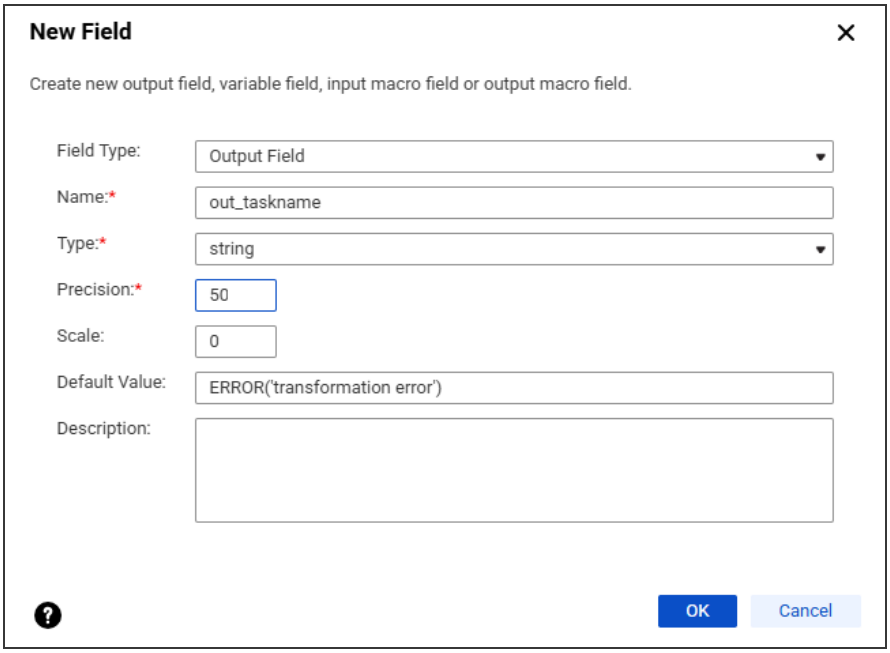
78. In the Expression enter **'job'**



79. Click **OK**.

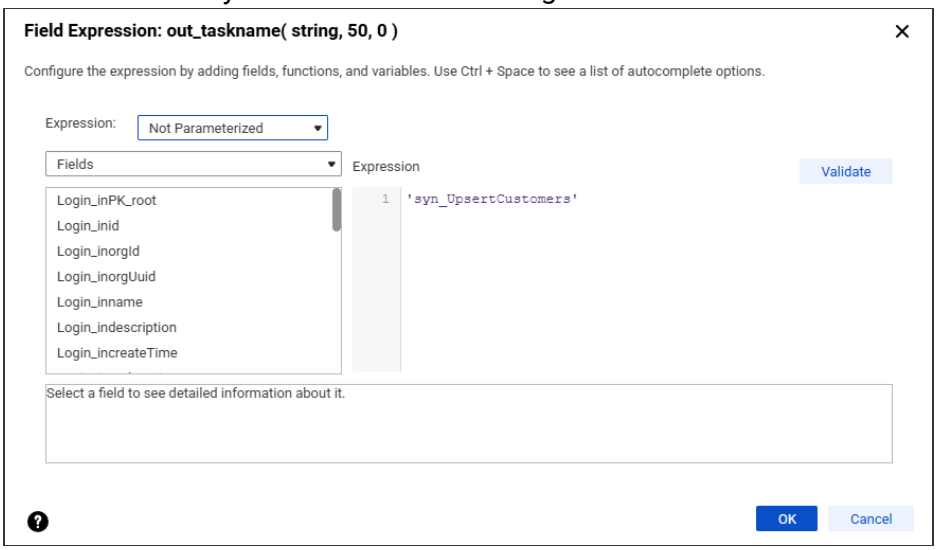
80. On the **Expression** tab add another output field and set the following properties:

Field Type:	Output Field
Name:	out_taskname
Type:	String
Precision:	50

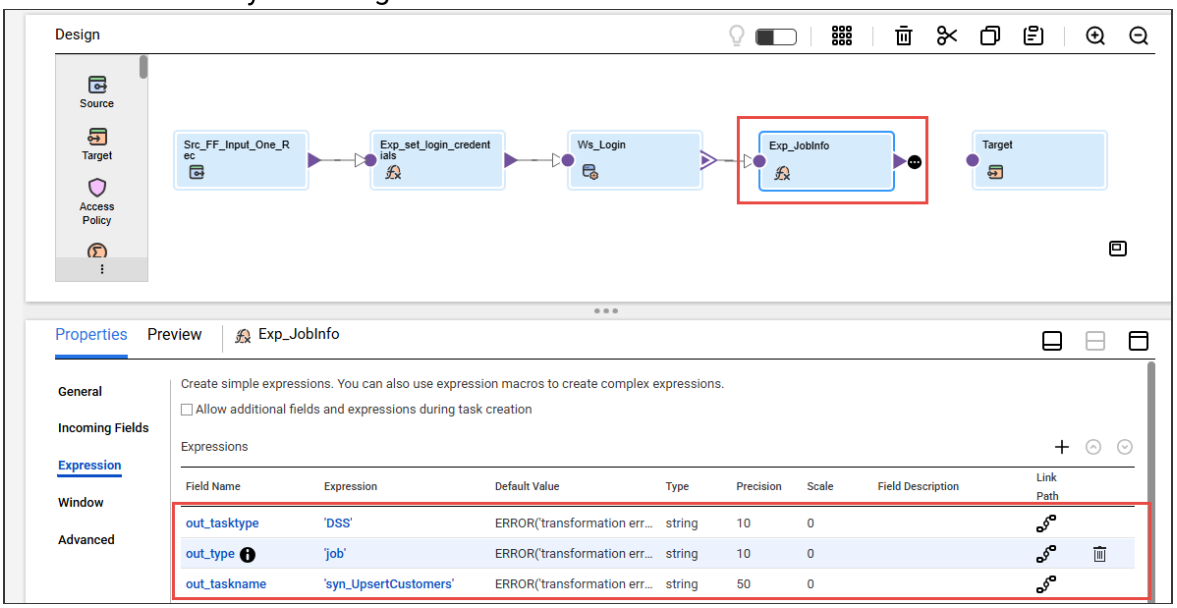


81. Click **OK** and then click **Configure** to define the taskname.

82. Enter the taskname as **'syn_UpsertCustomers'.** Ensure this is the name you have given your task. Alternatively choose another working task.



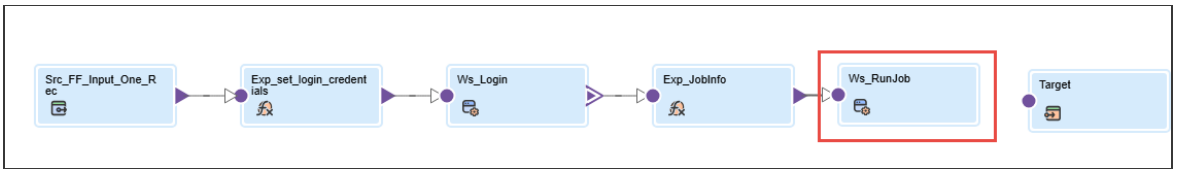
83. Click **OK** and **save** your changes.



Ensure that for your login, there is only 1 task with this name. If multiples exist, make one unique by renaming it and specify this. If you are unsure, click Explore and perform a search for the task.

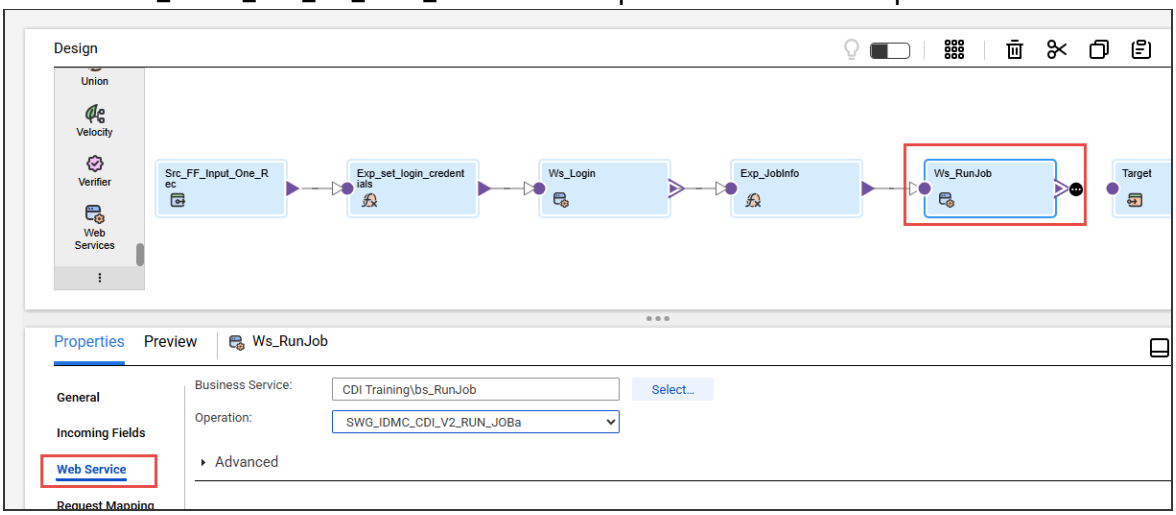
Configure a Web Service Transformation to Use the Defined Business Service and Run the Job Using Information Passed from the Expression Transformation

84. Add another web services transformation to the mapping and set the Name as **Ws_RunJob.**

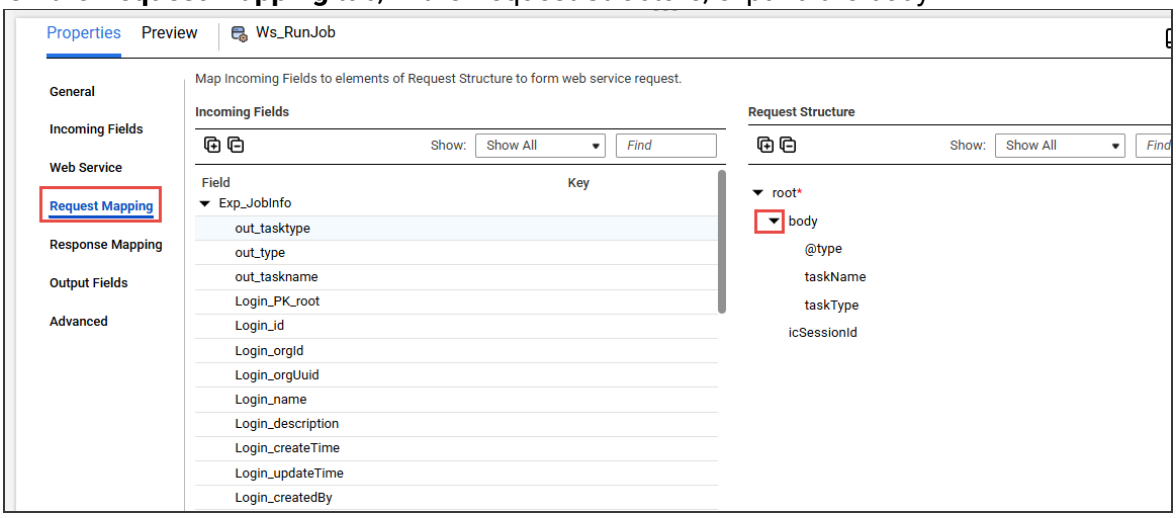


85. On the **Web Service** tab click **Select** to choose the Business Service, and navigate to your folder. Choose the business service we created earlier – **bs_RunJob**.

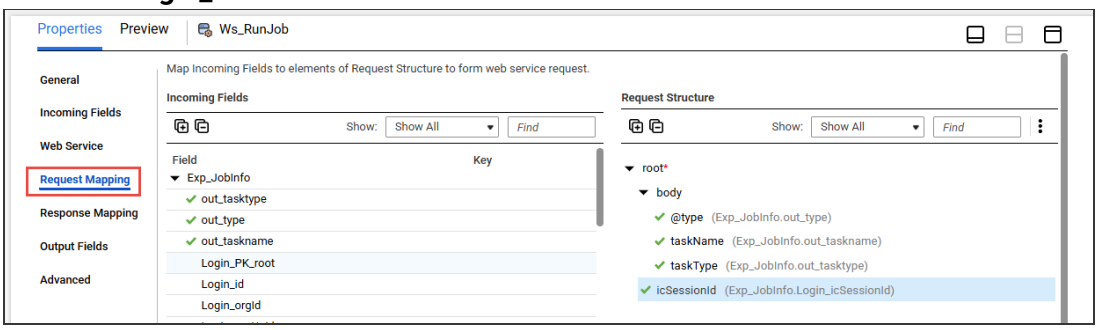
86. Select **SWG_IDMC_CDI_V2_RUN_JOBa** as the operation from the dropdown.



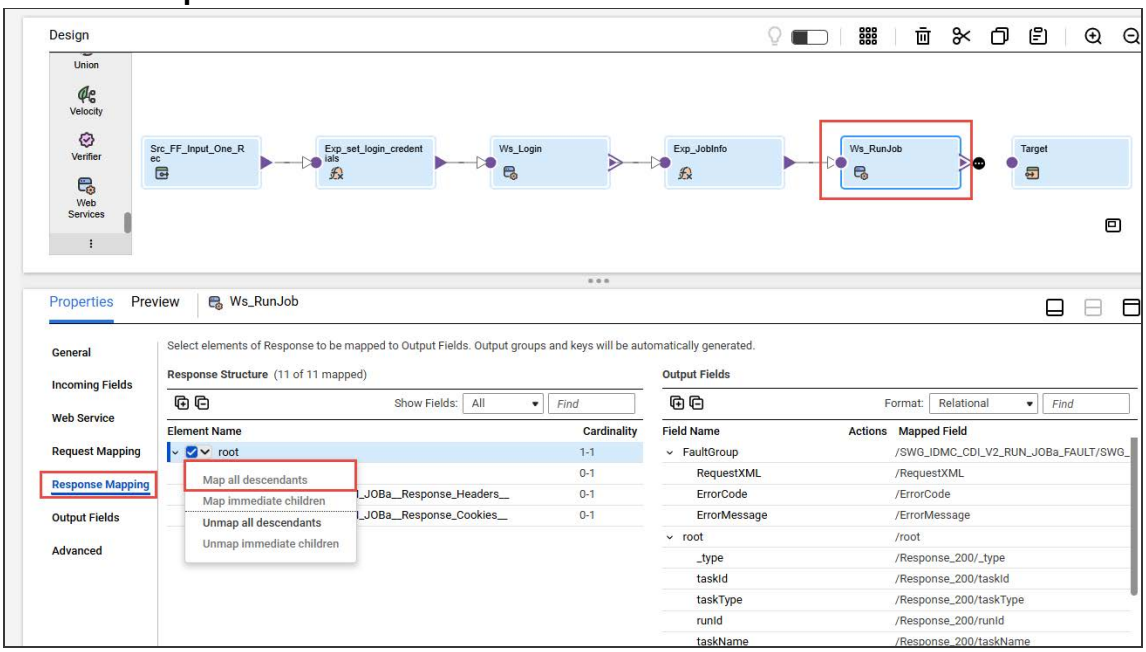
87. On the **Request Mapping** tab, in the Request Structure, expand the body.

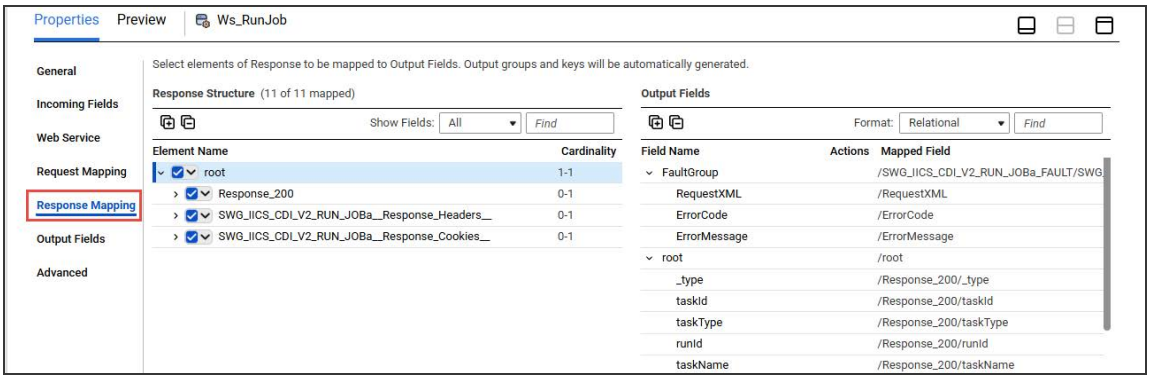


88. Map the following fields.
    - Connect **out_type** to body section → **type**
    - Connect **out_taskType** to body section→ **taskType**
    - Connect **out_taskName** to body section→ **taskName**
    - Connect **Login_icSessionId** to **icSessionId**



89. Move to the **Response Mapping** tab and under element name, click the root dropdown and choose to **Map all descendents**.

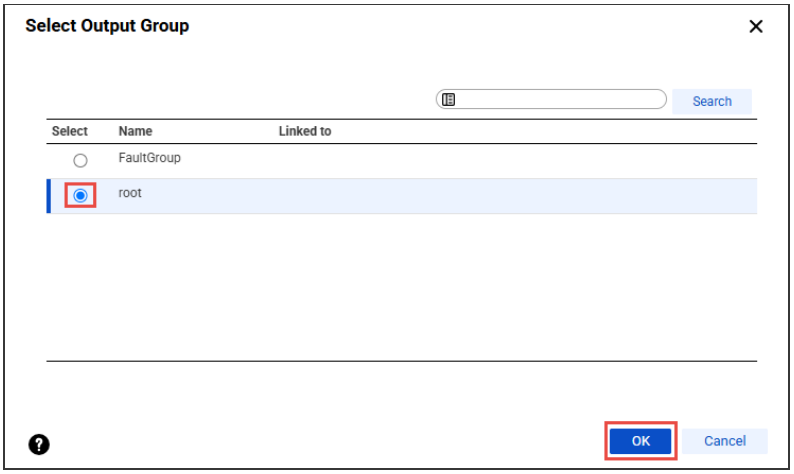




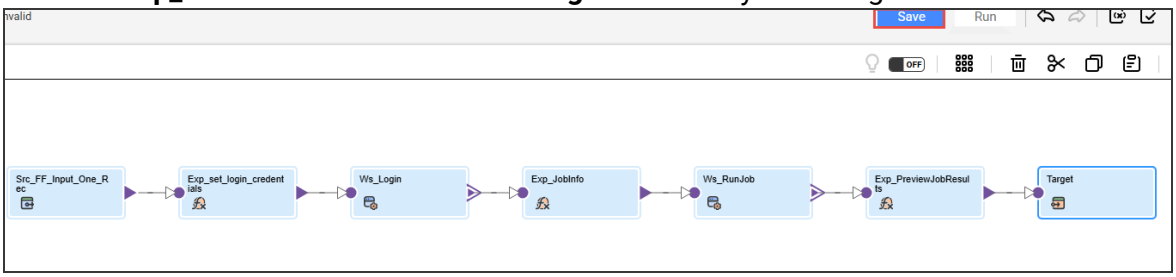
Configure an Expression Transformation to Preview the Output Data

90. Add an Expression transformation to the mapping to preview the output data. Set the name to **Exp_PreviewJobResults.**

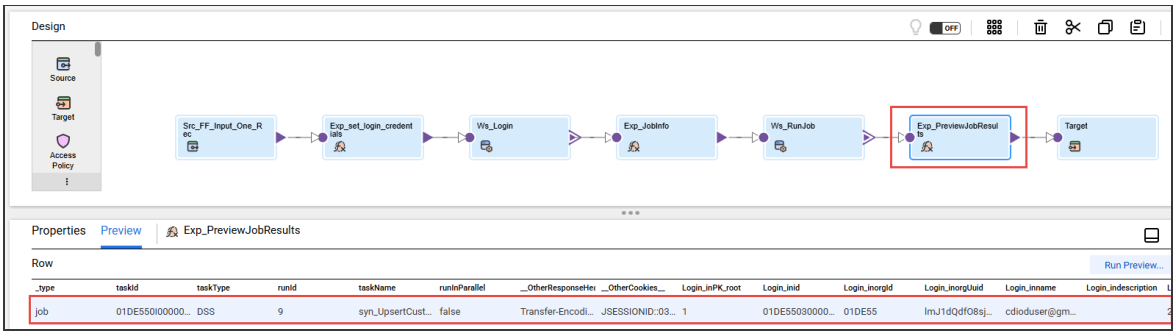
91. Connect **Ws_RunJob** to the **Exp_PreviewJobResults** choosing the output group **root,** and click **OK.**



92. Connect **Exp_PreviewJobResults** to the **Target** and **save** your changes.



93. Selecting the **Exp_PreviewJobResults**, select the **Preview** tab and run a preview of the data.



94. Scroll across and review the data output. This process should login and run a job. The return values should have a **status** and a **runid**. The **runid** should be visible in the preview so we can use that information to verify the job is running.

If this does not preview, check to ensure that the task name is unique.

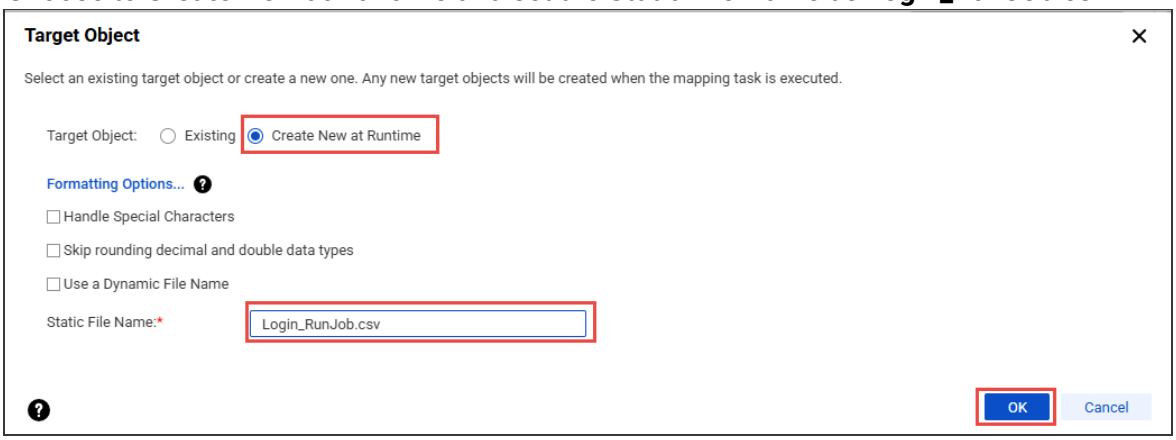
Configure the Target Transformation

95. Set the target name as **Tgt_Login_RunJob**

96. On the **Target** tab from the dropdown select your flatfile connection.

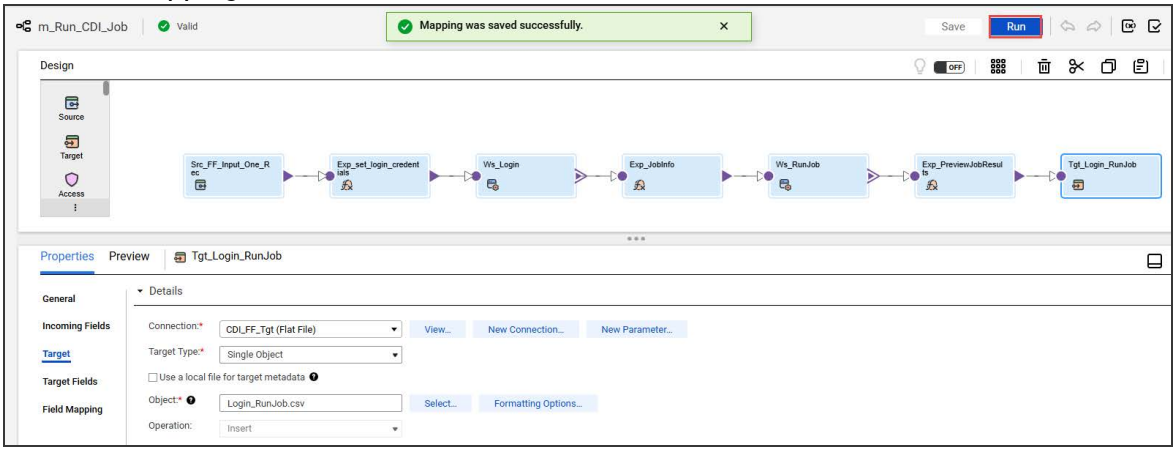
97. Click **Select** to define the output.

98. Choose to **Create New at Runtime** and set the Static File Name as **Login_RunJob.csv**.



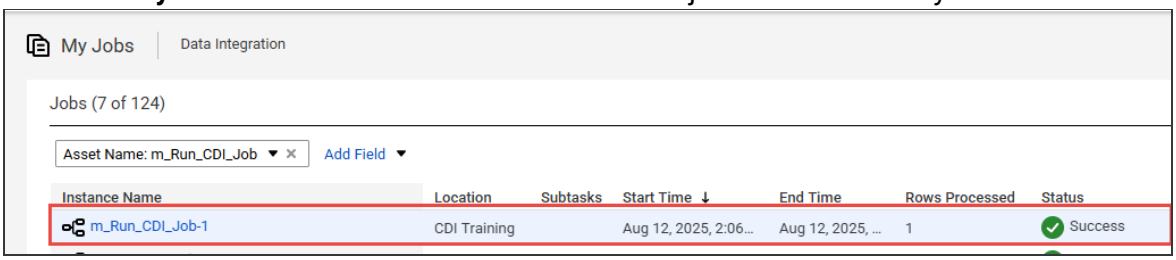
Save and Run the Mapping

99. **Save** the mapping and ensure it is valid.

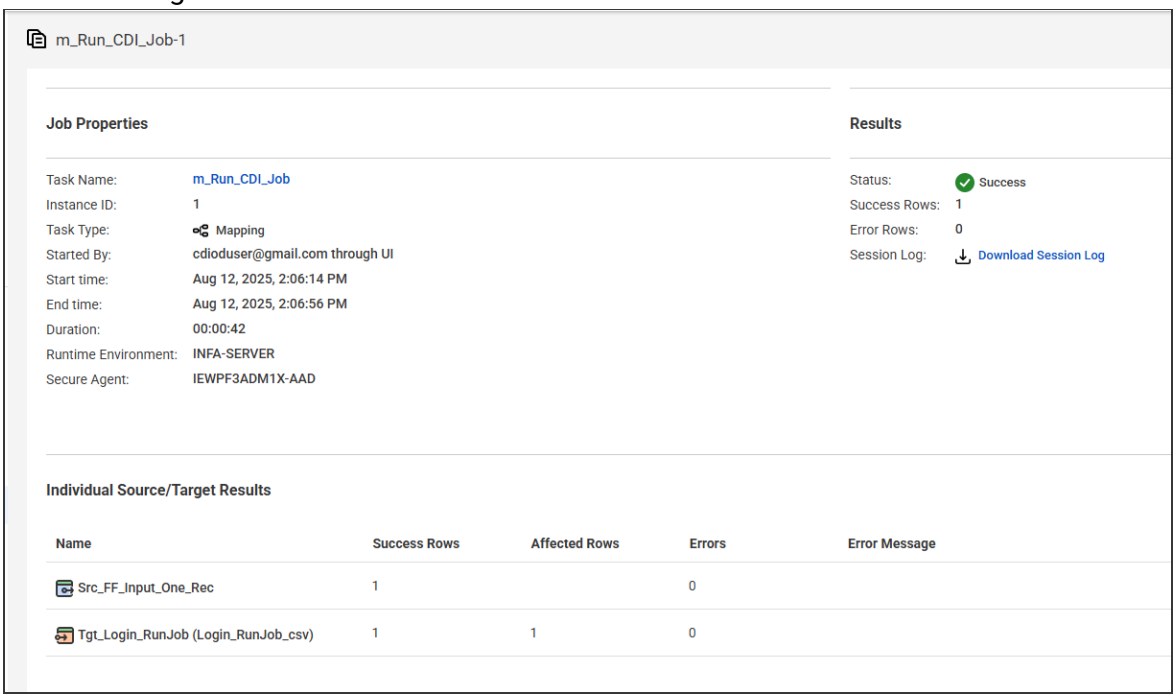


100. **Run** the mapping.

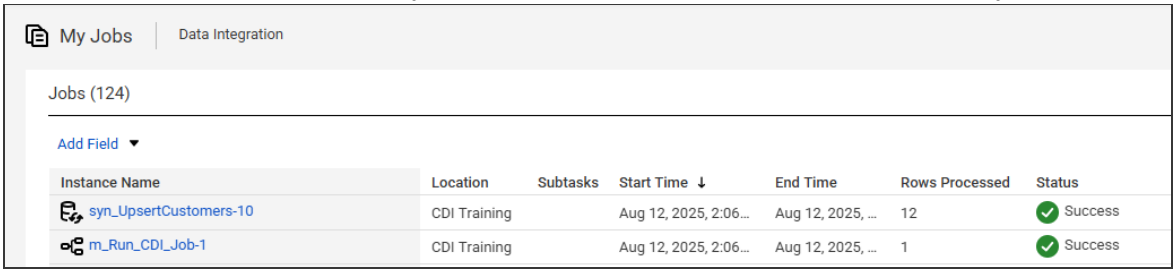
101. Move to **My Jobs** to monitor the run and ensure the job ran successfully.



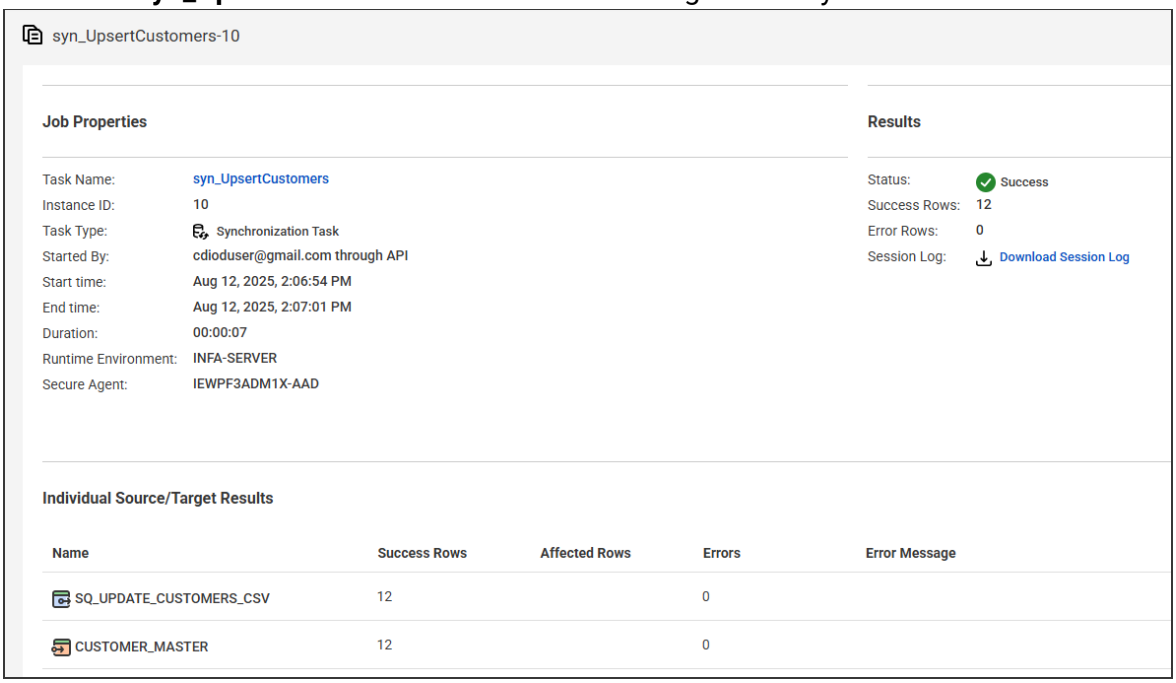
102. Click the **m_Run_CDI_Job-1** instance name to review the details. If required download and review the log.



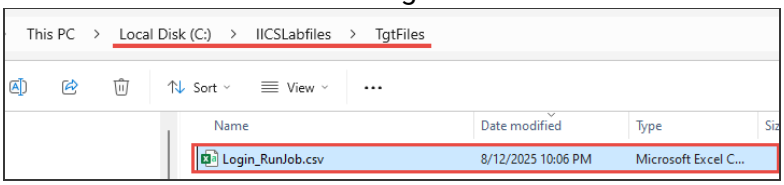
103. Return to the **My Jobs** page and clear the filter (if applied) and refresh the view. You should be able to see that the synchronization task has been run successfully.



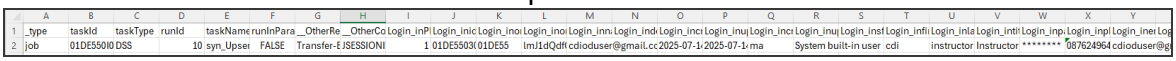
104. Click the **syn_UpsertCustomers** link to review the log and verify the task was run.



105. A file will be created in the target folder.



106. It contains the same data shown in the preview.



107. **Close** any open assets.



There are API calls for saml and other protocols that are likely more secure than a clear text password in an expression transformation.

This concludes the lab.